

MEGAETH · SECURITY REVIEW

MegaETH

EVM Audit.

A security review of EVM Audit.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
MegaETH	EVM Audit	Guardian	April 17 – July 5, 2026	July 13, 2026

§ 00

Contents.

199 pages — overview, confidence ranking, scope, 92 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 09
	Main Review · 66 findings	P. 16
	Remediation Review 1 · 16 findings	P. 149
	Remediation Review 2 · 10 findings	P. 179
§ 05	Disclaimer	P. 198
§ 06	About Guardian	P. 199

§ 01

Overview.

Guardian reviewed EVM Audit across 3 review rounds. The final report includes 15 High, 55 Medium, 22 Low findings.

AUDIT FIRM	REPOSITORY
Guardian	megaeth-labs/mega-evm
CLIENT	METHODOLOGY
MegaETH	Smart contract · Manual · Main Review · Remediation Review 1 · Remediation Review 2
PROJECT	MAIN REVIEW COMMIT
EVM Audit	a8e1a74f8b26
PREPARED BY	REMEDATION REVIEW 1 COMMIT
Guardian	aa4b3db287ae
AUDIT WINDOW	
April 17 – July 5, 2026	
FINAL REPORT	
July 13, 2026	

1.1 Confidence Ranking

1 / 5

Very Low Confidence

Guardian assigns a Confidence Ranking of 1 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.



1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	15	0	15	0	0
Medium	55	0	55	0	0
Low	22	0	22	0	0
Informational	0	0	0	0	0
All severities	92	0	92	0	0

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence ASSIGNED	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed EVM Audit in `megaeth-labs/mega-evm`. The scope includes 11,775 source lines across 86 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope 1/3.

FILE	NSLOC	TOTAL
crates/mega-evm/src/evm/instructions.rs	1,020	1,620
crates/mega-evm/src/evm/execution.rs	646	915
crates/state-test/src/runner.rs	527	652
crates/mega-evm/src/evm/context.rs	419	806
crates/mega-evm/src/limit/limit.rs	403	701
crates/mega-evm/src/evm/mod.rs	393	582
crates/mega-evm/src/block/executor.rs	389	574
crates/mega-evm/src/block/hardfork.rs	376	476
crates/mega-evm/src/block/limit.rs	354	1,023
crates/mega-evm/src/evm/host.rs	348	510
crates/mega-evm/src/sandbox/state.rs	337	516
crates/mega-evm/src/sandbox/execution.rs	328	485
crates/mega-evm/src/block/helpers.rs	293	364
crates/mega-evm/src/system/oracle.rs	286	430
crates/mega-evm/src/evm/state.rs	268	396
crates/mega-evm/src/evm/precompiles.rs	266	326
crates/mega-evm/src/system/intercept.rs	229	359
crates/mega-evm/src/test_utils/inspectors.rs	220	343
crates/mega-evm/src/limit/data_size.rs	185	325
crates/mega-evm/src/block/result.rs	173	240
crates/mega-evm/src/limit/frame_limit.rs	173	247
crates/mega-evm/src/sandbox/error.rs	172	222
crates/mega-evm/src/system/control.rs	170	225
crates/mega-evm/src/test_utils/opcode_gen.rs	159	213
crates/mega-evm/src/block/eips.rs	155	223
crates/mega-evm/src/evm/interfaces.rs	152	199
crates/mega-evm/src/limit/kv_update.rs	150	247
crates/mega-evm/src/evm/spec.rs	147	209
crates/mega-evm/src/external/test_utils.rs	138	262
crates/mega-evm/src/test_utils/database.rs	133	176
crates/mega-evm/src/system/limit_control.rs	131	175
crates/mega-evm/src/evm/result.rs	127	195
crates/mega-evm/src/access/volatile.rs	125	186
crates/mega-evm/src/block/factory.rs	123	197
crates/mega-evm/src/limit/compute_gas.rs	121	232
crates/mega-evm/src/limit/state_growth.rs	120	301

Files in Scope 2/3.

FILE	NSLOC	TOTAL
crates/mega-evm/src/sandbox/tx.rs	115	252
crates/state-test/src/types/transaction.rs	110	157
crates/mega-evm/src/evm/limit.rs	101	143
crates/mega-evm/src/evm/factory.rs	97	191
crates/mega-evm/src/access/tracker.rs	95	226
crates/state-test/src/types/test.rs	89	134
crates/mega-evm/src/limit/storage_call_stipend.rs	86	151
crates/mega-evm/src/limit/mod.rs	85	120
crates/state-test/src/compare.txt	85	85
crates/state-test/src/compare_failed.txt	85	85
crates/mega-evm/src/external/gas.rs	80	117
crates/state-test/src/types/test_unit.rs	79	144
crates/mega-evm/src/external/mod.rs	61	110
crates/state-test/src/main.rs	60	88
crates/state-test/src/types/test_authorization.rs	60	75
crates/mega-evm/src/constants.rs	54	180
crates/mega-evm/src/lib.rs	50	71
crates/mega-evm/src/test_utils/evm.rs	42	45
crates/mega-evm/src/system/keyless_deploy.rs	40	83
crates/state-test/src/types/spec.rs	40	71
crates/state-test/src/types/env.rs	37	70
crates/mega-evm/src/limit/AGENTS.md	31	36
crates/mega-evm/src/block/AGENTS.md	30	35
crates/mega-evm/src/system/tx.rs	30	87
crates/mega-evm/src/AGENTS.md	29	33
crates/mega-evm/src/external/salt.rs	29	111
crates/mega-evm/src/system/AGENTS.md	29	34
crates/mega-evm/src/access/AGENTS.md	27	32
crates/mega-evm/src/external/AGENTS.md	27	32
crates/mega-evm/src/evm/AGENTS.md	26	30
crates/state-test/src/types/deserializer.rs	25	30
crates/system-contracts/src/lib.rs	25	62
crates/state-test/src/types/mod.rs	20	30
crates/mega-evm/src/external/oracle.rs	14	62
crates/state-test/src/types/error.rs	14	21
crates/mega-evm/src/block/mod.rs	13	97

Files in Scope 3/3.

FILE	NSLOC	TOTAL
crates/mega-evm/src/system/mod.rs	12	15
crates/state-test/src/types/account_info.rs	11	17
crates/mega-evm/src/test_utils/mod.rs	10	13
crates/mega-evm/src/external/factory.rs	8	41
crates/mega-evm/src/sandbox/mod.rs	8	83
crates/mega-evm/src/test_utils/bytes.rs	8	10
crates/system-contracts/src/generated/oracle_artifacts.rs	7	19
crates/state-test/src/lib.rs	6	12
crates/mega-evm/src/types.rs	5	13
crates/state-test/src/types/test_suite.rs	5	8
crates/system-contracts/src/generated/access_control_artifacts.rs	5	14
crates/system-contracts/src/generated/keyless_deploy_artifacts.rs	5	14
crates/system-contracts/src/generated/limit_control_artifacts.rs	5	14
crates/mega-evm/src/access/mod.rs	4	12

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	KeylessDeploy bypasses block accounting	Logical Error	HIGH	ACKNOWLEDGED	p. 16
H-02	Keyless sandbox drops commit flags	Logical Error	HIGH	ACKNOWLEDGED	p. 18
H-03	Failed keyless deployments remain replayable	Validation	HIGH	ACKNOWLEDGED	p. 20
H-04	Pre-Block System Calls Can Fail Silently	Logical Error	HIGH	ACKNOWLEDGED	p. 22
H-05	Interceptors copy calldata without accounting	Validation	HIGH	ACKNOWLEDGED	p. 24
H-06	Precompile compute is recorded but not enforced	Logical Error	HIGH	ACKNOWLEDGED	p. 26
H-07	Precomputation can cause a VM crash	DoS	HIGH	ACKNOWLEDGED	p. 27
H-08	System address transactions are replayable	Logical Error	HIGH	ACKNOWLEDGED	p. 29
H-09	SELFDESTRUCT skips account-growth accounting	Logical Error	HIGH	ACKNOWLEDGED	p. 32
H-10	Mega calldata floor is raised after validation	Logical Error	HIGH	ACKNOWLEDGED	p. 34
H-11	Block gas overflow admits deposits	Validation	HIGH	ACKNOWLEDGED	p. 36
H-12	Create commits before revert accounting	Logical Error	HIGH	ACKNOWLEDGED	p. 38

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
H-14	Deposit create charges wrong storage address	Logical Error	HIGH	ACKNOWLEDGED	p. 41
H-15	Deposit mint callers miss creation	Logical Error	HIGH	ACKNOWLEDGED	p. 43
H-16	Relayer forced OOG drains keyless signer	Logical Error	HIGH	ACKNOWLEDGED	p. 45
M-01	Value calls use delegated emptiness	Logical Error	MEDIUM	ACKNOWLEDGED	p. 48
M-02	EIP-7702 auths bypass state growth	Logical Error	MEDIUM	ACKNOWLEDGED	p. 50
M-03	Keyless decoder accepts trailing junk	Validation	MEDIUM	ACKNOWLEDGED	p. 52
M-04	Rex2 Oracle Upgrade Clears Storage	Oracle	MEDIUM	ACKNOWLEDGED	p. 54
M-05	Failed KZG skips compute-gas accounting	Logical Error	MEDIUM	ACKNOWLEDGED	p. 56
M-06	KeylessDeploy Reuses Parent L1 Fee Cache	Logical Error	MEDIUM	ACKNOWLEDGED	p. 58
M-07	7702 Value Transfer Uses Pre-Auth State	Logical Error	MEDIUM	ACKNOWLEDGED	p. 60
M-08	System Interception Bypasses CallTooDeep	Logical Error	MEDIUM	ACKNOWLEDGED	p. 62
M-09	Keyless overhead loses rescued gas	Logical Error	MEDIUM	ACKNOWLEDGED	p. 64
M-10	Beneficiary access guard can be bypassed	Logical Error	MEDIUM	ACKNOWLEDGED	p. 66
M-11	Pre/post execution bypass state growth	Logical Error	MEDIUM	ACKNOWLEDGED	p. 68
M-12	Empty LOG0 spam bypasses block data accounting	Logical Error	MEDIUM	ACKNOWLEDGED	p. 71
M-15	CREATE can be charged for the wrong address	Logical Error	MEDIUM	ACKNOWLEDGED	p. 74

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-16	Create revert loses creator nonce accounting	Logical Error	MEDIUM	ACKNOWLEDGED	p. 75
M-17	Self destruct refund performs an unmetered scan	Validation	MEDIUM	ACKNOWLEDGED	p. 77
M-18	STORAGE_CALL_STIPEND has parts not charged	Logical Error	MEDIUM	ACKNOWLEDGED	p. 79
M-19	Variable opcodes can overrun before enforcement	Logical Error	MEDIUM	ACKNOWLEDGED	p. 81
M-20	wrappers do expensive DB/SALT work very early	Validation	MEDIUM	ACKNOWLEDGED	p. 83
M-21	Deploy not execute under the parent chain config	Logical Error	MEDIUM	ACKNOWLEDGED	p. 86
M-22	7702 Delegated Calls Hide Beneficiary	Logical Error	MEDIUM	ACKNOWLEDGED	p. 88
M-23	Keyless Drops Oracle Volatile Access	Logical Error	MEDIUM	ACKNOWLEDGED	p. 90
M-24	Stale OracleEnv Shadows Oracle Writes	Logical Error	MEDIUM	ACKNOWLEDGED	p. 92
M-25	Skipped 7702 Auths Count As Writes	Logical Error	MEDIUM	ACKNOWLEDGED	p. 94
M-26	7702 auth writes skip beneficiary tracking	Logical Error	MEDIUM	ACKNOWLEDGED	p. 95
M-27	Keyless success can return deleted deploy	Logical Error	MEDIUM	ACKNOWLEDGED	p. 97
M-28	Fee rewards bypass DataSize/KV accounting	Logical Error	MEDIUM	ACKNOWLEDGED	p. 99
M-29	CREATE overcounts balance-only accounts	Logical Error	MEDIUM	ACKNOWLEDGED	p. 101
M-30	Executor can drop tx runtime limits	Logical Error	MEDIUM	ACKNOWLEDGED	p. 103
M-31	Phantom Limits on Failed CALL Init	Logical Error	MEDIUM	ACKNOWLEDGED	p. 105

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-32	Block capacity returns InvalidTx	Validation	MEDIUM	ACKNOWLEDGED	p. 107
M-33	CALLCODE pays impossible creation gas	Validation	MEDIUM	ACKNOWLEDGED	p. 109
M-34	Cross frame refunds bypass growth limits	Logical Error	MEDIUM	ACKNOWLEDGED	p. 111
M-35	EIP7702 authorizations bypass dynamic pricing	Logical Error	MEDIUM	ACKNOWLEDGED	p. 113
M-36	Deposit failures become stateful receipts	Validation	MEDIUM	ACKNOWLEDGED	p. 115
M-37	Implicit CALL returndata copy bypasses metering	Logical Error	MEDIUM	ACKNOWLEDGED	p. 117
M-38	Oracle sendHint exposes unmetered output channel	Validation	MEDIUM	ACKNOWLEDGED	p. 119
M-39	Volatile hash prechecks rewrite stack failures	Logical Error	MEDIUM	ACKNOWLEDGED	p. 121
M-40	Malformed LOG topics expand memory first	Logical Error	MEDIUM	ACKNOWLEDGED	p. 123
M-42	CALL gas operand swallowed during abort	Logical Error	MEDIUM	ACKNOWLEDGED	p. 125
M-44	Create2 validates salt after code hashing	Logical Error	MEDIUM	ACKNOWLEDGED	p. 126
M-46	Deposit nonce creates unmetered accounts	Validation	MEDIUM	ACKNOWLEDGED	p. 128
M-47	Data limit Revert masks create OutOfGas	Logical Error	MEDIUM	ACKNOWLEDGED	p. 130
M-49	Stale external env across blocks	Validation	MEDIUM	ACKNOWLEDGED	p. 131
M-51	Hardfork defaults leave payloads unlimited	Validation	MEDIUM	ACKNOWLEDGED	p. 133
M-52	Pre validation accounting enables DoS	Logical Error	MEDIUM	ACKNOWLEDGED	p. 135

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-53	Frames / opcodes can burn compute that never records	Logical Error	MEDIUM	ACKNOWLEDGED	p. 138
L-01	Invalid transactions can force lookups	Validation	LOW	ACKNOWLEDGED	p. 140
L-02	Tx access set APIs are backed by long live cache	Validation	LOW	ACKNOWLEDGED	p. 142
L-03	Repeated value transferring CALLCODE overcounts	Validation	LOW	ACKNOWLEDGED	p. 144
L-05	Spec changes leave consensus limits stale	Validation	LOW	ACKNOWLEDGED	p. 145
L-06	Same tx SELFDESTRUCT does not refund prior usage	Validation	LOW	ACKNOWLEDGED	p. 147

REMEDIATION REVIEW 1

M-01	Create2 Failing Paths Skip Compute Metering	Logical Error	MEDIUM	ACKNOWLEDGED	p. 149
M-02	Oracle Hints Call Inputs Missed Validation	Validation	MEDIUM	ACKNOWLEDGED	p. 151
M-03	Keyless Gas Cap Enables Signer loss	Logical Error	MEDIUM	ACKNOWLEDGED	p. 153
M-04	Keyless Volatile Access Not Merged	Logical Error	MEDIUM	ACKNOWLEDGED	p. 155
M-05	Keyless Sandbox Gas Excluded From Block Gas	Logical Error	MEDIUM	ACKNOWLEDGED	p. 157
L-01	Failed Deposits Bypass Caller Creation Gas	Validation	LOW	ACKNOWLEDGED	p. 159
L-02	Precompile Stipends Return As Ordinary Gas	Logical Error	LOW	ACKNOWLEDGED	p. 161
L-03	Selfdestruct Meters Before Volatile Guard Runs	Logical Error	LOW	ACKNOWLEDGED	p. 163
L-04	Rex5 Block Validation Can Fail Persistently	Logical Error	LOW	ACKNOWLEDGED	p. 165

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-05	System Transaction Nonce Check Is Stale	Validation	LOW	ACKNOWLEDGED	p. 167
L-06	Per opcode metering does an O(call depth) scan	DoS	LOW	ACKNOWLEDGED	p. 169
L-07	CREATE2 Empty Initcode Observes Offset	Logical Error	LOW	ACKNOWLEDGED	p. 171
L-08	Keyless Empty-Code Drops Logs	Logical Error	LOW	ACKNOWLEDGED	p. 172
L-09	Value Self-Transfers Double Count DataSize/KV	Logical Error	LOW	ACKNOWLEDGED	p. 173
L-10	Staticcall Selfdestruct triggers unmetered work	Validation	LOW	ACKNOWLEDGED	p. 175
L-11	Auth List processing resource exhaustion	Validation	LOW	ACKNOWLEDGED	p. 177

REMEDIATION REVIEW 2

M-01	Oracle multiCall Drops sendHint	Logical Error	MEDIUM	ACKNOWLEDGED	p. 179
M-02	Keyless Empty-Code Replay	Logical Error	MEDIUM	ACKNOWLEDGED	p. 181
M-03	Disabled Child Can Send Oracle Hints	Logical Error	MEDIUM	ACKNOWLEDGED	p. 183
M-04	EIP-7702 authority writes survive limit failure	Validation	MEDIUM	ACKNOWLEDGED	p. 185
L-01	Repeated Recipient Overcounts Limits	Logical Error	LOW	ACKNOWLEDGED	p. 188
L-02	CREATE2 Oversize Pework	Logical Error	LOW	ACKNOWLEDGED	p. 189
L-03	Keyless Exists Read Missing Witness	Logical Error	LOW	ACKNOWLEDGED	p. 190
L-04	L1Block Fee Reads Missing Witness	Logical Error	LOW	ACKNOWLEDGED	p. 192
L-05	Failed deposits evade new account storage gas	Logical Error	LOW	ACKNOWLEDGED	p. 193

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-06	Sandbox overflow drops keyless deploy witnesses	Logical Error	LOW	ACKNOWLEDGED	p. 195

HIGH **H-01**

ACKNOWLEDGED

KeylessDeploy bypasses block accounting

CATEGORY	Logical Error
LOCATION	sandbox/execution.rs, evm/mod.rs, block/limit.rs, block/executor.rs
REVIEW	Main Review

— DESCRIPTION

KeylessDeploy performs the real deployment inside a separate sandbox transaction. That sandbox execution is internally bounded by gasLimitOverride and its own Mega runtime limits, but its resource usage is not reconciled back into the parent transaction outcome.

In sandbox/execution.rs, execute_keyless_deploy_call() charges only the fixed KEYLESS_DEPLOY_OVERHEAD_GAS in the parent Gas object and parent compute-gas tracker, then executes the actual deployment through execute_keyless_deploy_sandbox() with gas_limit = gasLimitOverride. After sandbox execution, apply_sandbox_state() merges the sandbox state into the parent journal and sandbox logs are forwarded into the parent context, but the sandbox's actual gas_used, compute gas, data size usage, KV updates, and state growth are not merged into the parent transaction outcome. evm/mod.rs exports only the parent AdditionalLimit usage, and block/limit.rs and block/executor.rs consume that parent-only outcome for cumulative block gas, receipt gas, and block resource-limit enforcement.

As a result, the chain can commit storage writes, logs, code deployment, and other execution effects produced by the sandbox while canonical transaction and block accounting omit the corresponding sandbox resource usage. A user can therefore cause materially more real execution to be admitted into a transaction and block than receipt gas, cumulative block gas, and multidimensional block counters indicate, weakening tx/block limit enforcement and creating a sequencer or full-node DoS surface through undercounted sandbox execution.

— RECOMMENDATION

Merge sandbox gas_used, compute gas, data size, KV updates, state growth, and log-derived data usage into the parent transaction outcome before receipt construction and block-limit checks. If the sandbox is intended to stay inside the outer transaction envelope, also cap gasLimitOverride by the outer transaction's effective executable budget.

PROOF OF CONCEPT https://github.com/GuardianOrg/mega-evm-team1-1776352187147/blob/c6e9657e54b46708d6dbb349dee004e9dd3d5ec/crates/mega-evm/tests/rex4/keyless_deploy.rs#L270

H-01 KeylessDeploy bypasses block accountingCONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/8>

HIGH **H-02**

ACKNOWLEDGED

Keyless sandbox drops commit flags

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/sandbox/execution.rs, crates/mega-evm/src/evm/state.rs
REVIEW	Main Review

— DESCRIPTION

KeylessDeploy merges sandbox state into the parent journal with account-status merging disabled. In `apply_sandbox_state()`, Mega calls `merge_evm_state_optional_status(&mut journal.state, &sandbox_state, false)`. For accounts that are not already present in the parent journal this preserves the sandbox account status, but for accounts that are already materialized in the parent journal, `merge_account_state()` only copies info and storage while keeping the parent account's old status flags.

This is unsafe when the parent journal already contains an account as a clean/read-only entry, for example through an access list or prior account/storage load. If the keyless sandbox later writes to that account, the modified storage is copied into the parent transaction state but the sandbox Touched/modified status is discarded. Revm database commit logic skips accounts that are not touched, so the sandbox write can be omitted from final committed state even though the keyless deployment transaction returned success.

This creates a real state-transition correctness issue. A keyless constructor can successfully perform a call that writes storage in another account, but if that account was already present in the parent journal without commit-relevant flags, the write can disappear during final commit. This can make constructor-visible execution, returned transaction state, logs, and final committed state disagree.

— RECOMMENDATION

When merging sandbox state, preserve commit-relevant status for accounts whose info or storage changed. At minimum, mark occupied parent accounts as touched when sandbox state changes their info or any storage slot. Preserve parent-owned coldness separately instead of dropping all sandbox status.

PROOF OF CONCEPT https://github.com/GuardianOrg/mega-evm-team1-1776352187147/blob/280c824f8e9adc474f035fc9bf0c1264b0511729/crates/mega-evm/tests/rex2/keyless_deploy.rs#L1382

H-02 Keyless sandbox drops commit flagsCONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/1>

HIGH **H-03**

ACKNOWLEDGED

Failed keyless deployments remain replayable

CATEGORY	Validation
LOCATION	crates/mega-evm/src/sandbox/execution.rs
REVIEW	Main Review

— DESCRIPTION

KeylessDeploy does not consume failed signed deployments in a replay-safe way. Mega uses the recovered signer nonce as the replay barrier, but it accepts signer nonce 0 or 1, while the keyless sandbox executes the signed deployment using nonce-0 semantics so the deployment address always follows the deterministic nonce-0 create path. Constructor failures such as `ExecutionReverted`, `ExecutionHalted`, and `EmptyCodeDeployed` are not surfaced as top-level keyless reverts. Instead, the outer keyless call returns success with encoded error data while signer-side state changes persist, including gas charging and other failure-side effects. As a result, a failed signed deployment can still consume signer funds without consuming the signed authorization in a way that prevents reuse.

After a failed keyless deployment, the signer nonce becomes 1, but the same signed payload remains valid. Replay is accepted because validation still allows nonce ≤ 1 , the sandbox again executes as nonce 0, and if no code exists at the deterministic target address, `ContractAlreadyExists` does not block reuse. This turns one signed keyless deployment payload into a reusable authorization: the signer can be charged for repeated executions even though they only signed once.

A single failed keyless deployment payload can therefore be replayed repeatedly by any party with the calldata, allowing repeated signer-side gas charges and any other persisted failure-side effects without fresh authorization. For value-bearing `EmptyCodeDeployed` failure paths that leave the target address code-less, replay can also repeatedly transfer signer-funded value on each reuse of the same signed payload.

This breaks replay protection and creates a real griefing and signer-drain vector for users or protocols relying on keyless deployment through relayers or other untrusted submission paths.

— RECOMMENDATION

Consume failed keyless deployments in a replay-safe way. The simplest fix is to require the recovered signer nonce to be exactly 0 before keyless execution, but this intentionally removes the current nonce-1 support. If nonce-1 compatibility must be preserved, failed

H-03 Failed keyless deployments remain replayable

CONTINUED

attempts should instead record a separate replay marker or otherwise consume the signer state so the same signed payload cannot be reused after execution begins, even when no code is deployed.

PROOF OF CONCEPT https://github.com/GuardianOrg/mega-evm-team1-1776352187147/blob/59bffa2c97975ebaeabb76e91f68d3a2db0b653d8/crates/mega-evm/tests/rex2/keyless_deploy.rs#L739

— RESOLUTION

Status: Acknowledged.

HIGH **H-04**

ACKNOWLEDGED

Pre-Block System Calls Can Fail Silently

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/block/eips.rs, crates/mega-evm/src/block/executor.rs
REVIEW	Main Review

— DESCRIPTION

Mega executes the mandatory pre-block EIP-2935 blockhash-history and EIP-4788 beacon-root system calls, but does not require those calls to succeed before continuing block execution.

`transact_blockhashes_contract_call()` and `transact_beacon_root_contract_call()` call `evm.transact_system_call(...)` and return the resulting `ResultAndState`.

However, `MegaBlockExecutor::pre_execution_changes()` discards the `ExecutionResult` and keeps only the returned outcome state.

That outcome is later passed to `commit_system_call_outcomes()` without checking whether the system call succeeded.

This is unsafe in Mega because these system calls can fail under Mega-specific metering.

The system-call path still runs through Mega's wrapped execution environment, including `ctx.on_new_tx()`, dynamic storage gas charging, additional-limit accounting, and normal halt/revert behavior.

In particular, the required EIP-2935/EIP-4788 `SSTORE` writes can fail while still using the fixed system-call gas limit.

As a result, a mandatory pre-block protocol update can halt or revert while block execution still proceeds.

The concrete outcome is that required blockhash-history or beacon-root storage updates can be silently omitted instead of making the block invalid.

Blocks can therefore be built or validated without applying mandatory pre-block EIP-2935/EIP-4788 state transitions.

— RECOMMENDATION

After each pre-block `transact_system_call(...)`, inspect the returned `ExecutionResult`.

H-04 Pre-Block System Calls Can Fail Silently**CONTINUED**

If the system call was attempted, reject block execution unless the result is `ExecutionResult::Success`.

PROOF OF CONCEPT https://github.com/GuardianOrg/mega-evm-team1-1776352187147/blob/25729641c379250d59d597a601dbd172964859a6/crates/mega-evm/tests/block_executor/pre_block_system_calls.rs#L87

— **RESOLUTION**

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/3>

HIGH **H-05**

ACKNOWLEDGED

Interceptors copy calldata without accounting

CATEGORY Validation
LOCATION Intercept.rs
REVIEW Main Review

DESCRIPTION

System contract interceptors run during `frame_init`, before normal child frame execution. For `AccessControl` and `LimitControl`, the interceptor always materializes the full call input into owned bytes before checking selectors

```
let input_bytes = call_inputs.input.bytes(ctx);
```

Then it attempts ABI decoding

```
IMegaLimitControl::remainingComputeGasCall::abi_decode(&input_bytes)
```

and for `AccessControl`

```
IMegaAccessControl::disableVolatileDataAccessCall::abi_decode(&input_bytes)
```

```
IMegaAccessControl::enableVolatileDataAccessCall::abi_decode(&input_bytes)
```

```
IMegaAccessControl::isVolatileDataAccessDisabledCall::abi_decode(&input_bytes)
```

When handled, these paths return synthetic results with all child gas remaining

```
InterpreterResult::new(  
  InstructionResult::Return,  
  Bytes::new(),  
  Gas::new(call_inputs.gas_limit),  
)
```

The synthetic result skips normal bytecode execution and normal per-frame compute accounting. The only thing `frame_init` does afterward is push an empty tracking frame for stack alignment.

For internal `CALL` / `STATICCALL`, the EVM call input is a memory range. A client can usually avoid copying the whole range unless the callee actually reads it. These interceptors force a full `input.bytes(ctx)` materialization in host code before deciding whether the selector is valid.

H-05 Interceptors copy calldata without accounting

CONTINUED

A contract can expand memory once, then repeatedly call a system contract with a huge argsSize

1. Put a valid or invalid 4-byte selector at memory offset 0.
2. Expand memory to N bytes once.
3. Loop:

STATICCALL or CALL to LIMIT_CONTROL_ADDRESS

argsOffset = 0

argsSize = N

Each iteration forces, full N-byte host-side copy, ABI decode attempt, synthetic return or cheap fallback path

but the parent mostly pays only the normal CALL-like overhead after memory has already been expanded. The copied bytes are not recorded as compute gas by `compute_gas_ext`, because this work happens in `frame_init`, outside the opcode wrapper.

So an attacker can turn cheap repeated system contract calls into a memory bandwidth and allocation DoS

— RECOMMENDATION

Replace `call_inputs.input.bytes(ctx)` selector probing with a helper that reads only the first 4 bytes from `CallInput::Bytes` or the shared memory slice, then dispatch `AccessControl` and `LimitControl` directly from that selector without ABI decoding owned calldata

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/5>

HIGH **H-06**

ACKNOWLEDGED

Precompile compute is recorded but not enforced

CATEGORY	Logical Error
LOCATION	Precompiles.rs
REVIEW	Main Review

— DESCRIPTION

In evm/precompiles.rs, the custom PrecompilesMap impl does this

```
context.additional_limit.borrow_mut().record_compute_gas(output.gas.spent());
```

But ignores the returned bool, So an expensive precompile can exceed the compute-gas limit and still run to completion. The tx may fail later, but the expensive CPU work already happened

This defeats the compute-gas DoS guard for precompiles, especially after volatile-data detention has lowered the compute budget

A single precompile call can burn far more compute than the current compute cap should permit

— RECOMMENDATION

Before delegating to the inner precompile provider, pass `min(gas_limit, context.additional_limit.borrow().current_call_remaining_compute_gas())` as the precompile gas limit so expensive precompiles fail their own gas precheck before CPU heavy work runs

Normalize the returned `InterpreterResult::gas` back to the original forwarded `gas_limit` with only the actual spent amount consumed, so the compute cap does not burn unrelated EVM gas

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/5>

HIGH **H-07**

ACKNOWLEDGED

Precomputation can cause a VM crash

CATEGORY DoS
LOCATION instructions.rs
REVIEW Main Review

DESCRIPTION

For CREATE2, the wrapper copies and hashes initcode before the real CREATE2 instruction runs

```
let code = Bytes::copy_from_slice(  
context.interpreter.memory.slice_len(initcode_offset, initcode_len).as_ref(),  
);  
let initcode_hash = keccak256(&code);
```

This happens before the wrapped `compute_gas_ext::create2` charges the normal CREATE2 hash cost and performs the normal checks.

A transaction can force extra host-side memory copying and hashing outside the normal gas/compute accounting.

The original instruction will hash the initcode again, so this wrapper duplicates potentially expensive work. If the original instruction later fails for gas or size reasons, the node has still performed the pre hash work.

Also the other Important Impact

A very small contract can execute CREATE2 with `initcode_len > 0` while memory is still empty

The wrapper calls `memory.slice_len(offset, len)` before `revm` expands/checks memory, so `slice_len(0, 1)` is already out of bounds

In debug/test builds, This panics

In release builds, `revm` uses `unreachable_unchecked()` on that OOB path, so it is undefined behavior. In practice this can be a process abort/crash or or worse, but Rust gives no safe guarantee

H-07 Precomputation can cause a VM crashCONTINUED

— RECOMMENDATION

Replace the wrapper around revm implementation with a single MegaEVM CREATE / CREATE2 handler that follows revm order: static / spec / stack checks, max-initcode-size check, initcode metering, CREATE2 hash gas charging, and memory expansion must all run before any memory slice or Keccak

Copy initcode once after those checks, compute the CREATE2 address from that bounded metered buffer for the storage gas lookup, charge storage gas, and only then build CreateInputs and forward child gas

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm/pull/296>

HIGH **H-08**

ACKNOWLEDGED

System address transactions are replayable

CATEGORY Logical Error
LOCATION execution.rs
REVIEW Main Review

DESCRIPTION

before_run() converts any legacy transaction whose recovered sender is MEGA_SYSTEM_ADDRESS and whose callee is whitelisted into an OP-style deposit-like transaction

```
if sent_from_mega_system_address(tx) {  
  if !is_mega_system_transaction(tx) {  
    return Err(...);  
  }  
  tx.deposit.source_hash = MEGA_SYSTEM_TRANSACTION_SOURCE_HASH;  
  tx.base.gas_price = 0;  
}
```

The whitelist check is very weak

```
pub fn check_if_mega_system_transaction(  
  tx_signer: Address,  
  tx_type: u8,  
  tx_kind: TxKind,  
) -> bool {  
  if tx_type == 0x0 && tx_signer == MEGA_SYSTEM_ADDRESS {  
    match tx_kind {  
      TxKind::Create => false,  
      TxKind::Call(address) => MEGA_SYSTEM_TX_WHITELIST.contains(&address),  
    }  
  } else {  
    false  
  }  
}
```

It only verifies

H-08 System address transactions are replayable

CONTINUED

```
legacy tx
sender == MEGA_SYSTEM_ADDRESS
callee in MEGA_SYSTEM_TX_WHITELIST
```

It does not preserve normal nonce replay protection. It also uses a single constant deposit source hash

```
pub const MEGA_SYSTEM_TRANSACTION_SOURCE_HASH: B256 =
b256!("852c082c0faff590c6300c2c34815d1f79882552fa95ba413cd5aeb1dba84957");
```

That constant is not unique per transaction, block, or message. Unlike real OP deposits, there is no external L1 source enforcing uniqueness.

Any previously valid signed legacy transaction from MEGA_SYSTEM_ADDRESS to a whitelisted system contract can be replayed indefinitely

A normal Ethereum transaction is protected by the sender nonce. Here, the nonce is still part of the signed payload, but the code deliberately turns the transaction into a deposit like transaction before validation, so the stale nonce is no longer rejected

A replay works like this

1. A legacy transaction T from MEGA_SYSTEM_ADDRESS to ORACLE_CONTRACT_ADDRESS appears once.
2. Later, attacker resubmits the exact same raw transaction T
3. before_run() sees:
 - recovered sender == MEGA_SYSTEM_ADDRESS
 - tx type == legacy
 - callee == whitelisted oracle
4. The code sets:
 - tx.deposit.source_hash = constant
 - tx.base.gas_price = 0
5. OpHandler treats it as deposit-like and bypasses nonce verification.
6. The same oracle/system call executes again

The impact depends on what the whitelisted oracle call does, but this is consensus critical if the oracle contract has any system sender gated state update path. A stale oracle update can be replayed after the intended block, potentially reverting oracle-visible state

H-08 System address transactions are replayable**CONTINUED**

to old values or repeating privileged maintenance actions. Even if the call is "only" a hint, the transaction becomes a replayable zero price system call

— RECOMMENDATION

We need to stop marking Mega system address legacy transactions as OP deposits, keep them as type-0 transactions so MEGA_SYSTEM_ADDRESS nonce/code validation and the normal nonce bump remain the replay barrier

and Implement the fee free behavior in a narrow Mega specific pre/post-execution branch for whitelisted zero value system calls, skipping only fee / balance / beneficiary / L1 /operator charging and not nonce validation

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/2>

HIGH **H-09**

ACKNOWLEDGED

SELFDESTRUCT skips account-growth accounting

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/evm/host.rs, crates/mega-evm/src/limit/limit.rs, crates/mega-evm/src/limit/data_size.rs, crates/mega-evm/src/limit/kv_update.rs, crates/mega-evm/src/limit/state_growth.rs
REVIEW	Main Review

— DESCRIPTION

Mega does not account for persistent account mutations caused by effective SELFDESTRUCT balance transfers.

When SELFDESTRUCT executes with a nonzero source balance and a different target, revm credits the target account, touches it, and zeroes the source balance where applicable. If the target was empty, this creates a new non-empty balance-only account.

Mega's Host::selfdestruct() only marks beneficiary access and records the Rex4 same-tx-created source-account state-growth refund. Its only resource hook is AdditionalLimit::on_selfdestruct(refund), which forwards only to StateGrowthTracker.

DataSizeTracker and KVUpdateTracker have no SELFDESTRUCT hook. They account account-info updates through normal value CALL, CREATE, and SSTORE paths, but not through SELFDESTRUCT.

StateGrowthTracker also only accounts new accounts through value-transfer frame initialization and CREATE, so it misses growth when SELFDESTRUCT transfers balance into an empty target.

The opcode path is wrapped only with compute-gas accounting, or Rex4 volatile-access plus compute-gas accounting. It is not wrapped with Mega's dynamic new-account storage-gas logic.

As a result, successful SELFDESTRUCT can persist source balance zeroing, target balance credit, target account update, and fresh balance-only target creation while omitting the corresponding DataSize, KVUpdate, StateGrowth, and dynamic account-creation storage-gas usage. Standard SELFDESTRUCT compute gas is charged, but Mega's multidimensional resource accounting is bypassed.

Minimal exploit: a pre-existing contract S has nonzero balance and bytecode SELFDESTRUCT(B), where B is empty. Calling S transfers S.balance to B; final state has S.balance = 0 and B.balance > 0. Mega reports only the normal transaction caller update, not account updates for S or B, nor new account growth for B.

H-09 SELFDESTRUCT skips account-growth accounting

CONTINUED

This weakens transaction and block resource enforcement, allowing account updates and balance-only account creation without consuming intended DataSize, KVUpdate, StateGrowth, or dynamic storage-gas budgets.

— RECOMMENDATION

Add explicit SELFDESTRUCT accounting after the inner selfdestruct result is known and only for effective SELFDESTRUCT effects.

For every effective SELFDESTRUCT balance transfer, account the source and target account-info updates in DataSizeTracker and KVUpdateTracker, with frame-aware discard on revert.

If the target was empty before the transfer and becomes non-empty, record state_growth += 1 and charge dynamic account-creation storage gas for the target bucket.

Keep the existing source refund behavior, but make it compose with target-side growth and account-update accounting.

PROOF OF CONCEPT https://github.com/GuardianOrg/mega-evm-team1-1776352187147/blob/9754693ac5fbb22761d8affa6fcd4626373cedd6/crates/mega-evm/tests/rex4/selfdestruct_state_growth.rs#L343

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/1>, <https://github.com/megaeth-labs/mega-evm-internal/pull/25>

HIGH **H-10**

ACKNOWLEDGED

Mega calldata floor is raised after validation

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/evm/execution.rs
REVIEW	Main Review

— DESCRIPTION

Mega can include and execute non-deposit transactions whose gas limit is below Mega's modified calldata floor gas.

During validation, Mega first calls revm's normal initial/floor gas validation through `validate_initial_tx_gas()`. That validates the inherited Prague/EIP-7623 floor gas against `tx.gas_limit()`.

Mega then applies MiniRex calldata storage pricing by increasing both `initial_and_floor_gas.initial_gas` and `initial_and_floor_gas.floor_gas`.

After this mutation, Mega only rechecks whether the updated initial gas exceeds the transaction gas limit.

It does not recheck whether the updated floor gas exceeds the transaction gas limit.

As a result, a transaction can pass validation with enough gas for the updated initial gas, but less gas than the updated Mega calldata floor.

For example, in MiniRex, a transaction with 100 nonzero calldata bytes and `gas_limit = 50,000` has updated initial gas around 38,600, but updated floor gas around 65,000.

Mega executes the transaction instead of rejecting it.

After execution, the EIP-7623 floor clamp attempts to charge the updated floor, but because the floor is above the transaction gas limit, the transaction simply consumes all provided gas.

The transaction can still produce a receipt and commit effects such as nonce changes, fee transfers, value movement, or other low-gas state changes.

Canonical validation should reject it before execution with `GasFloorMoreThanGasLimit`, producing no receipt and no state transition.

H-10 Mega calldata floor is raised after validation

CONTINUED

The same missing post-mutation recheck applies after Mega adds dynamic top-level account/create storage gas to `initial_gas`.

A transaction below the final Mega intrinsic requirement can reach pre-execution and be included as an OOG receipt instead of being rejected as an invalid transaction.

— RECOMMENDATION

After applying Mega's calldata floor modification, validate the updated floor gas against the transaction gas limit.

If `initial_and_floor_gas.floor_gas > ctx.tx().gas_limit()`, return `InvalidTransaction::GasFloorMoreThanGasLimit` before pre-execution state effects or frame execution.

PROOF OF CONCEPT https://github.com/GuardianOrg/mega-evm-team1-1776352187147/blob/5af058e4ca7a91c24723e09705b24e916dfcf205/crates/mega-evm/tests/mini_rex/gas.rs#L931

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/2>

HIGH **H-11**

ACKNOWLEDGED

Block gas overflow admits deposits

CATEGORY	Validation
LOCATION	crates/mega-evm/src/block/limit.rs, crates/mega-evm/src/block/executor.rs
REVIEW	Main Review

— DESCRIPTION

Mega's block gas admission check uses unchecked u64 addition when deciding whether a transaction fits in the remaining block gas.

BlockLimiter::pre_execution_check() checks remaining block gas by adding block_gas_used + gas_limit and comparing the result against block_gas_limit.

In release builds, this addition wraps on overflow.

After any prior transaction makes block_gas_used nonzero, a deposit transaction with an extreme gas limit such as u64::MAX can wrap the sum below the configured block gas limit.

For example, if block_gas_used == 21_000, then 21_000 + u64::MAX wraps to 20_999, so the admission check can pass even though the deposit's declared gas limit is far above the remaining block gas.

This check is used before execution and again before commit.

Canonical hardfork-derived limits leave tx_gas_limit as u64::MAX while setting the block gas limit from the block header, so the per-transaction Mega gas cap does not stop this case.

For OP deposit transactions, normal transaction environment validation does not provide the same guard as ordinary signed L2 transactions, making Mega's block limiter the effective remaining-gas check.

As a result, Mega can execute, receipt, and commit a deposit whose declared gas limit should have been rejected before execution.

The committed receipt can report cumulative gas above the block gas limit, and the block limiter's cumulative gas counter can exceed or wrap past the configured block gas limit.

Debug builds panic on the overflow, while release builds silently accept the invalid admission path.

H-11 Block gas overflow admits depositsCONTINUED

— RECOMMENDATION

Replace unchecked block gas arithmetic with remaining-gas or checked arithmetic.

Admission should compare the transaction gas limit against `block_gas_limit.saturating_sub(block_gas_used)` instead of adding first.

Post-execution cumulative gas updates should use checked addition and return a block validation error on overflow.

PROOF OF CONCEPT https://github.com/GuardianOrg/mega-evm-team1-1776352187147/blob/e2c4b5fab093d4776f86e226d7e8099665114868/crates/mega-evm/tests/block_executor/block_limit.rs#L963

— RESOLUTION

Status: Acknowledged.

HIGH **H-12**

ACKNOWLEDGED

Create commits before revert accounting

CATEGORY Logical Error
LOCATION process_next_action()
REVIEW Main Review

DESCRIPTION

A successful Create frame can be committed by revm, including deployed code and constructor state changes, and only afterward Mega records extra compute gas from frame action processing

If that late compute gas recording exceeds a Rex4+ frame-local compute budget, Mega mutates the already committed successful Create result into Revert, that creates a split outcome

State: the contract deployment and constructor writes are already committed

EVM result / accounting: the create is treated as failed / reverted, so Mega data, KV, and state growth trackers discard the child frame usage

This is important because a parent contract can finish successfully while the child create returns 0, yet the created contract and its constructor side effects remain in final state without the corresponding create data / KV / state growth accounting

The root cause is that In MegaEvm::frame_run, the ordering is

```
Self::after_frame_run_instructions(context, frame, &mut action)?;  
let gas_remaining_before = match (&action, context.spec.is_enabled(MegaSpecId::MINI_REX)) {  
  (InterpreterAction::Return(interpreter_result), true) => {  
    Some(interpreter_result.gas.remaining())  
  }  
  _ => None,  
};  
let mut frame_output = frame  
  .process_next_action::<_, ContextDbError<Self::Context>>(context, action)  
  ?;  
Self::after_frame_run(context, &mut frame_output, gas_remaining_before)?;
```

H-12 Create commits before revert accounting

CONTINUED

For a create frame, `process_next_action()` calls `revm return_create()`. Upstream `revm` commits the create checkpoint and sets the deployed code inside `return_create()` after the canonical create checks pass.

In the current `revm` source, `process_next_action()` calls `return_create()` for `FrameData::Create`, and `return_create()` commits the checkpoint and then calls `journal.set_code(address, bytecode)`

only after that runs

```
pub(crate) fn after_frame_run(
    &mut self,
    result: &mut FrameResult,
    gas_remaining_before_process_action: Option<u64>,
) {
    if let Some(gas_remaining_before) = gas_remaining_before_process_action {
        let compute_gas_cost = gas_remaining_before.saturating_sub(result.gas().remaining());
        if !self.record_compute_gas(compute_gas_cost) {
            mark_frame_result_as_exceeding_limit(
                result,
                self.exceeding_instruction_result(),
                Default::default(),
            );
        }
    }
    self.try_rescue_gas(result.gas());
}
```

At this point, mutating `result` is too late for create, the child checkpoint may already be committed. For `Rex4+` frame-local compute exceed, `exceeding_instruction_result()` returns `Revert`, so the result is changed from successful `CREATE` to reverted `CREATE` after the state commit

After `after_frame_run()` mutates the successful create result to `Revert`, `frame_return_result()` calls

```
ctx.additional_limit.borrow_mut().before_frame_return_result::<false>(&mut result);
```

The trackers then pop the child frame using

```
self.frame_tracker.pop_frame(result.instruction_result().is_ok());
```

H-12 Create commits before revert accounting

CONTINUED

For DataSizeTracker, KVUpdateTracker, and StateGrowthTracker, a reverted child frame drops discardable usage. But the revm journal state for the CREATE was already committed before Mega changed the result

So the final state can include

- created account
- deployed code
- constructor SSTORE writes
- constructor logs / touched state

while Mega accounting treats the child frame as reverted and discards

- created-account data size
- created-account KV update
- contract bytecode data size
- state-growth +1 for the account
- state-growth for constructor-created slots
- KV/data for constructor SSTOREs and logs

A create that reports failure can still deploy code. and the deployed account, bytecode, and constructor state changes can persist while the data size, KV update, and state growth trackers discard the child frame as reverted

— RECOMMENDATION

we need to not mutate a create result after process_next_action() has run

move any create specific additional limit decision that can change success into the pre commit phase, before return_create() is called

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/3>

HIGH **H-14**

ACKNOWLEDGED

Deposit create charges wrong storage address

CATEGORY Logical Error
LOCATION execution.rs
REVIEW Main Review

DESCRIPTION

In MegaHandler::validate()

```
TxKind::Create => {  
  let caller = tx.caller();  
  let nonce = tx.nonce();  
  let created_address = caller.create(nonce);  
  ctx.create_contract_storage_gas(created_address)  
}
```

For OP style deposit transactions, the transaction does not carry a consensus nonce, it is identified by sourceHash, and nonce checking is skipped, while deposits still execute with normal contract deployment behavior and the from account nonce is incremented for contract creation consistency

So for a deposit contract creation, MegaTransaction::nonce() is not guaranteed to equal the from account current state nonce, the nonce that the real CREATE path uses to derive the deployed address

Mega can therefore charge dynamic contract creation storage gas for

```
caller.create(tx.nonce()) // wrong / synthetic deposit nonce
```

while the EVM actually deploys at

```
caller.create(caller_state_nonce)
```

,

A public L1 deposit can deploy a contract into an expensive SALT bucket while paying the storage gas for a different, cheaper predicted address

RECOMMENDATION

Instead of using tx.nonce() to price top level deposit CREATE

H-14 Deposit create charges wrong storage address**CONTINUED**

for TxKind::Create, load the caller account and derive the created address from the same state nonce used by frame initialization,

or move the Mega storage gas charge into / after create frame initialization where the actual created_address is already known

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/3>

HIGH **H-15**

ACKNOWLEDGED

Deposit mint callers miss creation

CATEGORY Logical Error
LOCATION execution.rs
REVIEW Main Review

DESCRIPTION

Mega only charges top level dynamic new account storage gas for TxKind::Call(address) when

```
!ctx.tx().value().is_zero()
```

That is only the EVM call value

It does not cover OP / Mega deposit minting, where pre_execution() can credit mint to the transaction caller before the EVM frame starts

So a deposit transaction can create a fresh balance bearing caller account with

```
deposit.mint > 0
```

```
tx.value() = 0
```

and Mega will skip new_account_storage_gas(caller) entirely

This is a impactful resource accounting bypass because fresh accounts can be introduced through the forced-deposit path without paying SALT backed dynamic account creation gas and without touching the bucket accounting path that new_account_storage_gas() would normally populate

The exact flow is In MegaHandler::validate()

```
TxKind::Call(address) => {  
  let new_account = !ctx.tx().value().is_zero() &&  
    ctx.db_mut().basic(address)?.is_none_or(|acc| acc.is_empty());  
  let storage_gas =  
    if new_account { ctx.new_account_storage_gas(address) } else { Some(0) };  
  (address, storage_gas)  
}
```

The predicate is correct only for a normal value transfer to the recipient

It misses account creation caused by deposit minting to the caller

The actual balance mutation happens later through the delegated OP pre execution path

H-15 Deposit mint callers miss creation

CONTINUED

```
let eip7702_refund = self.pre_execution(evm)? as i64;
```

That path can credit / mint funds before the first EVM frame

Mega normal account creation pricing hooks are frame based or value recipient based, so this caller creation is outside the dynamic storage gas charging path

This lets forced deposits create fresh L2 accounts without consuming the resource that Mega uses to price account creation

```
new_account_storage_gas(address)
```

— RECOMMENDATION

Add a Mega pre execution accounting hook for deposit mint created accounts

The hook should cover dynamic storage gas, bucket access, state growth, data size, and KV updates consistently

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/5>

HIGH **H-16**

ACKNOWLEDGED

Relayer forced OOG drains keyless signer

CATEGORY Logical Error
LOCATION .
REVIEW Main Review

DESCRIPTION

A public relayer can repeatedly charge a keyless deploy signer by choosing a gasLimitOverride that passes the keyless check but is still below Mega post validation CREATE initial gas

The sandbox then debits the signer for a full failed create, but the create frame never starts, so the signer nonce / deploy address replay barrier is not consumed

In execute_keyless_deploy_call()

```
if gas_limit_override_u64 < tx_gas_limit {  
  return make_error!(KeylessDeployError::GasLimitTooLow { ... });  
}  
  
let sandbox_tx = TxEnv {  
  caller: deploy_signer,  
  kind: TxKind::Create,  
  gas_limit: gas_limit_override_u64,  
  nonce: 0,  
};
```

The relayer only has to provide gasLimitOverride >= signed_tx.gas_limit

But in MegaHandler::validate(), Mega checks initial_gas > tx.gas_limit() before adding the top-level create storage gas, then later adds

```
initial_and_floor_gas.initial_gas += storage_gas.ok_or_else()?;
```

There is no second validation error after that addition. The oversized initial gas is only caught later in before_execution()

```
if tx.gas_limit() < init_and_floor_gas.initial_gas {  
  let oog_frame_result = gen_oog_frame_result(tx.kind(), tx.gas_limit());  
  return Ok(Some(oog_frame_result));  
}
```

H-16 Relay forced OOG drains keyless signer

CONTINUED

That is too late for keyless create. The handler sequence is

```
validate()  
pre_execution()  
execution()
```

So the sandbox signer has already had fees deducted in `pre_execution()` before `before_execution()` synthesizes the OOG result

For create transactions, revm pre execution fee deduction does not bump the sender nonce, it only bumps the nonce for `tx.kind().is_call()`. The current revm source shows `validate_against_state_and_deduct_caller()` sets the caller balance, then only calls `caller.bump_nonce()` inside if `tx.kind().is_call()`; create nonce consumption is deferred to create frame initialization

So if attacker has a valid keyless legacy create transaction signed by the victim, with signed `gas_limit = G` and `gas_price = P`

The attacker calls

```
keylessDeploy(txBytes, G)
```

where $G \geq \text{signed gas_limit}$, so the keyless override check passes

Mega validation adds dynamic CREATE storage gas after the early validation check. If

```
standard_initial_gas + Mega calldata/Rex intrinsic + create_contract_storage_gas > G
```

then `validate()` still returns normally

Sandbox `pre_execution()` deducts the signer gas balance, but does not bump nonce because the sandbox transaction is `TxKind::Create`

`before_execution()` returns a synthetic `OutOfGas` create result before `first_frame_input()` / create frame initialization

No contract is deployed, no deploy address replay marker is created, and the signer nonce remains usable for the same keyless path

```
process_sandbox_transact_result() treats the sandbox halt as a SandboxOutcome::Failure;  
execute_keyless_deploy_call() applies the sandbox state and returns InstructionResult::Return  
with error data, so the outer relayer transaction does not roll back the signer debit.
```

The attacker repeats the same outer call and drains another $G * P$ from the signer

H-16 Relayer forced OOG drains keyless signer

CONTINUED

— RECOMMENDATION

Treat Mega's top-level CREATE/CALL dynamic storage gas as true intrinsic gas and reject before `pre_execution()`

For keyless specifically, also harden the replay barrier: any sandbox attempt that charges the signer must either consume a durable signer nonce/replay marker or be rejected before signer fee deduction

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/2>

MEDIUM **M-01**

ACKNOWLEDGED

Value calls use delegated emptiness

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/limit/state_growth.rs, crates/mega-evm/src/evm/instructions.rs
REVIEW	Main Review

— DESCRIPTION

Mega uses delegated-account inspection when deciding whether a value transfer should be treated as creating a new account. In `StateGrowthTracker::before_frame_init()`, a value `CALL` checks delegated state on `call_inputs.target_address` and records state growth if the delegated account is empty. The storage-gas wrapper applies the same delegated inspection to the raw call target when deciding whether to charge dynamic new-account storage gas.

This is incorrect for an existing EIP-7702 delegator. The recipient account already exists and contains delegation bytecode, so a value transfer to it does not create a new account. However, if its delegate target is empty, Mega can still record state growth and charge new-account storage gas as though the recipient were newly created. That makes state-growth and storage-gas accounting depend on the emptiness of the delegate target rather than the actual recipient account.

The same mistake has a narrower manifestation for `CALLCODE`. `CALLCODE` executes in the caller's account context while borrowing another account's code. The raw stack to address is only the code-source address and is not the created or value-receiving account. If Mega applies the new-account storage-gas check to that raw code-source address, then a value-bearing `CALLCODE` to an empty bytecode address is charged as though it created a new account even though `CALLCODE` cannot create that address. This `CALLCODE` branch affects storage-gas accounting only; the state-growth tracker is not affected because the frame target is the caller/current account.

As a result, Mega can overcount state growth and overcharge new-account storage gas on otherwise valid value-transfer paths. In Mega's multidimensional resource model, this can cause false state-growth usage, inflated storage-gas consumption, premature resource exhaustion, and spurious out-of-gas or resource-limit reverts even though no new account is actually created.

— RECOMMENDATION

Base value-transfer account-creation checks on the raw recipient account's emptiness, not delegated code-account emptiness. For `CALL`, determine whether the recipient is new from

M-01 Value calls use delegated emptiness

CONTINUED

the target account itself before any EIP-7702 delegate follow is used for code execution. For CALLCODE, do not apply new-account storage-gas charging to the raw bytecode address, because that address is only a code source and is not created or value-credited by the call.

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/1>, <https://github.com/megaeth-labs/mega-evm/pull/284>

MEDIUM **M-02**

ACKNOWLEDGED

EIP-7702 auths bypass state growth

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/limit/state_growth.rs, crates/mega-evm/src/limit/data_size.rs, crates/mega-evm/src/limit/kv_update.rs
REVIEW	Main Review

— DESCRIPTION

Mega does not fully account for state growth caused by EIP-7702 authorization processing. At transaction start, Mega records EIP-7702 authority-side effects for data size and KV updates, but StateGrowthTracker does not apply a corresponding pre-execution state-growth charge.

This matters because EIP-7702 authorization processing runs before normal frame execution begins and can materialize fresh authority accounts by installing delegation code and bumping nonce. When that happens, the transaction creates new persistent account state even though no normal CALL, CREATE, or SSTORE path was responsible for it. However, Mega's state_growth accounting only tracks the usual execution-time state-growth paths and omits this authorization-side account materialization path.

As a result, the resulting persistent account growth is omitted from reported state_growth. In Mega's multidimensional resource model, this allows transactions to consume real state-growth capacity without being charged the corresponding state-growth budget, and can let transactions pass configured state-growth limits that should have rejected them. Because per-transaction state-growth usage also feeds block-level enforcement, the same omission can undercount cumulative block state growth and weaken block-level state-growth limits as well.

— RECOMMENDATION

Account for EIP-7702 authorization effects in StateGrowthTracker at transaction start or at the point where the authorization list is applied, parallel to the existing data-size and KV-update accounting. If authorization processing materializes a previously empty authority account or otherwise introduces new persistent account state, that growth should be merged into transaction state-growth usage before limit enforcement and outcome reporting.

— RESOLUTION

Status: Acknowledged.

M-02 EIP-7702 auths bypass state growth

CONTINUED

<https://github.com/megaeth-labs/mega-evm-internal/pull/1>

MEDIUM **M-03**

ACKNOWLEDGED

Keyless decoder accepts trailing junk

CATEGORY	Validation
LOCATION	crates/mega-evm/src/sandbox/tx.rs, crates/mega-evm/src/sandbox/execution.rs
REVIEW	Main Review

— DESCRIPTION

KeylessDeploy accepts signed deployment transactions with trailing bytes that are not part of the signed RLP transaction. In `decode_keyless_tx()`, Mega calls `TxLegacy::rlp_decode_signed(&mut buf)` but does not verify that the decoder consumed the entire input buffer. As a result, `valid_signed_legacy_tx || trailing_bytes` is accepted as the same signed keyless deployment transaction.

This is unsafe because the keyless sandbox later sets `mega_tx.enveloped_tx = Some(tx_bytes.clone())` using the full original byte string supplied to `keylessDeploy`, including any trailing bytes. Mega currently runs with live nonzero OP-style L1 data fee inputs, and `enveloped_tx` is used as the input to that L1 data fee calculation. The appended bytes do not affect signer recovery, deployment address derivation, initcode, or signed transaction semantics, but they can still increase the L1 data fee charged to the signer.

A relayer can append trailing bytes to a valid signed keyless deployment payload. The deployment remains authorized by the original signature and produces the same deployment semantics, but the extra bytes increase fee accounting. The outer relayer also pays for the larger outer transaction, but the recovered keyless signer is additionally charged more inside the sandbox despite not signing those bytes.

— RECOMMENDATION

Require canonical full-input decoding for keyless deployment transactions. After `TxLegacy::rlp_decode_signed(&mut buf)`, reject the input unless `buf.is_empty()`. Alternatively, use a decoder API that enforces complete input consumption. The byte string used for `enveloped_tx` should correspond to the canonical signed transaction bytes, not caller-appended trailing data.

— RESOLUTION

Status: Acknowledged.

M-03 Keyless decoder accepts trailing junk

CONTINUED

<https://github.com/megaeth-labs/mega-evm/pull/269>

MEDIUM **M-04**

ACKNOWLEDGED

Rex2 Oracle Upgrade Clears Storage

CATEGORY	Oracle
LOCATION	crates/mega-evm/src/system/oracle.rs
REVIEW	Main Review

— DESCRIPTION

Mega deploys and upgrades the Oracle system contract during block pre-execution through `transact_deploy_oracle_contract()`.

When Rex2 is active, this function selects the Rex2 Oracle bytecode.

If the Oracle account already exists with the pre-Rex2 bytecode, the function updates the account code hash and bytecode, then marks the returned account as both touched and created.

The returned state is later committed by the block executor through `commit_system_call_outcomes()`.

revm's database commit logic treats `AccountStatus::Created` as a newly created account and clears the account's existing storage.

As a result, upgrading the Oracle contract from the pre-Rex2 bytecode to the Rex2 bytecode can wipe all existing storage at `ORACLE_CONTRACT_ADDRESS`.

This can affect Oracle reads because Mega's Oracle SLOAD path first consults `OracleEnv::get_oracle_storage()`, then falls back to database storage when the oracle environment returns `None`.

The protocol docs also describe Oracle on-chain storage as the persistent fallback used for replay and full-node determinism.

Therefore, any Oracle state expected to persist across Rex2 activation can be lost during the upgrade.

— RECOMMENDATION

Mark the account as Created only when the Oracle account did not previously exist.

For an in-place bytecode upgrade of an existing Oracle account, mark it touched but not created.

M-04 Rex2 Oracle Upgrade Clears Storage

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm/pull/278>

MEDIUM **M-05**

ACKNOWLEDGED

Failed KZG skips compute-gas accounting

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/evm/precompiles.rs
REVIEW	Main Review

— DESCRIPTION

Mega's KZG point-evaluation precompile override applies the fixed Mega KZG cost only on the successful upstream return path. In `kzg_point_evaluation::KZG_POINT_EVALUATION`, the wrapper first rejects calls with less than `GAS_COST`, then calls upstream KZG, and only after a successful return overwrites `output.gas_used` with Mega's fixed KZG cost of 100,000.

This misses invalid proofs that pass the wrapper's gas check and reach upstream verification. A caller can provide a structurally valid 192-byte KZG input with a versioned hash matching the commitment, but with an invalid proof. Upstream KZG then performs proof verification and returns a non-OOG precompile error before the Mega wrapper can overwrite `output.gas_used`.

Mega records precompile compute usage from the returned interpreter result's spent gas. On the non-OOG KZG failure path, the precompile provider returns `PrecompileError` without calling `record_cost(output.gas_used)`, so `gas.spent()` remains zero and the failed verification is omitted from `compute_gas_used`.

The issue is specific to Mega's separate compute-gas resource dimension: failed KZG verification work is not reflected in `compute_gas_used`. A contract can call precompile 0x0A with sufficient gas, provide an invalid proof that reaches verification, ignore the failed CALL result, and continue execution while Mega records no KZG compute cost for that attempt. Repeating this pattern weakens Mega's compute-gas limits and underreports transaction and block compute usage for failed KZG workloads.

— RECOMMENDATION

Charge and record the fixed KZG cost for every KZG invocation that has enough supplied gas and reaches execution, including invalid-proof failures. The fix should preserve the precompile error result while still making Mega's compute-gas accounting include `GAS_COST`.

— RESOLUTION

Status: Acknowledged.

M-05 Failed KZG skips compute-gas accounting

CONTINUED

<https://github.com/megaeth-labs/mega-evm-internal/pull/7>

MEDIUM **M-06**

ACKNOWLEDGED

KeylessDeploy Reuses Parent L1 Fee Cache

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/sandbox/execution.rs
REVIEW	Main Review

— DESCRIPTION

KeylessDeploy executes the signed deployment inside a sandbox transaction and sets the sandbox transaction's enveloped_tx to the signed keyless deployment bytes.

However, the sandbox context is built by cloning the parent L1BlockInfo from the outer relayer transaction.

That L1BlockInfo contains tx_l1_cost, a per-transaction cache populated during parent transaction validation from the outer transaction envelope.

op-revm's calculate_tx_l1_cost() returns the cached tx_l1_cost immediately when it is present, without recalculating from the passed transaction bytes.

As a result, the sandbox can charge the keyless signer using the outer relayer transaction's cached L1 data fee instead of the signed keyless deployment transaction's L1 data fee.

A relayer can submit a valid keyless deployment while padding the outer transaction envelope.

The signed keyless deployment transaction remains unchanged and canonical, but the parent L1 fee cache is inflated.

When the sandbox clones the parent chain info, the keyless signer is charged as though they signed the padded outer envelope.

The outer relayer is also charged for the padded outer transaction, so the padded L1 data cost is effectively charged twice.

This can burn extra ETH from the keyless signer or cause an otherwise sufficiently funded keyless deployment to fail due to an inflated sandbox L1 fee.

— RECOMMENDATION

Do not reuse a parent L1BlockInfo with a populated per-transaction L1 fee cache when constructing the keyless sandbox context.

M-06 KeylessDeploy Reuses Parent L1 Fee Cache

CONTINUED

Before passing chain info into the sandbox, clone the parent chain info and clear tx_l1_cost on the clone.

This forces op-revm to calculate the sandbox L1 data fee from sandbox_tx.enveloped_tx, which is the signed keyless deployment transaction bytes

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

ACKNOWLEDGED

7702 Value Transfer Uses Pre-Auth State

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/evm/execution.rs
REVIEW	Main Review

— DESCRIPTION

Mega computes top-level value-transfer new-account storage gas before EIP-7702 authorization effects are applied.

In `MegaExecution::validate()`, a top-level `TxKind::Call(address)` with nonzero value checks the recipient directly in the database and charges `new_account_storage_gas(address)` if the account is empty.

This runs before `pre_execution()`, where `revm` applies the EIP-7702 authorization list.

For an EIP-7702 transaction where the recipient is also an authority in the authorization list, the recipient can be empty during validation but materialized before normal execution starts.

Authorization processing installs delegation code and bumps the authority nonce.

By the time the value transfer executes, the account has already been materialized by the authorization itself.

As a result, the value-transfer-specific account-creation charge is based on stale pre-authorization state.

Mega can charge dynamic new-account storage gas for the value transfer even though the account was already materialized by EIP-7702 authorization processing before the transfer occurred. This can overcharge storage gas and cause out-of-gas failures for transactions near their gas limit.

— RECOMMENDATION

Compute top-level value-transfer new-account storage gas after EIP-7702 authorization processing, or make validation account for the exact successful authorization effects before checking recipient emptiness.

The value-transfer account-creation check should use the recipient state as it exists when the value transfer is actually executed.

M-07 7702 Value Transfer Uses Pre-Auth State

CONTINUED

Authorization payload bytes should still count as transaction input data, but ACCOUNT_INFO_WRITE_SIZE and the authority KV update should only be charged when the authority account is actually mutated.

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/15>

MEDIUM **M-08**

ACKNOWLEDGED

System Interception Bypasses CallTooDeep

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/evm/execution.rs, crates/mega-evm/src/system/intercept.rs
REVIEW	Main Review

— DESCRIPTION

Mega can return successful synthetic system-contract results at child call depths where normal EVM execution must fail with CallTooDeep.

In revm, the EVM call-depth guard is enforced during normal child frame initialization. When a call opcode creates a new child frame, revm checks the child depth and returns InstructionResult::CallTooDeep if the depth exceeds CALL_STACK_LIMIT.

Mega dispatches system-contract interceptors before that normal child-frame initialization path runs. In MegaEvm::frame_init(), Mega handles FrameInput::Call by calling dispatch_system_contract_interceptors(...). If an interceptor returns a synthetic FrameResult, Mega returns it immediately instead of delegating to the inner revm frame_init() where the depth guard is enforced.

As a result, a call to an intercepted system contract at excessive child depth can succeed synthetically even though an ordinary EVM call at the same depth would fail with CallTooDeep.

This is observable in current interceptors. MegaLimitControl.remainingComputeGas() can return a successful ABI-encoded value at excessive depth. MegaAccessControl.disableVolatileDataAccess() can also return success and mutate the volatile-access tracker at a depth where normal EVM execution should have failed before entering the callee.

This causes parent-visible call success, return data, and control flow to diverge from normal EVM call semantics. For MegaAccessControl, the intercepted call can also change volatile-access state that affects later execution in the same transaction.

— RECOMMENDATION

Apply the same child-depth admission rule before system-contract interceptor dispatch that revm applies during normal frame initialization.

If frame_init.depth > CALL_STACK_LIMIT, Mega should skip system-contract interceptor dispatch and let the normal frame initialization path produce CallTooDeep.

M-08 System Interception Bypasses CallTooDeep

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/5>

MEDIUM **M-09**

ACKNOWLEDGED

Keyless overhead loses rescued gas

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/sandbox/execution.rs, crates/mega-evm/src/limit/limit.rs, crates/mega-evm/src/evm/execution.rs
REVIEW	Main Review

— DESCRIPTION

Mega can charge the full outer keyless call gas limit when the fixed KeylessDeploy overhead pushes the transaction over the tx-level compute-gas limit, instead of rescuing the remaining gas as the normal additional-limit path does.

Mega's additional-limit model rescues remaining frame gas when a tx-level resource limit is exceeded. The rescued amount is stored in `AdditionalLimit::rescued_gas` and later erased from the final spent gas in `MegaEvm::last_frame_result()`.

The KeylessDeploy interceptor has a synthetic halt path that bypasses this lifecycle. In `execute_keyless_deploy_call()`, Mega first charges the fixed `KEYLESS_DEPLOY_OVERHEAD_GAS` from the local call gas. In REX3 and later, it also records that overhead as compute gas. If this pushes total transaction compute gas over the tx-level compute-gas limit, the code enters the tx-global exceeded-limit branch after `frame_local == false` and returns a synthetic `OutOfGas` result.

That synthetic result is built with `Gas::new_spent(gas.limit())`, which marks the entire outer call gas limit as spent. However, the path never calls `AdditionalLimit::try_rescue_gas()` or `rescue_gas()` on the local gas object before returning.

As a result, `rescued_gas` remains zero. When `last_frame_result()` runs, it has no rescued gas to erase from the final result, so the sender is charged as if the full outer keyless call gas limit was consumed, instead of only the fixed overhead plus any legitimately executed work.

This is reachable when pre-overhead transaction compute usage is below the tx compute-gas limit but within 100,000 gas of it, so the fixed KeylessDeploy overhead crosses the cap. The expected behavior is a tx-level additional-limit halt with the remaining outer frame gas rescued and refunded. The actual behavior is an `OutOfGas` result that spends the full outer call gas limit because the synthetic keyless halt skipped the gas-rescue step.

M-09 Keyless overhead loses rescued gas

CONTINUED

— RECOMMENDATION

Before returning the tx-global synthetic halt from `execute_keyless_deploy_call()`, call the normal additional-limit gas rescue path on the current local Gas object so the remaining outer call gas is recorded in `rescued_gas` and erased in `last_frame_result()`.

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/19>

MEDIUM **M-10**

ACKNOWLEDGED

Beneficiary access guard can be bypassed

CATEGORY Logical Error
LOCATION host.rs, instructions.rs
REVIEW Main Review

DESCRIPTION

In REX4/REX5, self destruct is intended to participate in beneficiary volatile access protection.

However, the implementation only checks and marks the SELFDESTRUCT target/beneficiary operand

```
SELFDESTRUCT(source = current execution account, target = stack[0])
```

The code ignores the source account, even though SELFDESTRUCT reads and mutates the source account's balance by transferring it to target.

This means that if the currently executing account is the block beneficiary, SELFDESTRUCT(non_beneficiary_target) can access and drain the beneficiary balance without

1. reverting under disableVolatileDataAccess(),
2. marking BENEFICIARY_BALANCE in VolatileDataAccessTracker,
3. applying the beneficiary compute-gas detention limit.

SELFDESTRUCT is a two-account balance operation:

source.balance -> target.balance

source.balance = 0

So beneficiary volatility exists when either:

source = block.beneficiary or target = block.beneficiary

The current code only handles the second case.

An exploit shape is, block.beneficiary = B

Factory:

M-10 Beneficiary access guard can be bypassed

CONTINUED

1. call MegaAccessControl.disableVolatileDataAccess()
2. create/deploy code at B, or otherwise enter execution at B
3. B executes SELFDESTRUCT(T), where $T \neq B$
4. Factory continues execution

At step 3

```
source = B // beneficiary account
target = T // non-beneficiary
```

The REX4 wrapper checks only `target == beneficiary`, so it does not revert.

The host also marks only `target`, so the volatile tracker never records beneficiary access.

The parent can then continue with undetained compute gas even though beneficiary balance was accessed and mutated inside the disabled volatile access scope

ordinary direct tx/call execution at the beneficiary is already tracked or blocked, so exploitation needs something like the block beneficiary being a deterministic contract created address or otherwise attacker controlled executable beneficiary

— RECOMMENDATION

Replace the generic stack-0 SELFDESTRUCT volatile wrapper with a dedicated REX4+ handler that checks both `context.interpreter.input.target_address()` (source) and `stack[0]` (recipient), reverting under `disableVolatileDataAccess()` if either is the block beneficiary

In `Host::selfdestruct`, mark beneficiary balance access for both address and target before the inner selfdestruct so non disabled source side or recipient side access sets the bitmap and applies detention

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/25>

MEDIUM **M-11**

ACKNOWLEDGED

Pre/post execution bypass state growth

CATEGORY	Logical Error
LOCATION	State_growth_used
REVIEW	Main Review

DESCRIPTION

Additional limit trackers only account for balance caused account growth when the balance movement happens through a normal EVM frame, such as CALL with value or CREATE with endowment

But some balance changes happen outside the frame lifecycle

1. OP deposit-style pre-execution balance minting/increments, handled by the delegated OpHandler::pre_execution.
2. Post-execution beneficiary fee rewards, handled by reward_beneficiary.

Those balance writes occur after on_new_tx() / before_tx_start() and outside AdditionalLimit::before_frame_init()

So StateGrowthTracker never sees them, A fresh account can receive balance and become non-empty without increasing state_growth_used.

This is a state growth limit bypass for Rex+ where BLOCK_STATE_GROWTH_LIMIT is intended to cap new accounts/storage growth

That's the affected flow, if the transaction lifecycle is:

```
self.before_run(evm)?;  
let init_and_floor_gas = self.validate(evm)?;  
let eip7702_refund = self.pre_execution(evm)? as i64;  
let mut exec_result = self.execution(evm, &init_and_floor_gas)?;  
self.post_execution(evm, &mut exec_result, init_and_floor_gas, eip7702_refund)?;
```

Inside before_run()

```
evm.ctx_mut().on_new_tx();
```

Inside on_new_tx()

```
self.additional_limit.borrow_mut().reset();  
self.additional_limit.borrow_mut().before_tx_start(&self.inner.tx);
```

M-11 Pre/post execution bypass state growth

CONTINUED

But StateGrowthTracker has no before_tx_start() implementation

It only records growth in

```
fn before_frame_init()
for
  FrameInput::Call(call_inputs) => {
    if call_inputs.transfers_value() {
      let to_account = journal.inspect_account_delegated(...)?;
      if is_empty {
        self.record_growth(1);
      }
    }
  }
  FrameInput::Create(_) => {
    self.record_growth(1);
  }
```

So only frame level value transfers and creates are counted

The delegated OP pre-execution hook runs later:

```
let eip7702_refund = self.pre_execution(evm)? as i64;
```

Any deposit-style balance mint/increment performed there happens after before_tx_start() and before frame tracking

If it creates a fresh balance-only account, Mega never records +1 state growth

The beneficiary reward has the same accounting blind spot. the balance write there also happens outside AdditionalLimit and is not included in MegaTransactionOutcome.state_growth_used

— RECOMMENDATION

Make the final MegaTransactionOutcome.state_growth_used authoritative by deriving it from the committed transaction EvmState diff after pre_execution, frame execution, and post_execution, comparing every touched account/storage item against its pre-transaction original state.

M-11 Pre/post execution bypass state growth

CONTINUED

Keep the existing frame tracker only as an early in-frame limit guard, but use the final diff-derived growth for transaction/block limit enforcement so deposit mint/nonce writes, beneficiary rewards, fee vaults, EIP-7702, and future non-frame account writes are covered in one place

If that final diff exceeds the transaction state-growth limit, convert the transaction to the normal state-growth-limit failure before committing; BlockLimiter should consume only this corrected final value.

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/5>, <https://github.com/megaeth-labs/mega-evm-internal/pull/20>

MEDIUM **M-12**

ACKNOWLEDGED

Empty LOG0 spam bypasses block data accounting

CATEGORY Logical Error
LOCATION BlockLimiter
REVIEW Main Review

DESCRIPTION

BlockLimiter::post_execution_check() trusts outcome.data_size, but outcome.data_size does not include any per-log overhead

For normal LOG opcodes, Mega only records

```
// DataSizeTracker::after_log
let size = num_topics * LOG_TOPIC_SIZE + data_size;
self.record_discardable(size);
```

So this normal EVM instruction

```
assembly { log0(0, 0) }
```

creates a persisted receipt log, but contributes

```
num_topics = 0
data_size = 0
tracked data size = 0
extra storage gas = 0
```

Then post_execution_check() commits the undercounted value

```
self.post_execution_update_raw(
outcome.result.gas_used(),
outcome.tx_size,
outcome.da_size,
outcome.data_size, // trusted, but misses empty logs
outcome.kv_updates,
outcome.compute_gas_used,
outcome.state_growth_used,
is_deposit,
);
```

M-12 Empty LOG0 spam bypasses block data accounting

CONTINUED

The actual receipt still contains one Log object per LOG0, with at least the emitting address and receipt / list overhead

post_execution_check() never recomputes or validates data usage from outcome.result / receipt logs

A public contract can emit hundreds of thousands or millions of empty logs while the block limiter believes almost no execution data was produced

Example attacker contract

```
contract EmptyLogBomb {
function bomb(uint256 n) external {
for (uint256 i = 0; i < n; ++i) {
assembly {
log0(0, 0)
}
}
}
}
```

Each iteration appends a receipt log.

But Mega's additional data accounting records zero bytes per log, because LOG0 has no topics and zero data

Similar Issue due the same root cause

additional_limit_ext::log records only the declared topic count and memory data length

```
if !additional_limit.on_log(N as u64, len as u64) {
context.interpreter.halt(additional_limit.exceeding_instruction_result());
}
```

AdditionalLimit::on_log() forwards that to the data size tracker

```
pub(crate) fn on_log(&mut self, num_topics: u64, data_size: u64) -> bool {
self.state_growth.after_log(num_topics, data_size);
self.data_size.after_log(num_topics, data_size);
!self.check_limit().exceeded_limit()
}
```

DataSizeTracker::after_log() accounts only topic bytes and data bytes

M-12 Empty LOG0 spam bypasses block data accounting

CONTINUED

```
fn after_log(&mut self, num_topics: u64, data_size: u64) {  
    let size = num_topics * LOG_TOPIC_SIZE + data_size;  
    self.record_discardable(size);  
}
```

So for LOG1(0, 0, topic), Mega counts 32 bytes

But the persisted receipt log must contain at least

```
20byte emitter address + 32-byte topic = 52 bytes
```

That excludes RLP / list / bytes overhead, so the real encoded receipt entry is larger than 52 bytes

— RECOMMENDATION

at the LOG accounting source, not in `post_execution_check`, add a spec-gated `log_receipt_data_size(num_topics, data_len)` helper and make `DataSizeTracker::after_log` record the encoded receipt-log size including address, topic/data RLP headers, and list overhead, so LOG0(0,0) has a 24-byte minimum while reverted logs still use the existing frame-discard path

If storage-gas economics should mirror generated receipt bytes, derive LOG storage gas from the same helper; do not recompute receipt size at block commit because that would miss tx level and frame local enforcement

Charge logs by persisted receipt size, including a per-log base cost. `LOG0(0,0)` must consume nonzero data/storage accounting and still be discardable on revert

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/23>

MEDIUM **M-15**

ACKNOWLEDGED

CREATE can be charged for the wrong address

CATEGORY Logical Error
LOCATION instructions.rs
REVIEW Main Review

— DESCRIPTION

The code computes a CREATE address using the nonce from

```
let Ok(creator) = context.host.inspect_account_delegated(spec, creator_address)
```

For REX4+, `inspect_account_delegated` follows one EIP-7702 delegation hop. But the CREATE address must be based on the executing account's own nonce, not the delegated code account's nonce

For an EIP-7702 delegated account executing CREATE, the wrapper can compute a different `created_address` than `revm` will actually create. Dynamic storage gas is then charged for the wrong bucket/address.

An attacker can choose delegation / nonce conditions that charge a cheaper bucket while creating an expensive bucket address

— RECOMMENDATION

we need to derive the precharged CREATE address from the original executing account exactly as `revm` does, use a non-delegating load / inspect of `creator_address` for the CREATE nonce, and do not inspect the creator at all for CREATE2

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm/pull/304>

MEDIUM **M-16**

ACKNOWLEDGED

Create revert loses creator nonce accounting

CATEGORY Logical Error
LOCATION data_size.rs, kv_update.rs
REVIEW Main Review

DESCRIPTION

CREATE has a special state side effect: the creator contract nonce is bumped before the create-frame checkpoint is made

If the create later fails or reverts, the newly-created account is reverted, but the creator nonce bump is not part of the create checkpoint and can persist if the parent frame succeeds

In revm's create-frame setup, the caller nonce is bumped before `create_account_checkpoint()` is created

DataSizeTracker and KVUpdateTracker currently record the creator/parent account update for `FrameInput::Create` as discardable usage inside the child create frame

```
// data_size.rs / kv_update.rs
FrameInput::Create(_) => {
    let parent_needs_update =
        self.frame_tracker.frame_mut().is_some_and(|entry| !entry.info.target_updated);
    self.push_frame(CallFrameInfo { target_address: None, target_updated: true });
    if parent_needs_update {
        self.record_discardable(ACCOUNT_INFO_WRITE_SIZE); // or +1 KV
    }
}
```

That is wrong for CREATE: the parent nonce bump is child-persistent and should survive child create failure as long as the parent frame succeeds

The `before_frame_run` hook makes this exploitable very cleanly in REX4+: the child create frame can pass `before_frame_init`, revm can bump the creator nonce during `inner.frame_init`, then `after_frame_init_on_frame` adds the created-account update, and only then `before_frame_run` detects the child frame-local Data/KV excess and returns a synthetic Revert

M-16 Create revert loses creator nonce accounting

CONTINUED

When the child frame returns as failed, Mega pops the child tracker frame with `success = false`, so it drops the creator account update together with the reverted created-account update

But the creator nonce bump has already happened and can persist

— RECOMMENDATION

For `FrameInput::Create`, charge the creator nonce update to the current creator frame before pushing the child frame, and mark that frame's target account as updated

If that parent-frame charge exceeds the current frame budget, return a synthetic failed CREATE before `inner.frame_init` runs and push only an empty tracking frame for return-pop alignment

And keep the created-account and deployed-code charges in the child create frame, so child failure drops only child state while the creator nonce charge survives exactly until the creator frame itself reverts.

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/21>

MEDIUM **M-17**

ACKNOWLEDGED

Self destruct refund performs an unmetered scan

CATEGORY Validation
LOCATION host.rs
REVIEW Main Review

DESCRIPTION

In `Host::selfdestruct`, REX4+ computes the state-growth refund for a same-transaction-created account by scanning the entire in-journal storage map

```
let selfdestruct_refund = if self.spec.is_enabled(MegaSpecId::REX4) {  
  let journal = &mut self.inner.journaled_state;  
  inspect_account(journal, address).ok().and_then(|account| {  
    if !account.status.contains(revm::state::AccountStatus::CreatedLocal) {  
      return None;  
    }  
    let slot_count = account  
      .storage  
      .values()  
      .filter(|slot| {  
        slot.original_value().is_zero() && !slot.present_value().is_zero()  
      })  
      .count() as u64;  
    Some(1 + slot_count)  
  })  
  } else {  
    None  
  };
```

The scan is done before the actual `selfdestruct` call

```
let result = self.inner.selfdestruct(address, target);
```

and only afterward does the code check whether the account had already been destroyed

```
if !state_load.data.previously_destroyed {  
  self.additional_limit.borrow_mut().on_selfdestruct(refund);  
}
```

M-17 Self destruct refund performs an unmetered scan

CONTINUED

So even repeated SELFDESTRUCTs that will not receive a refund still rescan the full storage map

The scan also iterates over all cached storage slots, not just written nonzero slots. A contract can fill `account.storage` with many zero-valued read slots using SLOAD; those slots do not contribute to the refund, but they are still scanned.

A public transaction can amplify CPU by making the host repeatedly walk a large storage map that mostly contains zero read slots. With N tens of thousands of cached slots and M repeated delegated selfdestruct attempts, this can become hundreds of millions of slot-iteration operations inside one transaction, while the gas model treats each later attempt as a fixed-cost operation

— RECOMMENDATION

we could move refund eligibility behind the `inner.selfdestruct` result and skip refund work unless REX4+, the account is `CreatedLocal`, and `previously_destroyed == false`

And don't derive the refund by scanning `account.storage`; maintain a frame-aware per-address count of currently refundable zero-to-nonzero storage growth in `StateGrowthTracker::after_sstore`, then consume `1 + count[address]` on the first effective same-transaction SELFDESTRUCT

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-18**

ACKNOWLEDGED

STORAGE_CALL_STIPEND has parts not charged

CATEGORY Logical Error
LOCATION storage_call_stipend.rs
REVIEW Main Review

— DESCRIPTION

In REX4+, value-transferring internal CALL/CALLCODE frames get extra gas

```
call_inputs.gas_limit = call_inputs.gas_limit.saturating_add(stipend);
```

The parent was not charged this extra gas when the CALL instruction executed. On return, the code burns only the unused stipend

```
let burn = gas.remaining().saturating_sub(original_limit);  
gas.record_cost(burn);
```

The used portion of the stipend is never deducted from the parent's gas. The comments say compute gas is capped so the stipend cannot be used for pure computation, but that cap is enforced by AdditionalLimit after opcodes/precompiles execute. A callee can spend stipend gas on a single expensive opcode or precompile before the frame-local compute limit is noticed

Repeated value transferring calls can amplify actual execution beyond the transaction / block gas charged.

The extra gas can be spent before enforcement, especially by memory-expanding opcodes or precompiles. Even for intended storage/log use, the used stipend is real execution resource not reflected in normal parent gas

Example

1. Parent repeatedly performs value-transferring calls with minimal forwarded gas.
2. REX4 adds STORAGE_CALL_STIPEND.
3. Callee spends the stipend on expensive computation or precompile work, then reverts.
4. Unused stipend is burned, but used stipend was never charged to parent gas.

Note: related finding due the same root cause, both come from inflating call_inputs.gas_limit at storage_call_stipend.rs, line 146, and tracking only current frame stipend in rescue_gas() at limit.rs, line 318

M-18 STORAGE_CALL_STIPEND has parts not charged

CONTINUED

STORAGE_CALL_STIPEND is tracked only for the frame that directly receives it. If that frame forwards the stipend-derived gas to a descendant frame, the descendant treats it as ordinary `gas_limit`. On a TX-level additional-limit halt, `rescue_gas()` excludes only the current frame's own stipend, so inherited stipend gas can be added to global `rescued_gas` and later refunded to the sender, the path is

storage_call_stipend.rs: value `CALL` / `CALLCODE` inflates `call_inputs.gas_limit` by `STORAGE_CALL_STIPEND`.

storage_call_stipend.rs: burn only subtracts the current frame's own stipend.

limit.rs: `rescue_gas()` uses only `current_frame_stipend()`, so descendant frames with `0` direct stipend rescue all remaining gas.

execution.rs: top-level `last_frame_result()` refunds global `rescued_gas` via `gas.erase_cost()`

attacker controlled nested value calls can turn system-granted stipend gas into refundable transaction gas after a TX-level limit failure, bypassing the intended "unused stipend is burned" invariant.

This is distinct from simply spending the stipend as child gas; the leak crosses into the global rescued-gas refund path

— RECOMMENDATION

we need to not add storage stipend to ordinary child gas. Track it as a separate storage only allowance, burn unused stipend, and exclude stipend derived gas from refunds / rescue

we need to not add storage stipend to ordinary child gas. Track it as a separate storage-only allowance, burn unused stipend, and exclude stipend-derived gas from refunds/rescue.

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/9>

MEDIUM **M-19**

ACKNOWLEDGED

Variable opcodes can overrun before enforcement

CATEGORY Logical Error
LOCATION instructions.rs, compute_gas.rs
REVIEW Main Review

DESCRIPTION

Compute gas is recorded only after the original opcode has fully executed

```
let gas_before = context.interpreter.gas.remaining();  
run_inner_instruction_or_abort!($original_fn, context);  
let mut gas_used = gas_before.saturating_sub(context.interpreter.gas.remaining());  
compute_gas!(context.interpreter, additional_limit, gas_used);
```

Volatile-data detention is also applied after the volatile opcode executes

```
run_inner_instruction_or_abort!($original_fn, context);  
apply_compute_gas_limit!(context);
```

So after a volatile access lowers the compute cap, the next opcode is still allowed to run to completion before compute-gas enforcement notices the overrun

A single opcode can consume far more than the detained compute budget before the limit is checked.

Example

```
TIMESTAMP // applies volatile compute cap only after this opcode  
huge CALLDATACOPY / CODECOPY / RETURNDATACOPY / MCOPY / RETURN // memory expansion runs fully
```

Expected behavior after volatile access

post volatile computation is capped to BLOCK_ENV_ACCESS_COMPUTE_GAS /
ORACLE_ACCESS_COMPUTE_GAS

Actual behavior

the next variable-cost opcode can consume up to the remaining standard EVM gas

only after that work completes does compute_gas.record_gas_used() detect the exceed

M-19 Variable opcodes can overrun before enforcement

CONTINUED

In REX4+, the same issue weakens per-frame compute guardrails. A child frame can exceed its frame compute budget by one large variable-cost opcode before being reverted.

— RECOMMENDATION

Replace the opcode-specific preflight list with one generic `compute_gas_ext` enforcement mode: before executing any non-frame-producing inner opcode, temporarily cap `interpreter.gas.remaining()` to `min(evm_remaining, current_call_remaining_compute_gas())`, then restore withheld gas after recording the delta

This lets `revm`'s existing `gas_or_fail!` and `resize_memory!` checks reject oversized `KECCAK/COPY/RETURN/LOG/EXP/MLOAD/MSTORE` work before allocation, copying, hashing, or exponentiation, without duplicating opcode gas formulas

Handle `CALL/CREATE` separately around their local pre-frame costs so forwarded gas and storage-gas wrappers still see the original EVM gas; after detention is applied, call `check_limit()` immediately to stop already-over-limit frames

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-20**

ACKNOWLEDGED

wrappers do expensive DB/SALT work very early

CATEGORY Validation
LOCATION instructions.rs
REVIEW Main Review

— DESCRIPTION

Several wrappers do expensive host / account / storage / SALT work before the canonical revm opcode implementation gets a chance to perform its normal gas, static-call, balance, and validity checks

For SSTORE, the outer wrapper loads storage before the actual SSTORE instruction runs

```
let Ok(slot) = context.host.inspect_storage(mega_spec, target_address, index) else {
context.interpreter.halt(InstructionResult::FatalExternalError);
return;
};
```

Then storage_gas_ext::sstore loads it again and may query dynamic storage gas

```
let Ok(slot) = context.host.inspect_storage(mega_spec, target_address, index) else {
context.interpreter.halt(InstructionResult::FatalExternalError);
return;
};
if original_value.is_zero() && present_value.is_zero() && !new_value.is_zero() {
let Some(sstore_set_storage_gas) =
context.host.sstore_set_storage_gas(target_address, index)
else {
context.interpreter.halt(InstructionResult::FatalExternalError);
return;
};
gas!(context.interpreter, sstore_set_storage_gas);
}
```

For call like opcodes, the wrapper unconditionally inspects the target account before it even checks whether value is transferred

M-20 wrappers do expensive DB/SALT work very early

CONTINUED

```
let Ok(to_account) = context.host.inspect_account_delegated(mega_spec, to) else {
context.interpreter.halt(InstructionResult::FatalExternalError);
return;
};
let is_empty = to_account.state_clear_aware_is_empty(spec);
let has_transfer = if $has_transfer_logic {
} else {
false
};
```

This affects zero value CALL, STATICCALL, and DELEGATECALL too, even though those cannot create a new account and do not need dynamic new-account storage gas

A callee with only enough gas to execute a few cheap stack/calldata opcodes can reach one of these wrappers with essentially no gas left. The wrapper then performs database and SALT work before the real opcode fails with OOG or static-call violation

low gas SSTORE pattern

```
child bytecode:
PUSH1 0x01 // new value
PUSH1 0x00
CALLDATALOAD // attacker-chosen slot key
SSTORE
```

Parent repeatedly calls the child with just enough gas for the two pushes and CALLDATALOAD, but not enough gas for SSTORE

Result behavior

```
child reaches SSTORE with near-zero gas
additional_limit_ext::sstore loads storage slot from DB
storage_gas_ext::sstore may query SALT bucket cost
gas!(dynamic_sstore_cost) then fails
child OOGs
parent catches failure and repeats
```

A similar account read DoS exists for call-like opcodes because `wrap_call_with_storage_gas` inspects the target account before knowing whether storage gas is relevant. A low gas STATICCALL / DELEGATECALL / zero value CALL can force account and possibly code / delegation loads before the canonical instruction rejects or runs out of gas

M-20 wrappers do expensive DB/SALT work very early

CONTINUED

— RECOMMENDATION

Replace these pre inspecting wrappers with opcode specific Mega handlers that preserve revm's canonical order: stack / spec / static / memory / balance / depth / address checks first, then exactly one host state load or mutation, then Mega storage / additional limit accounting

Make sure to not query SALT or inspect target storage/accounts on paths that cannot create or mutate state; for CALL/CREATE, defer dynamic new-account/contract gas to frame initialization after canonical transfer / create validity succeeds and fail before child code runs if the charge cannot be paid

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-21**

ACKNOWLEDGED

Deploy not execute under the parent chain config

CATEGORY Logical Error
LOCATION execution.rs
REVIEW Main Review

DESCRIPTION

In sandbox/execution.rs, `execute_keyless_deploy_sandbox()` builds a brand new context with either `MegaContext::new(sandbox_db, mega_spec)`, or

`MegaContext::new_with_shared_ext_envs(sandbox_db, mega_spec)`
and then only copies `block`, `chain` (the `L1BlockInfo` object)

It does not copy the parent `cfg()`

But `MegaContext::new` initializes inner from

```
revm::Context::op().with_db(db).with_cfg(CfgEnv::new_with_spec(spec.into_op_spec()))
```

So the sandbox gets a fresh default config, not the actual execution config of the parent transaction

That means constructor execution inside keyless deploy can see the wrong values/semantics for config-driven behavior, most notably `CHAINID`

The opcode path is normal (`compute_gas_ext::chainid`), so it reads the sandbox `cfg`, not the parent `cfg`

This lets a relay execute a valid signed keyless deployment under the wrong chain/config environment

An example is a constructor that intentionally gates deployment by chain ID

```
constructor() {  
  require(block.chainid == EXPECTED_CHAIN, "wrong chain");  
  // deploy runtime code  
}
```

M-21 Deploy not execute under the parent chain config

CONTINUED

If the parent MegaETH execution config uses chain ID X, but the sandbox default cfg uses Y, then:

a normal deployment on chain X would revert

the keylessDeploy sandbox can still succeed if the constructor condition matches Y

Because `apply_sandbox_state()` then merges the sandbox result back into the parent journal, the unintended deployment becomes canonical

— RECOMMENDATION

clone `ctx.cfg()` before `journal_mut()`, convert it with `into_megaeth_cfg(mega_spec)`, and apply `.with_cfg(parent_cfg)` to both sandbox context construction branches before `transact_raw()`

Because this changes constructor semantics for existing keyless deployments, gate it behind the next spec/hardfork if historical Rex2 Rex5 replay must remain frozen

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-22**

ACKNOWLEDGED

7702 Delegated Calls Hide Beneficiary

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/evm/instructions.rs, crates/mega-evm/src/evm/host.rs
REVIEW	Main Review

— DESCRIPTION

Mega's Rex4 beneficiary volatile-access protection checks the raw CALL-like target address, but does not check the account that revm later resolves through EIP-7702 delegation.

For CALL-like opcodes, Mega checks the stack target against the block beneficiary before the normal call executes. `load_account_delegated()` also marks beneficiary access only for the original address passed into the host call.

This misses delegated execution. An account A can contain EIP-7702 delegation code pointing to the current block beneficiary B. A contract can then call A after volatile data access has been disabled. Mega checks A, sees that it is not the beneficiary, and allows the call. Revm later resolves the delegation and loads the beneficiary account/code path through B, but Mega never marks beneficiary volatile access for the resolved delegated account.

This is not intended under the Rex4 beneficiary volatile model. Beneficiary-targeted account access is supposed to participate in volatile-data tracking and gas detention, including call-frame recipient cases.

As a result, a transaction can access beneficiary code/account metadata through an EIP-7702 delegator while Mega classifies the transaction as non-beneficiary access.

This bypasses both sides of the Rex4 beneficiary volatile-data protection model. `disableVolatileDataAccess()` does not reject the delegated beneficiary access, and gas detention is not applied because the parent transaction never receives the beneficiary access signal.

This breaks the volatile-access invariant used by `disableVolatileDataAccess()`, gas detention, and parallel-execution conflict reduction. Transactions that should be blocked or detained as beneficiary-dependent can execute as ordinary non-volatile calls

— RECOMMENDATION

Check and mark beneficiary access for both the raw CALL-like target and the resolved EIP-7702 delegated account.

M-22 7702 Delegated Calls Hide Beneficiary

CONTINUED

If either address is the block beneficiary and volatile access is disabled, execution should revert before entering the delegated path. If either address is the block beneficiary and volatile access is allowed, Mega should mark beneficiary access and apply the beneficiary detention cap.

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/25>

MEDIUM **M-23**

ACKNOWLEDGED

Keyless Drops Oracle Volatile Access

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/sandbox/execution.rs, crates/mega-evm/src/evm/context.rs, crates/mega-evm/src/evm/host.rs
REVIEW	Main Review

— DESCRIPTION

KeylessDeploy executes the signed deployment inside a fresh sandbox MegaContext.

For Rex4 and later, the sandbox shares the parent external environments, including `oracle_env`. That means Oracle reads performed by constructor code are real block-scoped Oracle reads. However, the sandbox does not share the parent `VolatileDataAccessTracker` or parent `AdditionalLimit`.

When constructor code reads Oracle storage, the Oracle SLOAD marks only the sandbox-local volatile tracker and applies detention only inside the sandbox. After execution, KeylessDeploy merges sandbox state and logs into the parent transaction, but it does not merge the sandbox volatile-access state.

A top-level KeylessDeploy transaction can therefore commit code, storage, or logs derived from Oracle data while the parent transaction still reports no Oracle volatile access.

This means a committed KeylessDeploy transaction can be misclassified as non-Oracle and non-volatile even though its committed effects depend on Oracle storage.

This breaks Mega's transaction-scoped volatile dependency invariant used by gas detention and parallel-execution conflict reasoning. Any scheduler, builder, witness, or conflict layer that trusts the parent transaction's volatile-access metadata can treat an Oracle-dependent KeylessDeploy transaction as independent or non-volatile

— RECOMMENDATION

When applying sandbox results, merge sandbox volatile-access state into the parent context.

If the sandbox accessed Oracle storage, the parent transaction should mark Oracle access and apply the corresponding transaction-level detention state. The same should apply for block-env and beneficiary volatile accesses that affect committed sandbox execution.

M-23 Keyless Drops Oracle Volatile Access

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/8>

MEDIUM **M-24**

ACKNOWLEDGED

Stale OracleEnv Shadows Oracle Writes

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/evm/host.rs, crates/mega-evm/src/external/factory.rs, crates/mega-evm/src/evm/factory.rs, crates/mega-evm/src/block/executor.rs, docs/spec/system-contracts/oracle.md
REVIEW	Main Review

DESCRIPTION

Mega Oracle reads consult the external OracleEnv before reading current journal or database state.

In `MegaContext::sload()`, Oracle storage reads first call `oracle_env.get_oracle_storage(key)`.

If that returns `Some(value)`, Mega returns the external value immediately and does not fall back to the current Oracle contract storage.

This matches the external-first Oracle design, but it creates a consistency gap after an earlier transaction in the same block has already persisted an Oracle update.

The Oracle persistence model relies on `setSlot / setSlots` system transactions making externally sourced Oracle values available on-chain, so later execution can be replayed from committed state.

However, the block-scoped external environment is created once for the block and reused across transaction execution.

Committing a `MEGA_SYSTEM_ADDRESS` Oracle `setSlot` transaction updates the EVM database state, but it does not update, invalidate, or cross-check the block's `oracle_env`.

As a result, if a non-empty OracleEnv is stale or inconsistent after a same-block Oracle write, it can shadow the value that was just committed to the Oracle contract.

An ordered `MEGA_SYSTEM_ADDRESS` `setSlot` transaction can commit Oracle storage in the same EVM/block, yet a later Oracle `SLOAD` for the same slot still returns the stale external value if OracleEnv returns `Some`.

This makes execution depend on external environment freshness rather than only on parent state and the ordered transaction list.

The same block can produce different receipts, logs, return values, or downstream state writes depending on whether execution uses stale non-empty OracleEnv or an empty/replay-style Oracle environment.

M-24 Stale OracleEnv Shadows Oracle Writes

CONTINUED

This affects any contract reading Oracle-backed data, including `HighPrecisionTimestamp.timestamp()`

— RECOMMENDATION

Keep the external-first Oracle model, but require consistency once Oracle storage has been persisted in the current block.

Maintain a per-block Oracle overlay updated whenever committed `setSlot` / `setSlots` system transactions write Oracle storage, and consult that overlay before returning stale external values.

Alternatively, if OracleEnv returns a value for a slot already written earlier in the block, require it to equal the committed value. Validation and replay paths should also avoid using non-empty OracleEnv after persisted Oracle writes are meant to be authoritative.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-25**

ACKNOWLEDGED

Skipped 7702 Auths Count As Writes

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/limit/data_size.rs, crates/mega-evm/src/limit/kv_update.rs
REVIEW	Main Review

— DESCRIPTION

Mega accounts EIP-7702 authority writes before knowing whether revm will actually apply the authorization.

At transaction start, `DataSizeTracker::before_tx_start()` and `KVUpdateTracker::before_tx_start()` charge `ACCOUNT_INFO_WRITE_SIZE` and one KV update for every authorization whose authority can be recovered.

However, upstream EIP-7702 processing can still skip a recovered authorization without mutating the authority account. It skips after additional checks such as chain-id mismatch, `u64::MAX` nonce, invalid authority code state, and on-chain nonce mismatch.

As a result, a transaction can include recoverable-but-skipped authorizations and make Mega count persistent authority writes that never occur. This inflates `data_size` and `kv_updates`, can prematurely hit transaction or block resource limits, and can false reject otherwise valid transactions

— RECOMMENDATION

Only charge authority account write data and KV updates after mirroring the same conditions required for upstream authorization application.

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/15>

MEDIUM **M-26**

ACKNOWLEDGED

7702 auth writes skip beneficiary tracking

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/evm/context.rs, crates/mega-evm/src/evm/execution.rs, crates/mega-evm/src/access/tracker.rs, revm-handler/src/pre_execution.rs
REVIEW	Main Review

— DESCRIPTION

Mega's Rex4 beneficiary volatile-access tracking does not account for EIP-7702 authorization-list writes.

At transaction start, Mega only marks beneficiary volatile access when the raw transaction caller or raw top-level recipient is the block beneficiary. That check runs in `MegaContext::check_tx_beneficiary_access()` during `on_new_tx()`.

After validation, upstream revm applies EIP-7702 authorizations in `pre_execution()`. For every applied authorization, revm loads the authority account, sets or clears EIP-7702 delegation code, increments the authority nonce, and marks the authority account touched. This mutation happens before the first EVM frame and outside Mega's opcode/host volatile-access wrappers.

If a transaction contains an applied authorization where `authority == block.beneficiary`, but `tx.caller != block.beneficiary` and `tx.to != block.beneficiary`, the transaction mutates the beneficiary account without marking beneficiary volatile access and without applying the Rex4 beneficiary detention cap.

As a result, a transaction can write the block beneficiary's account state while Mega classifies it as non-beneficiary and non-volatile.

This breaks the beneficiary volatile-access invariant used for gas detention and parallel-execution conflict reduction. Any scheduler, builder, witness, or conflict layer that trusts Mega's volatile-access metadata can treat the transaction as independent from beneficiary state even though its committed pre-execution effects modify that state.

The missed access happens during EIP-7702 pre-execution authority processing, before any normal opcode wrapper or host call can observe it

— RECOMMENDATION

When applying EIP-7702 authorizations, check each successfully applied authority against the block beneficiary.

M-26 7702 auth writes skip beneficiary tracking

CONTINUED

If `authority == block.beneficiary`, Mega should mark beneficiary volatile access and apply the beneficiary detention cap to the transaction.

The mark should only be applied for authorizations that pass revm's normal application gates and actually mutate the authority account.

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/15>

MEDIUM **M-27**

ACKNOWLEDGED

Keyless success can return deleted deploy

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/sandbox/execution.rs, crates/mega-evm/src/evm/state.rs
REVIEW	Main Review

— DESCRIPTION

KeylessDeploy can return success with a nonzero deployedAddress even though the merged parent-visible state contains no contract code at that address.

process_sandbox_transact_result() treats Output::Create(bytecode, Some(addr)) as success whenever bytecode is non-empty.

But the sandbox-created account can also be marked Created | SelfDestructed.

This can happen when keyless initcode DELEGATECALLs helper code that executes SELFDESTRUCT.

The helper selfdestructs the under-construction deployment account, then control returns to initcode, which can still return non-empty runtime bytecode.

KeylessDeploy therefore returns deployedAddress = addr and empty errorData.

During apply_sandbox_state(), merge_evm_state_optional_status() treats Created | SelfDestructed accounts as empty and replaces them with an empty touched account.

The final state has no code at addr.

This breaks the result invariant that nonzero deployedAddress with empty errorData means non-empty contract code was deployed.

Deployment workflows can record or act on an address that is empty after commit

— RECOMMENDATION

Before returning success, verify that the sandbox deployment account was not selfdestructed and that the parent-visible account at deployedAddress has non-empty code.

Otherwise return EmptyCodeDeployed with deployedAddress = 0.

M-27 Keyless success can return deleted deploy

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/19>

MEDIUM **M-28**

ACKNOWLEDGED

Fee rewards bypass DataSize/KV accounting

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/evm/execution.rs, crates/mega-evm/src/evm/mod.rs, crates/mega-evm/src/limit/data_size.rs, crates/mega-evm/src/limit/kv_update.rs, crates/mega-evm/src/block/limit.rs, op-revm/src/handler.rs
REVIEW	Main Review

— DESCRIPTION

Mega does not account post-execution fee-recipient balance updates in DataSizeTracker or KVUpdateTracker.

During normal non-deposit transaction execution, Mega finalizes AdditionalLimit frame accounting in `last_frame_result()`.

After that, OP post-execution fee settlement runs `reward_beneficiary()`.

That path increments balances for the block beneficiary and OP fee recipients, including the base-fee vault, L1-fee vault, and operator-fee vault.

Those balance increments are journaled account updates and can be included in the final EvmState committed for the transaction.

However, MegaTransactionOutcome exports `data_size` and `kv_updates` only from `additional_limit.get_usage()`.

DataSourceTracker accounts tx-start caller/auth updates plus in-frame CALL, CREATE, SSTORE, and log effects.

KVUpdateTracker accounts tx-start caller/auth updates plus in-frame CALL, CREATE, and SSTORE effects.

Neither tracker accounts post-execution fee-recipient balance mutations.

As a result, an executed non-deposit transaction can commit fee-recipient account writes while MegaTransactionOutcome.`data_size` and `kv_updates` omit those writes.

This can apply even when EVM execution reverts or returns an execution halt, as long as the transaction reaches post-execution fee settlement.

BlockLimiter then consumes the understated counters and under-accumulates block DataSource/KVUpdate usage.

This weakens DataSource and KVUpdate block-limit accuracy across normal fee-paying transactions.

M-28 Fee rewards bypass DataSize/KV accounting

CONTINUED

— RECOMMENDATION

Account post-execution fee-recipient balance updates before exporting MegaTransactionOutcome and before BlockLimiter consumes it.

For each fee recipient whose account info actually changes, charge one account-info write in DataSize and one KV update.

Alternatively, derive DataSize/KV usage from the final committed state diff.

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/20>

MEDIUM **M-29**

ACKNOWLEDGED

CREATE overcounts balance-only accounts

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/limit/state_growth.rs
REVIEW	Main Review

— DESCRIPTION

Mega records state growth for every CREATE frame even when the created address already exists as a balance-only account.

State growth is defined as the net increase in on-chain state caused by new accounts and new storage slots.

Deploying code to an existing balance-only account should not increase account-count state growth because the account already exists in state.

However, `StateGrowthTracker::before_frame_init()` unconditionally records one unit of state growth for every `FrameInput::Create(_)`.

This is incorrect for counterfactual deployments.

A CREATE or CREATE2 destination can be prefunded before deployment, leaving an account with nonzero balance, nonce zero, and empty code.

Revm permits contract creation at that address because create collision checks reject non-empty code or nonzero nonce, but do not reject nonzero balance.

The deployment mutates the existing account by installing nonce and code, but it does not create a new account entry.

Mega still records one unit of state growth.

As a result, Rex-family state-growth usage is overstated.

A valid deployment to a prefunded balance-only address can be falsely rejected by a transaction state-growth limit, and block state-growth capacity can be prematurely exhausted even though no new account was added.

— RECOMMENDATION

For CREATE and CREATE2, derive the created address and inspect its pre-create state before recording account state growth.

M-29 CREATE overcounts balance-only accounts

CONTINUED

Only increment state growth when the target account does not already exist as an account entry.

If the target is balance-only, charge the appropriate account and code update costs, but do not count new account growth.

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/21>

MEDIUM **M-30**

ACKNOWLEDGED

Executor can drop tx runtime limits

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/block/executor.rs, crates/mega-evm/src/block/factory.rs, crates/mega-evm/src/evm/factory.rs
REVIEW	Main Review

— DESCRIPTION

Some block-executor construction paths install `MegaBlockExecutionCtx.block_limits` into the block limiter, but not into the EVM's per-transaction runtime limits.

The inherent `MegaBlockExecutorFactory::create_executor()` path converts the block context limits into EVM runtime limits with `block_ctx.block_limits.to_evm_tx_runtime_limits()` and applies them through `with_tx_runtime_limits(runtime_limits)` before constructing the executor.

However, the standard `alloy_evm::block::BlockExecutorFactory` implementation receives a prebuilt EVM and directly calls `MegaBlockExecutor::new(evm, ctx, ...)`.

`MegaBlockExecutor::new()` initializes `block_limiter` from `ctx.block_limits.to_block_limiter()`, but it does not synchronize the corresponding transaction runtime limits into the EVM.

As a result, callers using the Alloy block-executor factory path, or direct `MegaBlockExecutor::new()`, can provide tightened transaction limits in `MegaBlockExecutionCtx.block_limits` while the EVM still executes with its existing runtime limits.

The block limiter observes the configured block-level limits, but transaction execution may not enforce the configured compute, data-size, KV-update, state-growth, or detention caps.

A transaction that should halt under the supplied block context can therefore execute successfully and commit when built through one executor path, while the inherent factory path enforces the intended limits.

— RECOMMENDATION

Ensure every `MegaBlockExecutor` construction path applies `ctx.block_limits.to_evm_tx_runtime_limits()` to the EVM before execution.

A fix is to make `MegaBlockExecutor::new()` responsible for synchronizing the EVM runtime limits from `ctx.block_limits`, so all factory paths share the same behavior.

M-30 Executor can drop tx runtime limits

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/6>

MEDIUM **M-31**

ACKNOWLEDGED

Phantom Limits on Failed CALL Init

CATEGORY	Logical Error
LOCATION	crates/mega-evm/src/limit/limit.rs, crates/mega-evm/src/limit/data_size.rs, crates/mega-evm/src/limit/kv_update.rs, crates/mega-evm/src/limit/state_growth.rs, crates/mega-evm/src/evm/execution.rs
REVIEW	Main Review

— DESCRIPTION

Mega can enforce DataSize, KV-update, and StateGrowth limits for value-transfer CALL effects before revm proves that the child call can actually commit any value-transfer effect.

MegaEvm::frame_init() calls AdditionalLimit::before_frame_init() before delegating to revm frame initialization.

Inside that hook, Mega pushes child tracker frames and records value-transfer CALL account-update and account-growth usage from the pending FrameInput::Call.

Only after this does revm run frame-admission checks such as call-depth rejection and value-transfer balance checks.

If the call is too deep, or if the caller lacks enough balance for the requested value transfer, revm returns a normal failed CALL result without executing child code, moving value, updating accounts, or creating an empty target account.

Mega can instead trip DataSize, KV-update, or StateGrowth limits on the pre-recorded phantom usage before revm rejects the frame.

As a result, a normal failed CALL result can be replaced by a Mega resource-limit failure.

In Rex4+, the parent can observe MegaLimitExceeded revert data instead of the normal failed CALL result.

In earlier specs, the transaction can halt due to resource usage that no committed state effect could have produced.

This can falsely reject executions near transaction or frame resource limits.

— RECOMMENDATION

Delay value-transfer account update and state-growth accounting until after revm has accepted the CALL frame and the value transfer or account creation can actually occur.

M-31 Phantom Limits on Failed CALL Init

CONTINUED

Alternatively, mirror revm's frame-admission checks before recording these resource effects, or rollback the pre-frame accounting before evaluating Mega limits when revm would reject the child frame.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-32**

ACKNOWLEDGED

Block capacity returns InvalidTx

CATEGORY	Validation
LOCATION	crates/mega-evm/src/block/limit.rs, crates/mega-evm/src/block/result.rs
REVIEW	Main Review

DESCRIPTION

Mega exposes temporary block-capacity failures through the permanent invalid-transaction error channel.

The block-limit docs distinguish transaction-level violations from block-level capacity violations.

Transaction-level violations are permanent invalid transaction errors because the transaction can never be included under the configured per-transaction limits.

Block-level capacity violations are different.

They mean the transaction does not fit in the remaining capacity of the current block, or that the current block has already reached a cumulative resource limit.

The documented behavior is to skip the transaction for this block and try another transaction.

However, `BlockLimiter::pre_execution_check()` returns `BlockValidationError::InvalidTx` for several block-capacity failures, including block transaction-size capacity, block DA-size capacity, block data usage, block KV-update usage, and block state-growth usage.

`MegaBlockLimitExceededError` also implements `InvalidTxError`, so these temporary block-capacity failures are exposed through the same invalid-transaction channel used for permanent transaction invalidity.

As a result, a transaction that is valid by itself but does not fit in the remaining current block capacity can be misclassified as permanently invalid.

A normal builder or transaction-pool integration that treats `InvalidTx` as a permanent validation failure can discard a valid transaction that merely failed to fit the current block and should be retried in a later block.

M-32 Block capacity returns InvalidTxCONTINUED

— RECOMMENDATION

Return a distinct temporary block-capacity error for block-level limit failures.

Keep InvalidTx only for per-transaction intrinsic invalidity, such as exceeding a transaction-level gas, encoded-size, or DA-size limit.

Block-level remaining-capacity failures should use a retryable block-building error analogous to TransactionGasLimitMoreThanAvailableBlockGas.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-33**

ACKNOWLEDGED

CALLCODE pays impossible creation gas

CATEGORY	Validation
LOCATION	crates/mega-evm/src/evm/instructions.rs, crates/mega-evm/src/evm/execution.rs
REVIEW	Main Review

— DESCRIPTION

Note: this one is part of M-01, in more detailed way

Mega charges dynamic new-account storage gas for CALLCODE using the raw bytecode-address operand, even though CALLCODE cannot create or fund that address.

In the CALL-like instruction wrapper, Mega reads the stack to operand and checks whether that address is empty.

When the opcode transfers nonzero value and the operand address is empty, Mega charges `new_account_storage_gas(to)`.

This logic is applied to both CALL and CALLCODE.

For CALL, the to operand is the balance recipient and can become a newly created account.

For CALLCODE, the to operand is only the bytecode address.

CALLCODE executes that code in the caller's account context.

The to address is not the balance recipient, is not funded, and cannot be created by the call.

As a result, a valid value-transferring CALLCODE to an empty bytecode address can be charged dynamic new-account storage gas and a SALT bucket dependency for an account that cannot appear in final state.

This can falsely exhaust transaction gas or block gas, depending on the target account's dynamic storage-gas multiplier.

A transaction can be rejected or forced out-of-gas even though the charged account creation is impossible.

— RECOMMENDATION

Exclude CALLCODE from bytecode-address new-account storage-gas charging.

M-33 CALLCODE pays impossible creation gasCONTINUED

For CALLCODE, account-creation storage gas should not be charged for the to operand because that address cannot be created or funded by the opcode.

Only CALL should charge new-account storage gas for a nonzero-value transfer to an actually empty recipient account.

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm/pull/284>

MEDIUM **M-34**

ACKNOWLEDGED

Cross frame refunds bypass growth limits

CATEGORY Logical Error
LOCATION Host::selfdestruct()
REVIEW Main Review

DESCRIPTION

SSTORE accounting credits refunds to the currently executing frame whenever the write resets a slot back to its transaction original value

```
let loaded_data = SStoreResult { original_value, present_value, new_value };
additional_limit.on_sstore(target_address, index, &loaded_data)
```

The downstream trackers then do

```
// state_growth.rs
(true, false, true) => {
  self.record_refund(1);
}
```

And similarly

```
// data_size.rs / kv_update.rs
else if store_result.is_original_eq_new() {
  self.record_refund(...);
}
```

This is unsafe in REX4+ because frame local budgets are enforced using

```
entry.used().saturating_sub(entry.refund) > entry.limit
```

So a child frame can clear storage slots that were created or modified by an ancestor frame, receive the refund inside the child frame, and then spend that refund on unrelated new storage writes

The refund belongs to the ancestor frame prior charge, but it is credited to the child before the child own frame limit check

That breaks the REX4+ 98 / 100 per-frame resource isolation rule

M-34 Cross frame refunds bypass growth limits

CONTINUED

A transaction can make an arbitrarily deep child frame perform up to the transaction level amount of persistent storage growth, even if that child was forwarded almost no frame local budget

The final transaction level usage can still appear within limits because the ancestor created slots are cleared, but the child frame was allowed to spend budget it never received

— RECOMMENDATION

Track local and cross frame SSTORE reset refunds separately, or otherwise prevent ancestor paid refunds from increasing the current child frame spendable budget

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-35**

ACKNOWLEDGED

EIP7702 authorizations bypass dynamic pricing

CATEGORY	Logical Error
LOCATION	MegaHandler::validate()
REVIEW	Main Review

— DESCRIPTION

The dynamic pricing charges SALT backed storage gas for new accounts through only three visible paths

- 1 - Top-level TxKind::Call / TxKind::Create in MegaHandler::validate()
- 2 - Internal value CALL / CALLCODE in storage_gas_ext::wrap_call_with_storage_gas!
- 3 - Internal CREATE / CREATE2 in storage_gas_ext::create()

but EIP-7702 authorization list processing can also create or materially update authority accounts before execution, and there is no dynamic pricing hook for those authority accounts

The code partially recognizes that EIP-7702 authorities are persistent account writes, because DataSizeTracker::before_tx_start() and KVUpdateTracker::before_tx_start() charge data. /KV usage for every valid authorization authority

But no equivalent call to

```
ctx.new_account_storage_gas(authority)
```

or any other SALT backed dynamic pricing function is made for the same authority accounts

As a result, an attacker can submit a transaction with many EIP-7702 authorizations from fresh generated EOAs and cause those authority accounts to be created or updated while paying only the normal EIP-7702 intrinsic / authentication cost, not the dynamic account creation storage gas

The root cause is that MegaHandler::validate() adds Mega dynamic storage gas only for the transaction's main kind

There is no loop over tx.authorization_list() here to charge dynamic account storage gas for authority accounts that are empty before authorization processing

M-35 EIP7702 authorizations bypass dynamic pricing

CONTINUED

By contrast, the data and KV trackers do loop over the authorization list

```
for authorization in tx.authorization_list() {  
  if authorization.authority().is_some() {  
    self.frame_tracker.tx_mut().persistent_usage += ACCOUNT_INFO_WRITE_SIZE;  
  }  
}  
  
for authorization in tx.authorization_list() {  
  if authorization.authority().is_some() {  
    self.frame_tracker.tx_mut().persistent_usage += 1;  
  }  
}
```

So the codebase already treats EIP-7702 authorities as account info writes, but the dynamic pricing layer omits them

— RECOMMENDATION

Add dynamic pricing for EIP-7702 authorities that will actually be applied, including SALT account pricing, before final gas affordability/fee deduction.

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/15>

MEDIUM **M-36**

ACKNOWLEDGED

Deposit failures become stateful receipts

CATEGORY Validation
LOCATION execution.rs
REVIEW Main Review

DESCRIPTION

MegaTransaction is an alias for `op_revm::OpTransaction`, and `OpTransaction` has a dangerous asymmetry `tx_type()` becomes `DEPOSIT_TRANSACTION_TYPE` whenever `deposit.source_hash != B256::ZERO` but `gas_limit()` still returns only `base.gas_limit()` unchanged

the root cause is that for deposit transactions, `OpHandler::validate_env()` returns early and skips normal mainnet environment validation because deposits are assumed to be pre verified on L1

that means deposit class transactions do not go through the same validation path that normally enforces environment/gas-limit constraints

The codebase then adds Mega specific intrinsic / storage gas in `MegaHandler::validate()`

```
initial_and_floor_gas.initial_gas += calldata_storage_gas;  
if ctx.spec.is_enabled(MegaSpecId::REX) {  
    initial_and_floor_gas.initial_gas += constants::rex::TX_INTRINSIC_STORAGE_GAS; // 39,000  
}  
if initial_and_floor_gas.initial_gas > ctx.tx().gas_limit() {  
    return Err(InvalidTransaction::CallGasCostMoreThanGasLimit { }.into());  
}
```

For a normal transaction, that error rejects the tx

For a deposit class transaction, `OpHandler::catch_error()` catches transaction errors, reverts the journal, then commits deposit-side effects, it bumps the caller nonce, credits the deposit mint, commits the tx, and returns a `FailedDeposit` halt with `gas_used = tx.gas_limit()`

So the invariant, if Mega intrinsic gas exceeds `MegaTransaction::gas_limit()`, the transaction is invalid and has no state effect, is false for deposits

M-36 Deposit failures become stateful receipts

CONTINUED

This could be danger when public L1 deposits or any deposit like admission path can provide gas limits that are not already Mega aware

An attacker can force inclusion of deposit transactions that fail Mega own gas limit rule while still committing deposit-side state

The block gas charged is the attacker controlled `MegaTransaction::gas_limit()`, not the Mega intrinsic / storage gas that made the transaction invalid, this creates an underpriced inclusion path

— RECOMMENDATION

Add a Mega aware deposit gas validation before OP deposit error recovery. If forced inclusion must produce a receipt, charge/account at the Mega minimum, not only `tx.gas_limit`

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-37**

ACKNOWLEDGED

Implicit CALL returndata copy bypasses metering

CATEGORY Logical Error
LOCATION execution.rs
REVIEW Main Review

— DESCRIPTION

call_inputs.return_memory_offset is attacker chosen by the caller CALL like opcode

Mega preserves it in CallOutcome, but MegaEvm::frame_return_result() pops / checks AdditionalLimit before delegating to inner.frame_return_result(result), where revm copies callee returndata into the caller memory at that range

So the expensive operation

```
copy min(returndata.len, call_inputs.return_memory_offset.len()) bytes
```

happens outside compute_gas_ext, outside after_frame_run, and after the frame trackers are popped

A contract can pre expand memory once, trigger volatile data detention, then repeatedly call a helper / precompile with

```
CALL(gas, callee, 0, in_off, in_len, ret_off, HUGE_LEN)
```

using the same already expanded return buffer

changing ret_len from 0 to Huge_len causes large host memcpy work, but compute_gas_used is effectively unchanged except for the callee / precompile own metered work

This bypasses Rex4+ compute detention and per frame compute accounting for post call return copying

— RECOMMENDATION

Meter the implicit return copy before calling inner.frame_return_result(result), charging it to the parent frame

M-37 Implicit CALL returndata copy bypasses metering

CONTINUED

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-38**

ACKNOWLEDGED

Oracle sendHint exposes unmetered output channel

CATEGORY Validation
LOCATION intercept.rs
REVIEW Main Review

DESCRIPTION

In Rex2+ OracleHintInterceptor

```
let input_bytes = call_inputs.input.bytes(ctx);  
if let Ok(call) = IOracle::sendHintCall::abi_decode(&input_bytes) {  
    ctx.oracle_env.borrow().on_hint(call_inputs.caller, call.topic, call.data);  
}
```

IOracle::sendHintCall.data is ABI decoded and exported to OracleEnv::on_hint() during frame_init, but the bytes are not charged to Mega data size limits, block data limits, receipt / log accounting, or callee gas

This is worse than ordinary callback CPU metering: the decoded data becomes a durable external side channel payload while the EVM outcome can contain no log, no state change, and even a failed child call

A contract can allocate a large bytes buffer once, then repeatedly call the oracle with zero child gas

Each child call has gas_limit = 0, so the real oracle contract cannot execute normally

But the interceptor runs first and still delivers payload through

```
on_hint(caller, topic, call.data)
```

The parent ignores the failed calls and can still finish successfully

A tiny successful transaction can force the node/ oracle backend to ingest many copies of a large sendHint.data payload while the transaction / block data counters stay almost unchanged

This bypasses the intended 12.5 MB style data controls and can create memory, disk, network for any production OracleEnv that stores, forwards, or acts on hint data

M-38 Oracle sendHint exposes unmetered output channel

CONTINUED

— RECOMMENDATION

Make hints committed and metered, or require the normal oracle call to succeed before emitting them

Consider rejecting zero gas and reverted hint calls and charge hint payload bytes against a transaction or block limit

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/5>

MEDIUM **M-39**

ACKNOWLEDGED

Volatile hash prechecks rewrite stack failures

CATEGORY	Logical Error
LOCATION	instructions.rs
REVIEW	Main Review

DESCRIPTION

volatile_data_ext::wrap_op_detain_gas_unconditional! checks volatile_access_disabled() before running the real opcode

That is safe for zero operand opcodes like TIMESTAMP, NUMBER, BASEFEE, etc, but not for

```
BLOCKHASH number
```

```
BLOBBHASH index
```

Both require a stack operand, when volatile access is disabled, Mega returns a synthetic Revert immediately

```
if context.host.volatile_access_disabled() {  
    context.interpreter.bytecode.set_action(InterpreterAction::new_return(  
        InstructionResult::Revert,  
        volatile_data_access_disabled_revert_data($access_type),  
        context.interpreter.gas,  
    ));  
    return;  
}
```

So blockhash / blobhash do not first validate stack availability, opcode activation, or canonical operand handling

A malformed blockhash or Blobhash with no number / index becomes a catchable, gas refunding REVERT instead of the canonical exceptional halt

Example

M-39 Volatile hash prechecks rewrite stack failures

CONTINUED

1. Contract calls `MegaAccessControl.disableVolatileDataAccess()`
2. It then enters code that executes bare `BLOCKHASH` or `BLOBHASH` with an empty stack.
3. Canonical EVM result should be `StackUnderflow` / exceptional halt.
4. Mega instead returns `VolatileDataAccessDisabled()` as `REVERT` with remaining gas.

In call / create contexts, this can change failure class, gas burn behavior, and parent frame control flow for code that should have failed canonically

— RECOMMENDATION

Use operand aware wrappers for `BLOCKHASH` and `BLOBHASH`

Validate the required stack operand and opcode activation before returning the synthetic volatile disabled revert

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-40**

ACKNOWLEDGED

Malformed LOG topics expand memory first

CATEGORY Logical Error
LOCATION instructions.rs
REVIEW Main Review

DESCRIPTION

The codebase wraps all LOG0 - LOG4 handlers as

```
additional_limit_ext::log<N>  
-> storage_gas_ext::log<N>  
-> compute_gas_ext::logN  
-> revm::instructions::host::log<N>
```

Both wrappers only pre read the data length

```
let Some(len) = context.interpreter.stack.inspect::<1>() else {  
context.interpreter.halt(InstructionResult::StackUnderflow);  
return;  
};  
let len = as_usize_or_fail!(context.interpreter, len);
```

They do not verify that the stack contains the remaining N topic operands before executing the inner LOG

The inner revm LOG flow is dangerous for this wrapper, it pops only offset,len, charges LOG gas, expands memory for offset + len, copies the memory bytes, and only then pops the N topics

If topics are missing, it returns StackUnderflow after the expensive memory work has already happened

The upstream source shows this order: popn!([offset, len]), resize_memory(, offset, len) , Bytes::copy_from_slice(, and only afterward stack.popn::<N>()

Mega then makes this consensus relevant because compute_gas_ext aborts on the exceptional result before recording compute gas, and additional_limit_ext::log aborts before on_log() records data usage

This can bypasses the resource model that Mega adds on top of EVM gas, the attacker still spends ordinary EVM gas, but the expensive work is not reflected in

M-40 Malformed LOG topics expand memory first

CONTINUED

```
outcome.compute_gas_used
outcome.data_size
outcome.kv_updates
outcome.state_growth_used
```

Because LOG storage / data accounting scales with len, while revm memory expansion scales with offset + len

— RECOMMENDATION

validate the full `2 + N` LOG operand stack before charging storage gas or entering the revm LOG handler

also consider making compute accounting run for faulting opcodes that have already consumed gas or performed host work

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-42**

ACKNOWLEDGED

CALL gas operand swallowed during abort

CATEGORY Logical Error
LOCATION instructions.rs
REVIEW Main Review

DESCRIPTION

In Call / Callcode / Delegatecall / staticcall, revm first consumes the stack gas operand, caps it, deducts the child gas from the parent, and builds CallInputs.gas_limit

For value calls, the normal CALL_STIPEND is added after the parent deducted gas

The codebase then records compute gas inside compute_gas_ext, If record_compute_gas() fails, compute_gas! halts immediately

The outer forward_gas_ext wrapper then aborts via run_inner_instruction_or_abort! before it reaches the code that adjusts / refunds the child gas

An attacker controlled CALL gas operand has already been deducted, but no child frame runs and no frame result returns that gas.

RECOMMENDATION

when a compute limit halt cancels an unexecuted NewFrame, refund the parent deducted forwarded gas before replacing the action.

then clear the pending action and apply the additional limit halt / revert

RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/23>

MEDIUM **M-44**

ACKNOWLEDGED

Create2 validates salt after code hashing

CATEGORY Logical Error
LOCATION instructions.rs
REVIEW Main Review

DESCRIPTION

Note : this is the same part of H-07 but in more detailed way

In `storage_gas_ext::create::< IS_CREATE2 = true >`

```
let initcode_offset = stack.inspect::<1>()?;  
let initcode_len = stack.inspect::<2>()?;  
let code = Bytes::copy_from_slice(  
    context.interpreter.memory.slice_len(initcode_offset, initcode_len).as_ref(),  
);  
let initcode_hash = keccak256(&code);  
let salt = stack.inspect::<3>()?; // checked only after copy + hash
```

Create2 requires four operands

```
[value, initcode_offset, initcode_length, salt]
```

but the wrapper copies and hashes attacker controlled memory using `initcode_offset` and `initcode_length` before checking that salt exists

a malformed CREATE2 with exactly three stack items can force large host-side memory copy and Keccak work, then halt with `StackUnderflow`

canonical revm would reject immediately on stack underflow, before memory reads, hashcost, CREATE2 gas, or storage-gas address prediction

Example

M-44 Create2 validates salt after code hashing

CONTINUED

```
PUSH1 0x01
PUSH3 0x07ffff
MSTORE8
// Stack has only value, offset, length. No salt.
PUSH3 0x080000 // initcode_length
PUSH0 // initcode_offset
PUSH0 // value
CREATE2 // missing salt
```

The wrapper performs

```
copy memory[0..0x80000]
keccak256(copy)
then StackUnderflow on missing salt
```

no canonical CREATE2 execution occurs, so the extra host work is outside normal CREATE2 metering and outside `compute_gas_ext::create2`

for plain CREATE, the wrapper performs creator account inspection and dynamic SALT / storage gas lookup before checking any required operands

```
[value, initcode_offset, initcode_length]
```

So even malformed CREATE stack underflows can reach host side account / SALT work first

a contract can repeatedly call a child that performs the malformed Create2, causing large copy/hash work that is not charged as Create2 hashcost and not recorded as Mega compute gas

this bypasses the intended compute gas and volatile detention accounting for exactly the host work Mega added in the wrapper

— RECOMMENDATION

Validate full CREATE/CREATE2 stack before any memory read, hashing, account inspection, or SALT lookup.

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm/pull/296>

MEDIUM **M-46**

ACKNOWLEDGED

Deposit nonce creates unmetered accounts

CATEGORY Validation
LOCATION MegaContext::on_new_tx()
REVIEW Main Review

DESCRIPTION

Note: H-15 and M-46 are concrete deposit caller variants: mint-created caller and nonce created caller, So it's the same root cause

A fresh OP deposit caller can become a real non empty L2 account solely through the deposit nonce bump, even with

```
deposit.mint = 0
```

```
tx.value = 0
```

```
no CREATE
```

The AdditionalLimit system does not account for that caller side account creation

In MegaContext::on_new_tx()

```
self.additional_limit.borrow_mut().before_tx_start(&self.inner.tx);
```

before_tx_start() charges data/KV for the caller, but

```
self.state_growth.before_tx_start(tx);
```

is effectively a no op for caller creation

Then normal validation only prices new accounts for

```
TxKind::Create => created_address
```

```
TxKind::Call(address) => recipient if tx.value() != 0
```

It never charges

```
new_account_storage_gas(tx.caller())
```

for a deposit caller that is empty before OP deposit pre execution but becomes non empty because the deposit nonce is written

an attacker can submit many deposits from fresh L1 controlled addresses and create many L2 accounts while

M-46 Deposit nonce creates unmetered accounts

CONTINUED

StateGrowthTracker records 0 growth for the caller account creation

dynamic SALT backed new_account_storage_gas(caller) is never charged, and block state_growth_used remains underreported

That bypasses the Rex / Rex4 state growth cap and account creation pricing for deposit originated caller accounts

— RECOMMENDATION

Before deposit pre-execution, charge empty caller creation caused by deposit nonce or mint exactly once.

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/5>

MEDIUM **M-47**

ACKNOWLEDGED

Data limit Revert masks create OutOfGas

CATEGORY Logical Error
LOCATION .
REVIEW Main Review

— DESCRIPTION

A nested CREATE can fail Mega post run code deposit storage charge and produce OutOfGas then Rex4+ frame local data limit handling overwrites that OutOfGas with Revert inside after_frame_run_instructions

Because Revert returns unused child gas to the parent, an attacker can loop large CREATE returns and dodge the intended code deposit burn

Repeated nested CREATEs can avoid the intended exceptional gas burn

— RECOMMENDATION

In after_frame_run_instructions, apply Rex4+ data-limit Revert only to successful returns and avoid overwriting OutOfGas so failed CREATE keeps its exceptional gas burn

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-49**

ACKNOWLEDGED

Stale external env across blocks

CATEGORY	Validation
LOCATION	.
REVIEW	Main Review

— DESCRIPTION

MegaETH's external-env layer is explicitly designed around block-specific SALT and oracle snapshots: block context is fixed when the factory creates the env instance, not per call, and the factory docs say those queries should read the parent block for the block being executed

But the block-transition path does not actually refresh those block-bound envs.

`with_block()` only updates `inner.block` and calls `on_new_block()`, and `on_new_block()` only resets `dynamic_storage_gas_cost`; the only API that replaces `salt_env` / `oracle_env` is the separate `with_external_envs()` builder.

So a reused MegaContext / MegaEvm can advance to block N+1 while still holding the external snapshot created for block N

That is consensus visible, not just stale bookkeeping

Oracle reads: `Host::load()` checks `oracle_env.get_oracle_storage()` first and returns that value before DB fallback, so stale `oracle_env` means block N+1 can observe old oracle data

SALT pricing: storage/account/contract creation gas flows through `DynamicGasCost` into `salt_env.get_bucket_capacity()`

`DynamicGasCost` even stores a `parent_block`, and `on_new_block()` updates it, but the actual capacity lookup ignores that field and just queries the retained `salt_env`.

Exploit example

1. Execute block N with an env snapshot bound to parent N-1.
2. In block N, attacker expands a SALT bucket or changes oracle-visible data.
3. Reuse the same EVM/context for block N+1, updating only `BlockEnv` (`with_block` / equivalent).
4. A public tx in block N+1: reads the oracle contract, or does zero->nonzero SSTORE / empty-account value transfer / CREATE in that bucket.

M-49 Stale external env across blocks

CONTINUED

5. Execution still uses the old external snapshot, so block N+1 runs against parent N-1 data instead of parent N data

That can flip branches, change return values, and change OOG vs success on storage-priced ops

In other words: block-boundary consensus divergence if the same context/EVM is reused across blocks without an explicit `with_external_envs(factory.external_envs(new_block))`

Condition of this issue is that

the normal block execution path in this codebase creates a fresh EVM per block through

`MegaEvmFactory::create_evm()`, which calls `external_env_factory.external_envs(block_number)`.

An attacker cannot trigger this by transaction input alone; it requires a node/tool/integration to reuse

`MegaContext` / `MegaEvm` across blocks via public APIs like `with_block`, `set_block`, or `block_env_mut`

— RECOMMENDATION

Replace public cross-block `with_block` / `set_block` usage with an atomic

`with_block_and_external_envs(block, factory.external_envs(block.number))` path, and reserve bare block mutation for internal initialization or tests only.

That path must update `inner.block`, replace `salt_env` / `oracle_env`, and recreate `DynamicGasCost` from the same block snapshot, so a context cannot represent block N+1 with block N external data.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-51**

ACKNOWLEDGED

Hardfork defaults leave payloads unlimited

CATEGORY Validation
 LOCATION —
 REVIEW Main Review

DESCRIPTION

The block limiter has fields for raw encoded transaction size and DA size

```
tx_encode_size_limit
block_txs_encode_size_limit
tx_da_size_limit
block_da_size_limit
```

pre_execution_check() enforces them

```
if tx_size > self.limits.tx_encode_size_limit { }
if tx_size + self.block_tx_size_used > self.limits.block_txs_encode_size_limit { }
if da_size > self.limits.tx_da_size_limit { }
if da_size + self.block_da_size_used > self.limits.block_da_size_limit { ... }
```

But BlockLimits::from_hardfork_and_block_gas_limit() starts from no_limits()

```
let limits = Self::no_limits()
.with_tx_runtime_limits(tx_runtime_limits)
.with_block_gas_limit(block_gas_limit);
```

and for Mega hardforks sets only execution-data, KV, and state-growth limits

```
MegaHardfork::Rex |
MegaHardfork::Rex1 |
MegaHardfork::Rex2 |
MegaHardfork::Rex3 |
MegaHardfork::Rex4 |
MegaHardfork::Rex5 => Self {
  block_txs_data_limit: crate::constants::mini_rex::BLOCK_DATA_LIMIT,
  block_kv_update_limit: crate::constants::mini_rex::BLOCK_KV_UPDATE_LIMIT,
  block_state_growth_limit: crate::constants::rex::BLOCK_STATE_GROWTH_LIMIT,
  ..limits
}
```

M-51 Hardfork defaults leave payloads unlimited

CONTINUED

It does not set

```
tx_encode_size_limit  
block_txs_encode_size_limit  
tx_da_size_limit  
block_da_size_limit
```

So the pre-execution raw-payload limits remain `u64::MAX`

this is impactful if this constructor is used for production limits

Execution data limits are not a substitute for raw transaction-body / DA-size limits.

A transaction can be rejected or halted during EVM execution due to `tx_data_limit`, but the raw transaction bytes have already been accepted into the block and propagated. `BlockLimiter` has a pre-execution layer specifically meant to reject oversized transaction bodies cheaply, but the hardfork constructor leaves those caps disabled.

Attack shape

1. Construct a transaction with very large calldata / encoded body.
2. Keep declared gas within block gas by using the cheapest calldata pattern available.
3. `BlockLimiter::pre_execution_check` accepts because `tx_encode_size_limit` and DA limits are `u64::MAX`.
4. The EVM may later fail the transaction on data-size limits, but the oversized raw tx payload is already included

This can cause oversized block bodies, oversized DA payloads, large decode / propagation / mempool memory pressure

— RECOMMENDATION

In `BlockLimits::from_hardfork_and_block_gas_limit`, require / apply finite sequencer-configured tx body and DA caps instead of inheriting `u64::MAX`

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-52**

ACKNOWLEDGED

Pre validation accounting enables DoS

CATEGORY	Logical Error
LOCATION	execution.rs::MegaHandler::before_run context.rs::MegaContext::on_new_tx data_size.rs::DataSizeTracker::before_tx_start kv_update.rs::KVUpdateTracker::before_tx_start
REVIEW	Main Review

DESCRIPTION

before_run() calls ctx.on_new_tx() before validate() and before pre_execution():

```
self.before_run(evm)?;  
let init_and_floor_gas = self.validate(evm)?;  
let eip7702_refund = self.pre_execution(evm)? as i64;
```

Inside before_run()

```
evm.ctx_mut().on_new_tx();
```

Inside on_new_tx()

```
self.additional_limit.borrow_mut().reset();  
self.additional_limit.borrow_mut().before_tx_start(&self.inner.tx);
```

before_tx_start() then performs authorization-list accounting. In DataSizeTracker:

```
for authorization in tx.authorization_list() {  
    if authorization.authority().is_some() {  
        self.frame_tracker.tx_mut().persistent_usage += ACCOUNT_INFO_WRITE_SIZE;  
    }  
}
```

And in KVUpdateTracker:

```
for authorization in tx.authorization_list() {  
    if authorization.authority().is_some() {  
        self.frame_tracker.tx_mut().persistent_usage += 1;  
    }  
}
```

This happens before

```
validate_env()  
validate_initial_tx_gas()
```

M-52 Pre validation accounting enables DoS

CONTINUED

```
sender nonce check  
sender balance check  
fee deduction
```

A public admission path that executes this handler can be spammed with invalid transactions containing large EIP-7702 authorization lists

Attack example

tx.sender = unfunded account or bad nonce

tx.authorization_list = many entries

tx.gas_limit = invalid / too low / not affordable

submit repeatedly

Execution path

```
before_run()  
on_new_tx()  
before_tx_start()  
iterate authorization list  
call authorization.authority() in DataSizeTracker  
call authorization.authority() again in KVUpdateTracker  
validate/pre_execution later rejects sender state  
attacker pays nothing
```

If `authorization.authority()` performs signature recovery or validation, this becomes an unpaid cryptographic CPU amplifier. Even if authority recovery is cached elsewhere, this is still unpaid linear work over attacker-controlled transaction contents before cheap sender-state rejection.

This can slow or exhaust public validation / sequencing nodes through unpaid EIP-7702 authorization iteration, possible repeated signature recovery, duplicated work across data-size and KV trackers

— RECOMMENDATION

Split "reset per-transaction state" from "account intrinsic resource usage."

1. In `before_run()`, only reset trackers and volatile state.
2. Run `validate_env()` and cheap sender-state checks first.

M-52 Pre validation accounting enables DoS

CONTINUED

3. Only after the transaction is known to have a valid sender nonce/balance, run `before_tx_start()`.
4. Avoid calling `authorization.authority()` just for size/KV accounting before validation.
5. Cache recovered authorities once and share the result across trackers.

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/15>

MEDIUM **M-53**

ACKNOWLEDGED

Frames / opcodes can burn compute that never records

CATEGORY	Logical Error
LOCATION	instructions.rs, host.rs, limit.rs
REVIEW	Main Review

DESCRIPTION

MegaEVM records compute gas and applies volatile-data detention only after the wrapped revm opcode returns successfully

Most opcode wrappers use `run_inner_instruction_or_abort!`.

That macro executes the inner revm instruction, then immediately returns if the instruction ended with an exceptional result.

As a result, any opcode that performs real host work before later failing can skip Mega's post-instruction accounting.

The affected pattern is:

```
let gas_before = context.interpreter.gas.remaining();
run_inner_instruction_or_abort!($original_fn, context);
let gas_used = gas_before.saturating_sub(context.interpreter.gas.remaining());
compute_gas!(context.interpreter, additional_limit, gas_used);
```

If the inner instruction fails, the wrapper returns before `compute_gas!` runs.

This is unsafe because several EVM instructions can perform expensive work before failing with `OutOfGas`, `StackUnderflow`, or another exceptional result.

Examples include

- `SLOAD`, which can load storage or query `oracle_env.get_oracle_storage()` before the gas charge fails.
- `BALANCE`, `EXTCODESIZE`, `EXTCODECOPY`, and `EXTCODEHASH`, which can load account or code state before failing.
- `SELFDESTRUCT`, which can mark beneficiary access, inspect the source account, scan storage for refunds, and cold-load the target before failing.

M-53 Frames / opcodes can burn compute that never records

CONTINUED

- Any faulting frame where revm later burns remaining gas, while Mega never records that burned amount as compute gas

For volatile-data opcodes, the same early return also skips `apply_compute_gas_limit!`

This means a failed volatile access can mark the tracker or perform volatile-dependent host work, but the detained compute cap is not synchronized into `AdditionalLimit`

An attacker can repeatedly trigger under-gassed or malformed opcode executions that perform real database, oracle, account, code, or SELFDESTRUCT-side work while Mega underreports `compute_gas_used`.

For beneficiary or oracle paths, the attacker can also bypass volatile-data detention on the failing opcode path.

This weakens Mega's compute-gas DoS protection and can let transactions consume significantly more node CPU or backend I/O than the reported compute usage and configured limits allow.

— RECOMMENDATION

Make compute accounting and volatile detention run on exceptional paths too

Consider

1. Changing opcode wrappers so they record gas spent even when the inner instruction returns an exceptional result
2. Apply volatile-data detention after volatile host access even if the opcode fails
3. For faulting frames that burn all remaining gas, charge the burned remaining gas to compute accounting during frame finalization
4. Move expensive Mega-specific host work behind canonical affordability checks where possible, especially for `SLOAD`, account/code opcodes, and `SELFDESTRUCT`.

we could replace the early return macro with a "finally-style" wrapper that always performs required Mega accounting after the inner instruction, then preserves the original exceptional result

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Invalid transactions can force lookups

CATEGORY Validation
LOCATION execution.rs
REVIEW Main Review

DESCRIPTION

MegaHandler::validate() performs top-level dynamic storage-gas work before the normal state validation and fee/balance checks run in pre_execution()

For calls

```
TxKind::Call(address) => {  
  let new_account = !ctx.tx().value().is_zero() &&  
  ctx.db_mut().basic(address)?.is_none_or(|acc| acc.is_empty());  
  let storage_gas =  
  if new_account { ctx.new_account_storage_gas(address) } else { Some(0) };  
  (address, storage_gas)  
}
```

For creates

```
TxKind::Create => {  
  let tx = ctx.tx();  
  let caller = tx.caller();  
  let nonce = tx.nonce();  
  let created_address = caller.create(nonce);  
  let storage_gas = if is_rex_enabled {  
  ctx.create_contract_storage_gas(created_address)  
  } else {  
  ctx.new_account_storage_gas(created_address)  
  };  
  (created_address, storage_gas)  
}
```

Those calls can hit

```
ctx.db_mut().basic(address)?  
ctx.new_account_storage_gas(address)  
ctx.create_contract_storage_gas(created_address)
```

L-01 Invalid transactions can force lookups

CONTINUED

which eventually reach the external SALT environment

```
self.dynamic_storage_gas_cost.borrow_mut().new_account_gas(address)
self.dynamic_storage_gas_cost.borrow_mut().create_contract_gas(address)
```

But sender nonce/balance validation has not happened yet. In the handler flow

```
let init_and_floor_gas = self.validate(evm)?;
let eip7702_refund = self.pre_execution(evm)? as i64;
```

So an invalid transaction can force database reads and SALT bucket lookups before it is rejected for bad nonce or insufficient balance

A public sequencer/mempool/RPC validation path using this handler can be spammed with invalid transactions that never pay gas but still force expensive state/SALT work

— RECOMMENDATION

Split the current validate() path so it performs only envelope and intrinsic gas checks before sender state validation

Run the canonical nonce / code / balance check before any top-level callee account read or dynamic storage-gas/SALT lookup, then compute the top-level account/create storage gas only after that check succeeds and before building the first frame

And do this inside MegaHandler, not as an external mempool precheck, so every EVM admission / execution caller gets the same protection

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Tx access set APIs are backed by long live cache

CATEGORY Validation
LOCATION gas.rs, context.rs ,executor.rs ,state.rs
REVIEW Main Review

— DESCRIPTION

The code exposes "accessed during transaction execution" APIs, but the returned data is not transaction-local

For SALT buckets

```
fn load_bucket_cost_multiplier() {  
    match self.bucket_cost_multitipers.entry(bucket_id) {  
        Entry::Occupied() =>  
        Entry::Vacant(vacant_entry) => {  
            vacant_entry.insert(multiplier);  
        }  
    }  
}
```

And later

```
pub fn get_accessed_bucket_ids(&self) -> Vec<BucketId> {  
    self.evm.ctx_ref().dynamic_storage_gas_cost.borrow().get_bucket_ids()  
}
```

on_new_tx() never resets or snapshots dynamic_storage_gas_cost; only on_new_block() resets it. So get_accessed_bucket_ids() returns the whole block-global cache, not the current tx's bucket set.

For block hashes

```
impl<DB: Database> BlockHashes for State<DB> {  
    fn get_accessed_block_hashes(&self) -> BTreeMap<u64, B256> {  
        self.block_hashes.clone()  
    }  
}
```

L-02 Tx access set APIs are backed by long live cache

CONTINUED

```
pub fn get_accessed_block_hashes(&self) -> BTreeMap<u64, B256> {  
    self.evm.db().block_hashes.clone()  
}
```

There is no tx-local reset at all in this layer, and `on_new_block()` does not clear the underlying `State<DB>::block_hashes` cache.

So both getters can include

accesses from earlier transactions in the same block, accesses from speculative/discarded executions, and, for block hashes if the same `State<DB>` is reused, accesses from prior blocks too.

These APIs are shaped and documented as transaction-local access metadata. If a builder, scheduler, prover, or witness generator uses them after each tx, an attacker can make the reported access sets grow monotonically.

— RECOMMENDATION

Keep the SALT multiplier cache and revm block-hash cache as execution caches only, and add explicit tx-local trackers in `MegaContext` for accessed bucket IDs and block hashes

Reset those trackers in `on_new_tx()`, record bucket IDs on every storage-gas lookup and block hashes in `Host::block_hash()` after a successful lookup, and make both public getters return only these trackers

And make sure to not clear the pricing or DB caches per transaction; this preserves cache performance while preventing committed, rejected, and speculative executions from poisoning later tx metadata

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Repeated value transferring CALLCODE overcounts

CATEGORY	Validation
LOCATION	DataSizeTracker
REVIEW	Main Review

— DESCRIPTION

In DataSizeTracker::before_frame_init and KVUpdateTracker::before_frame_init, the code checks the parent frame's target_updated flag to decide whether to charge the parent caller-account update.

But after charging, the parent flag is never flipped

So repeated successful value-transferring CALLCODEs from the same parent frame can keep re-charging the same parent account update, inflating data_size and kv_updates.

Impact is premature tx-level additional-limit failures, phantom block-capacity consumption for data/KV accounting.

— RECOMMENDATION

Add a child frame charged_parent_update marker when value CALL/CALLCODE or CREATE charges the caller/parent account update

On successful frame return, merge that marker into the parent by setting parent.info.target_updated = true; on revert/failure, discard it with the child frame

Also gate this to REX5 / the latest unstable spec so stable historical specs keep their existing accounting

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm/pull/304>

LOW **L-05**

ACKNOWLEDGED

Spec changes leave consensus limits stale

CATEGORY	Validation
LOCATION	context.rs
REVIEW	Main Review

— DESCRIPTION

with_cfg changes self.spec and inner.cfg.spec, it does not rebuild

```
additional_limit  
volatile_data_tracker  
dynamic_storage_gas_cost  
spec-dependent dynamic gas formulas  
code-size/initcode-size limits when moving away from MiniRex
```

For example, switching from EQUIVALENCE to REX4 through with_cfg leaves the old AdditionalLimit and DynamicGasCost internals constructed for the previous spec. Switching back from MiniRex does not clear MiniRex contract-size limits

Also, ContextSetters::set_block delegates directly to inner and does not call on_new_block(), so dynamic gas bucket caches can remain from the previous block.

Any EVM construction path that creates a default context and later calls with_cfg or set_block can run with mismatched instruction semantics, stale dynamic gas pricing, wrong transaction limits, wrong volatile-data caps, or stale contract-size limits.

The main block executor factory path mostly constructs with the correct spec and reapplies runtime limits, so this is not clearly exploitable in the primary block production path by default

This is Low / informational because it's used only in execution paths like mega-t8n and state-test which use

```
MegaContext::default().with_cfg()
```

— RECOMMENDATION

centralize spec synchronization in one private helper used by constructors and with_cfg: apply the new cfg, rebuild AdditionalLimit and VolatileDataAccessTracker from EvmTxRuntimeLimits::from_spec(spec), recreate DynamicGasCost with the current salt_env and parent block, and recompute MiniRex code-size defaults from the incoming cfg instead of preserving stale injected values

L-05 Spec changes leave consensus limits stale

CONTINUED

Implement ContextSetters::set_block manually as self.inner.set_block(block); self.on_new_block(); so SALT bucket caches are reset on every mutable block update

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Same tx SELFDESTRUCT does not refund prior usage

CATEGORY	Validation
LOCATION	DataSizeTracker, KVUpdateTracker
REVIEW	Main Review

— DESCRIPTION

SSTORE and account creation are counted into data_size / kv_update

DataSizeTracker::after_sstore() adds STORAGE_SLOT_WRITE_SIZE

KVUpdateTracker::after_sstore() adds +1

after_frame_init_on_frame() in both trackers also counts the created account itself

But when a same tx created contract is later destroyed

Host::selfdestruct() computes a refund only for state growth

AdditionalLimit::on_selfdestruct() forwards that refund only to state_growth

DataSizeTracker and KVUpdateTracker have no after_selfdestruct() refund path

So a flow like

1. CREATE
2. many fresh-slot SSTOREs
3. same-tx SELFDESTRUCT

can leave no final account / no final storage, while data_size and kv_update still stay charged as if those writes persisted

— RECOMMENDATION

Extend the existing first effective same tx SELFDESTRUCT refund path to return structured refund details and forward them to DataSizeTracker and KVUpdateTracker in the same frame as the state-growth refund

L-06 Same tx SELFDESTRUCT does not refund prior usage

CONTINUED

Refund `ACCOUNT_INFO_WRITE_SIZE + live_new_slots * STORAGE_SLOT_WRITE_SIZE` from data size and 1 + `live_new_slots` from KV updates, using the same `CreatedLocal`, `previously_destroyed`, and `original == 0 && present != 0` checks already used for state growth

If deployed bytecode was counted for that created account, refund that byte length too or skip recording it when the create frame is already selfdestructed

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Create2 Failing Paths Skip Compute Metering

CATEGORY Logical Error
LOCATION instructions.rs
REVIEW Remediation Review 1

DESCRIPTION

This issue happens because the Create2 wrapper performs expensive work before all Create2 inputs are proven valid

The wrapper reads initcode offset and length, expands memory, copies the requested bytes into a new buffer, and hashes them

Only after that does it check whether the salt exists on the stack

If the salt is missing, the opcode stops with StackUnderflow after the memory work has already happened

The same gap can happen if storage gas charging or the inner Create2 path fails later

The resize gas is recorded into Mega compute gas only after the inner Create2 returns successfully

That means the node can spend CPU and memory bandwidth that AdditionalLimit never sees as compute usage

The original memory safety fix removed an out of bounds read, but it did not fix the validation and accounting order

```
let gas_before_resize = context.interpreter.gas.remaining();
resize_memory!(context.interpreter, initcode_offset, initcode_len);
resize_gas = gas_before_resize.saturating_sub(context.interpreter.gas.remaining());
let code = Bytes::copy_from_slice(
context.interpreter.memory.slice_len(initcode_offset, initcode_len).as_ref(),
);
let initcode_hash = keccak256(&code);
```

The cause is that salt validation happens after the resize, copy, and hash

The final compute accounting also happens after the inner CREATE2 succeeds, so every earlier return skips it

M-01 Create2 Failing Paths Skip Compute Metering

CONTINUED

```
let Some(salt) = context.interpreter.stack.inspect::<3>() else {  
context.interpreter.halt(InstructionResult::StackUnderflow);  
return;  
};  
run_inner_instruction_or_abort!(compute_gas_ext::create2, context);  
if resize_gas > 0 {  
compute_gas!(context.interpreter, additional_limit, resize_gas);  
}
```

Any contract can trigger this with a crafted Create2 stack and a large memory range, Impact is execution undercounted compute work

— RECOMMENDATION

<https://gist.github.com/GuardianAudits/df0ff485f940e55fd5c8b576de113904>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Oracle Hints Call Inputs Missed Validation

CATEGORY Validation
LOCATION `intercept.rs`
REVIEW Remediation Review 1

DESCRIPTION

This issue happens in the oracle hint interceptor, not in `AccessControl` or `LimitControl`

The selector peek avoids full copying for non matching calls, which is good

For a matching `sendHint` selector, the interceptor immediately materializes the whole call input with `input.bytes(ctx)`

That input can be a `SharedBuffer` range backed by caller memory, so materializing it copies the full `argsSize`

The child oracle frame has not started yet, so child gas failure cannot stop this host side copy.

The `Rex5` guard only skips calls with `gas_limit` equal to zero

A caller can forward one gas, bypass the guard, and still make the interceptor copy a huge buffer

The later metering records only decoded topic plus decoded data length, so huge trailing bytes can be copied but not charged

```
if is_rex5_enabled && call_inputs.gas_limit == 0 {  
    return None;  
}  
let selector = peek_selector(&call_inputs.input, ctx)?;  
if selector != IOracle::sendHintCall::SELECTOR {  
    return None;  
}  
let input_bytes = call_inputs.input.bytes(ctx);
```

The cause is that full input materialization is before resource accounting, the accounting uses decoded semantic bytes, not the actual bytes copied from the caller input

```
let Ok(call) = IOracle::sendHintCall::abi_decode(&input_bytes) else {  
    return None;
```

M-02 Oracle Hints Call Inputs Missed Validation

CONTINUED

```
};  
let bytes_recorded = (call.data.len() as u64) + Self::HINT_TOPIC_BYTES;  
let within = ctx.additional_limit.borrow_mut().record_oracle_hint_bytes(bytes_recorded);
```

Any contract can call the oracle address with selector matching calldata and a very large argsSize, Impact is host memory allocation and bandwidth pressure before normal gas accounting can protect the node.

— RECOMMENDATION

<https://gist.github.com/GuardianAudits/c859f02895302258a1d6df5c799b1c22>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Keyless Gas Cap Enables Signer loss

CATEGORY	Logical Error
LOCATION	executor.rs
REVIEW	Remediation Review 1

DESCRIPTION

This issue happens because KeylessDeploy validates one gas value but executes with a different smaller gas value.

The function first checks that gasLimitOverride is at least the gas limit signed inside the keyless transaction.

That check appears to protect the signer from executions with less gas than they authorized.

In Rex5, the code then caps gasLimitOverride to the parent call remaining gas.

This cap happens after validation and is not followed by a second validation.

A relayer can pass a large override, then choose an outer transaction gas limit that leaves less gas after KeylessDeploy overhead.

The sandbox transaction is built with the reduced gas limit.

The signer pays inside the sandbox using the signed gas price, but execution may fail only because the relayer reduced the envelope

```
let tx_gas_limit = keyless_tx.gas_limit();
let mut gas_limit_override_u64: u64 = gas_limit_override.try_into().unwrap_or(u64::MAX);
if gas_limit_override_u64 < tx_gas_limit {
    return make_error!(KeylessDeployError::GasLimitTooLow {
        tx_gas_limit,
        provided_gas_limit: gas_limit_override_u64,
    });
}
```

The reason is the post validation cap, the actual sandbox transaction is created after the cap, so the value that was validated is not the value that is executed

```
if ctx.spec.is_enabled(MegaSpecId::REX5) {
    gas_limit_override_u64 = gas_limit_override_u64.min(gas.remaining());
}
```

M-03 Keyless Gas Cap Enables Signer loss

CONTINUED

```
}  
let tx = TxEnv {  
  gas_limit: gas_limit_override_u64,  
  gas_price: keyless_tx.effective_gas_price(None),  
  nonce: 0,  
  ..Default::default()  
};
```

An untrusted relayer can force under gassed signer funded failures and repeat them while no code is deployed, Impact is draining prefunded keyless signer balances and blocking intended deployments

— RECOMMENDATION

<https://gist.github.com/GuardianAudits/44fb58a3c9b3300c06451caada4f309b>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

ACKNOWLEDGED

Keyless Volatile Access Not Merged

CATEGORY	Logical Error
LOCATION	execution.rs
REVIEW	Remediation Review 1

— DESCRIPTION

The M-23 resolution says the KeylessDeploy sandbox volatile-access tracker merges back into the parent under Rex5. However, the current implementation only merges sandbox AdditionalLimit resource usage into the parent transaction.

The remediation partially addresses the original issue by making Rex4+ KeylessDeploy sandboxes share the parent SALT and Oracle external environments, and by making Rex5 merge sandbox compute gas, data size, KV updates, and state growth usage back into the parent AdditionalLimit.

For Rex4 and later, KeylessDeploy executes the signed deployment inside a sandbox MegaContext that receives the parent salt_env and oracle_env, so constructor code can read real block-scoped Oracle data. But MegaContext::new_with_shared_ext_envs still allocates a fresh VolatileDataAccessTracker for the sandbox. Any Oracle, block-environment, or beneficiary volatile access is therefore recorded only in the sandbox tracker.

After sandbox execution, KeylessDeploy extracts sandbox LimitUsage and merges it into the parent AdditionalLimit via merge_usage. SandboxOutcome carries limit_usage, but no volatile-access bitmap or detention metadata. The sandbox VolatileDataAccessTracker is dropped with the sandbox context and is never merged into the parent transaction.

A KeylessDeploy constructor can therefore commit code, storage, or logs derived from Oracle, block-env, or beneficiary volatile data while the parent transaction still reports no corresponding volatile access. Any scheduler, builder, witness, or conflict layer that trusts the parent transaction's volatile-access metadata can incorrectly treat the transaction as non-volatile even though committed state depended on volatile data.

— RECOMMENDATION

Merge the sandbox VolatileDataAccessTracker into the parent context when applying sandbox results. If the sandbox accessed Oracle, block environment, or beneficiary volatile data, the parent transaction should mark the same volatile access and apply the corresponding transaction-level detention state.

M-04 Keyless Volatile Access Not Merged

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

MEDIUM **M-05**

ACKNOWLEDGED

Keyless Sandbox Gas Excluded From Block Gas

CATEGORY	Logical Error
LOCATION	execution.rs, limit.rs
REVIEW	Remediation Review 1

— DESCRIPTION

KeylessDeploy performs the real deployment inside a separate sandbox transaction.

The remediation now merges sandbox AdditionalLimit usage back into the parent transaction, so sandbox compute gas, data size, KV updates, and state growth are reflected in the parent resource trackers. However, normal sandbox success and normal sandbox execution-failure paths still do not reconcile the sandbox's EVM gas_used into the parent transaction outcome.

In execute_keyless_deploy_call, the parent gas meter is charged for the fixed KeylessDeploy overhead, then execute_keyless_deploy_sandbox runs the actual deployment with gas_limit set from gasLimitOverride. After sandbox execution, the implementation merges sandbox resource-lane usage, applies sandbox state, and forwards sandbox logs, but it does not charge sandbox_gas_used to the parent Gas object on normal success or normal sandbox execution failure. sandbox_gas_used is charged to the parent gas meter only on the residual overflow path.

This does not imply sandbox execution is unpaid. The signer may still pay sandbox gas through merged sandbox state. The residual issue is that canonical transaction, receipt, and block gas accounting consume the parent ExecutionResult::gas_used(), which excludes normal sandbox EVM gas.

BlockLimiter::post_execution_update uses outcome.result.gas_used(), and receipt construction uses the resulting cumulative gas. Therefore, KeylessDeploy can execute substantial sandbox EVM work while receipt cumulative gas and block gas accounting reflect only the outer wrapper gas.

— RECOMMENDATION

Either explicitly specify that KeylessDeploy sandbox EVM gas is intentionally excluded from canonical receipt and block gas accounting, or merge normal sandbox gas_used into the parent transaction gas outcome before receipt construction and block gas accounting.

M-05 Keyless Sandbox Gas Excluded From Block Gas

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Failed Deposits Bypass Caller Creation Gas

CATEGORY	Validation
LOCATION	execution.rs
REVIEW	Remediation Review 1

DESCRIPTION

Rex5 validation now correctly notices when a deposit like transaction will materialize a fresh caller account

It adds dynamic new account storage gas for that caller before the final initial gas check

For ordinary transactions, this is safe because a validation error stops execution before nonce or balance changes

Deposits are different because OP deposit handling can recover transaction validation errors as FailedDeposit results

A forced deposit can set a nonzero mint, use a fresh empty caller, and provide a very low gas limit

Mega validation sees that final initial gas is too high and returns CallGasCostMoreThanGasLimit

That error can still enter OP catch_error and become a failed deposit receipt, the failed deposit path can preserve caller nonce and minted balance while block gas records only raw tx.gas_limit

```
if is_deposit_like_transaction(&ctx.inner.tx, system_address) {  
    let caller_is_empty = ctx.db_mut().basic(caller)?.is_none_or(|acc| acc.is_empty());  
    if caller_is_empty {  
        let storage_gas = ctx.new_account_storage_gas(caller).ok_or_else(|| {  
            Self::Error::from_string(format!("Failed to get storage gas for deposit caller"))  
        })?;  
        initial_and_floor_gas.initial_gas += storage_gas;  
    }  
}
```

The code computes the correct required gas, but OP catch_error can still preserve deposit side effects

L-01 Failed Deposits Bypass Caller Creation Gas

CONTINUED

```
if initial_and_floor_gas.initial_gas > gas_limit {  
  return Err(InvalidTransaction::CallGasCostMoreThanGasLimit {  
    gas_limit,  
    initial_gas: initial_and_floor_gas.initial_gas,  
  })  
  .into();  
}  
let result = self.op.catch_error(evm, error)?;
```

Forced deposits can create funded caller accounts without paying the full dynamic account creation gas

The main impact is state growth and balance materialization charged only at the attacker supplied deposit gas limit

— RECOMMENDATION

<https://gist.github.com/GuardianAudits/a1f4dd57b3ab9bc7cccf6fbccf7e27e4>

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Precompile Stipends Return As Ordinary Gas

CATEGORY	Logical Error
LOCATION	precompiles.rs
REVIEW	Remediation Review 1

DESCRIPTION

Rex4 introduced `STORAGE_CALL_STIPEND` for value transferring internal `CALL` and `CALLCODE` frames

The stipend exists so receivers can afford storage heavy operations such as Mega priced `LOG` costs.

It is not meant to increase the caller ordinary gas balance

The tracker inflates the child gas limit and caps compute gas to the original pre stipend limit.

On return, it burns only the part of remaining gas that is above the original limit

Rex5 precompile handling changes the returned `Gas` object for successful or reverting precompile calls

It runs the precompile with capped effective gas, then normalizes returned `Gas` back to the original call gas limit

If the precompile spends more than the stipend amount, remaining gas can be below the original limit and the burn becomes zero

```
let effective_gas_limit = if is_rex5_enabled {  
  let remaining = context.additional_limit.borrow().current_call_remaining_compute_gas();  
  gas_limit.min(remaining)  
} else {  
  gas_limit  
};
```

The root cause is the interaction between precompile gas normalization and stipend burn logic

Normalization restores the inflated gas limit, while the stipend burn only checks whether remaining gas is above the original limit

L-02 Precompile Stipends Return As Ordinary Gas

CONTINUED

```
let mut normalized = Gas::new(gas_limit);
normalized.set_spent(spent);
normalized.record_refund(refunded);
output.gas = normalized;
let original_limit = gas.limit().saturating_sub(stipend);
let burn = gas.remaining().saturating_sub(original_limit);
```

Contracts can repeatedly recover system granted stipend gas through value transferring precompile calls, causing gas undercharging and execution beyond the intended transaction and block gas accounting

— RECOMMENDATION

<https://gist.github.com/GuardianAudits/a88bb996c0ce0bb16939ca03a45906be>

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Selfdestruct Meters Before Volatile Guard Runs

CATEGORY	Logical Error
LOCATION	instructions.rs
REVIEW	Remediation Review 1

DESCRIPTION

Rex5 changed Selfdestruct dispatch to run `storage_gas_ext` before `volatile_data_ext`

That means the outer storage wrapper executes before the volatile access disabled check.

The wrapper reads the Selfdestruct target, inspects the target account, inspects the caller account, and may query SALT.

If the target is the block beneficiary, disabled volatile access is supposed to reject before touching beneficiary related state.

Instead, the code can perform target dependent reads and dynamic new account gas charging first.

The later volatile wrapper can still revert, but forbidden work and gas burn already happened.

There is also incomplete accounting on successful Selfdestruct transfers

The Rex5 wrapper only meters empty target creation with nonzero source balance and misses existing target account writes

```
let Ok(target_account) = context.host.inspect_account(target) else {
    context.interpreter.halt(InstructionResult::FatalExternalError);
    return;
};
let is_empty = target_account.state_clear_aware_is_empty(eth_spec);
let Ok(caller_account) = context.host.inspect_account(caller) else {
    context.interpreter.halt(InstructionResult::FatalExternalError);
    return;
};
```

The volatile guard runs only after storage gas and resource tracking have already happened

L-03 Selfdestruct Meters Before Volatile Guard Runs

CONTINUED

```
if is_empty && has_value {  
  let Some(cost) = context.host.new_account_storage_gas(target) else {  
    context.interpreter.halt(InstructionResult::FatalExternalError);  
    return;  
  };  
  gas!(context.interpreter, cost);  
  context.host.additional_limit().borrow_mut().on_selfdestruct_new_account();  
}  
run_inner_instruction_or_abort!(volatile_data_ext::selfdestruct, context);
```

Contracts can trigger forbidden pre guard work and can also hit incomplete SELFDESTRUCT account write accounting, Impact is gas griefing, volatile access policy bypass at the precheck layer, and undercounted account writes

— RECOMMENDATION

<https://gist.github.com/GuardianAudits/55428edddf2eeafa372f3745387b833d>

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Rex5 Block Validation Can Fail Persistently

CATEGORY Logical Error
LOCATION eips.rs, executor.rs
REVIEW Remediation Review 1

DESCRIPTION

Rex5 changed EIP 2935 and EIP 4788 handling from fail open to fail closed

That is correct only if the mandatory calls are also given enough gas to survive valid Mega storage costs

The code still runs both calls through the default `transact_system_call` entry point

That default path keeps the upstream fixed 30 million gas budget

Mega dynamic storage gas can multiply the cost of zero to nonzero `SSTORE` based on `SALT` bucket capacity

When the multiplier is high, one required EIP storage write can exceed the fixed budget

The call then returns a non success result, and Rex5 rejects the whole block, the same state can reject every following block because the mandatory update never commits

The external trigger requires a more specific state condition, the EIP system slot must still be empty or zero

The same `SALT` bucket must be very crowded before the system call writes that slot, then Rex5's pre block system call tries zero to nonzero

That first write becomes too expensive for the fixed 30 million gas budget, Rex5 rejects the block

```
let res = evm.transact_system_call(  
    alloy_eips::eip4788::SYSTEM_ADDRESS,  
    alloy_eips::eip2935::HISTORY_STORAGE_ADDRESS,  
    parent_block_hash.0.into(),  
)?;
```

SequencerRegistry uses a widened gas path for dynamic storage gas risk, but the mandatory EIP calls do not

L-04 Rex5 Block Validation Can Fail Persistently

CONTINUED

```
if is_rex_5 && !result.is_success() {  
    return Err(BlockValidationError::BlockHashContractCall {  
        message: std::format!("EIP 2935 pre block system call did not succeed"),  
    })  
    .into();  
}  
  
let gas_limit =  
    evm.block_env_ref().gas_limit.max(crate::constants::rex5::SYSTEM_CALL_GAS_LIMIT_FLOOR);
```

A valid high SALT state can turn mandatory pre block work into persistent block validation failure

— RECOMMENDATION

<https://gist.github.com/GuardianAudits/c886b22321d29a08519423c606c0b4a1>

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

System Transaction Nonce Check Is Stale

CATEGORY	Validation
LOCATION	execution.rs
REVIEW	Remediation Review 1

DESCRIPTION

Rex5 added nonce and chain id validation for legacy transactions sent from the Mega system address

That validation happens in `before_run` during transaction execution

The block executor API separates `run_transaction` from `commit_transaction_outcome`

That means an outcome can be produced from a state snapshot and committed later

The commit step acknowledges parallel race conditions, but it only rechecks block resource limits

It does not rerun system transaction nonce validation against the latest committed account nonce

Two same nonce system transactions can both run before either one commits

Both observe the same system account nonce and both pass validation

The first commit bumps nonce in committed state, but the second commit still applies its already computed state

```
validate_account_nonce_and_code(  
    &mut state_account.info,  
    tx_nonce,  
    disable_eip3607,  
    disable_nonce_check,  
)?;
```

commit time only checks block limiter state, It does not revalidate the sender nonce or the Mega system transaction promotion rules

```
self.block_limiter.pre_execution_check(  
    outcome.tx.tx().tx_hash(),  
    outcome.tx.tx().gas_limit(),
```

L-05 System Transaction Nonce Check Is Stale

CONTINUED

```
outcome.tx_size,  
outcome.da_size,  
outcome.tx.tx().ty() == DEPOSIT_TRANSACTION_TYPE,  
)?;  
self.evm.db_mut().commit(state);
```

The precondition is privileged system transaction access or sequencer behavior, but the effect is consensus sensitive, duplicate privileged oracle or system updates for a single nonce

— RECOMMENDATION

If that is trusted to not happen acknowledge it, otherwise mitigate for caution

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Per opcode metering does an O(call depth) scan

CATEGORY	DoS
LOCATION	FrameLimitTracker
REVIEW	Remediation Review 1

— DESCRIPTION

every compute-metered opcode calls `record_compute_gas()`, and `record_compute_gas()` immediately calls the global `AdditionalLimit::check_limit()`

That global check walks data-size, KV-update, compute-gas, and state growth trackers in order. Each tracker's TX-level `tx_usage()` ultimately uses `FrameLimitTracker::net_usage()`,

which iterates over every active frame in `frame_stack`. In a deeply nested call, every cheap opcode can therefore trigger multiple full depth stack scans

The vulnerable sink is generic, not opcode specific: the `compute_gas_ext::wrap_op_compute_gas` macro wraps ordinary opcodes, records gas before / after the inner opcode, then calls `compute_gas!()`, which feeds into `AdditionalLimit::record_compute_gas()`

The wrapper is applied broadly to cheap opcodes and call/control opcodes, so an attacker does not need logs, precompiles, Create2, selfdestruct, oracle hints, keyless deploy, or deposits

For example, If someone deploy a contract that recursively enters many active call frames, then at the deepest frame runs a long loop of cheap instructions such as Add, Push, Dup, Jumpdest, or simple control flow

Each opcode is charged normal EVM gas and normal compute gas, but the host metering path repeatedly scans all active `FrameLimitTracker` entries

With depth D and N cheap opcodes, node side accounting becomes roughly $O(D \times N)$, while the protocol visible work remains close to $O(N)$. The `net_usage()` recomputes total usage by looping over `&self.frame_stack` every time it is called

Impact is public transaction CPU amplification. A single valid transaction can force validators to spend disproportionate CPU in metering code while receipts still reflect ordinary gas / compute usage

L-06 Per opcode metering does an $O(\text{call depth})$ scanCONTINUED

— RECOMMENDATION

we could make FrameLimitTracker maintain cached aggregate totals for active frames, updating them on record / push / pop, instead of recomputing with for entry in self.frame_stack on every tx_usage()

— RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

CREATE2 Empty Initcode Observes Offset

CATEGORY	Logical Error
LOCATION	instructions.rs
REVIEW	Remediation Review 1

— DESCRIPTION

MegaEVM's REX4+/REX5 CREATE2 wrapper validates the initcode offset, expands memory, slices memory, and hashes the slice before delegating to the canonical CREATE2 implementation.

Canonical revm ignores `code_offset` when CREATE2 initcode length is zero. For `len == 0`, it does not convert the offset, expand memory, or slice memory, and the CREATE2 address is computed from `keccak256("")`.

MegaEVM therefore incorrectly observes offset for empty-initcode CREATE2. A valid CREATE2 with `len == 0` and a non-canonical offset, such as `offset = U256::MAX`, halts in REX5 instead of proceeding with empty initcode. A large finite offset can still succeed, but it consumes extra EVM gas and compute gas for memory expansion even though the offset should be ignored.

— RECOMMENDATION

Handle `len == 0` before offset conversion, memory expansion, slicing, and hashing. For empty initcode, compute the CREATE2 address using the empty-code hash and forward to canonical CREATE2 without touching offset or memory. Add regression coverage for `len == 0` with a non-canonical offset and with large finite offsets.

— RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

Keyless Empty-Code Drops Logs

CATEGORY	Logical Error
LOCATION	execution.rs
REVIEW	Remediation Review 1

— DESCRIPTION

KeylessDeploy performs the real deployment inside a sandbox transaction and then maps the sandbox result back into the parent call.

When sandbox CREATE succeeds with non-empty runtime bytecode, the implementation stores the sandbox logs in SandboxResult and forwards them to the parent with `ctx.log(log)`. However, when the same successful CREATE returns empty runtime bytecode, `process_sandbox_transact_result` reclassifies the success as `SandboxOutcome::Failure with EmptyCodeDeployed`. That failure variant carries state, error, and resource usage, but not the logs returned by the successful inner CREATE.

The failure branch still merges sandbox state and returns success-style `errorData`, so the signer charge and other sandbox effects persist. The dropped logs therefore create a mismatch: the constructor executed successfully enough to emit logs and persist state effects, but the parent receipt omits those logs.

The KeylessDeploy spec says merged sandbox state includes resulting logs when applicable and also says empty deployed bytecode returns success-style so merged state persists. A keyless deployer can trigger this with `initcode` that emits `LOG0` or `LOG1` and then returns empty runtime bytecode.

— RECOMMENDATION

Preserve logs when converting successful empty-code CREATE into `EmptyCodeDeployed`. For example, carry logs in the failure outcome for this case or introduce a dedicated empty-code outcome that forwards logs before returning success-style `errorData`.

— RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

Value Self-Transfers Double Count DataSize/KV

CATEGORY	Logical Error
LOCATION	data_size.rs, kv_update.rs, frame_limit.rs
REVIEW	Remediation Review 1

— DESCRIPTION

REX5 fixes repeated parent-frame value-transfer overcounting by deduplicating caller account updates across nested value-transferring CALL/CREATE operations. However, a top-level value self-transfer still double counts the same account in DataSize and KV update accounting.

At transaction start, `before_tx_start` records the transaction caller account update as persistent transaction usage. When the transaction is also a value transfer to the same address, the first CALL frame has `caller == target`. The value-transfer accounting path then records the target account update again.

The REX5 frame-level deduplication does not prevent this case because there is no active parent frame at top level. `push_call_frame` cannot mark a parent frame as already updated, but the `has_transfer` branch still records the target-side account update.

A normal REX5 transaction from A to A with non-zero value therefore records two account updates for the same account: DataSize includes an extra account-info write and KV updates increments by 2 instead of 1. This can cause false transaction or block resource-limit failures for self-transfers and diverges from the intended REX5 dedup behavior.

— RECOMMENDATION

Deduplicate value-transfer target accounting when the target is the same account already recorded as the transaction caller. For example, skip the target-side DataSize/KV account update for top-level `caller == target` self-transfers, or initialize top-level frame metadata so self-transfer account updates are handled consistently with REX5 nested-frame deduplication.

— RESOLUTION

Status: Acknowledged.

L-09 Value Self-Transfers Double Count DataSize/KV

CONTINUED

<https://github.com/megaeth-labs/mega-evm-internal/pull/26>

LOW **L-10**

ACKNOWLEDGED

Staticcall Selfdestruct triggers unmetered work

CATEGORY	Validation
LOCATION	Instructions.rs
REVIEW	Remediation Review 1

DESCRIPTION

REX5 Selfdestruct storage gas wrapper performs target controlled host / SALT work before validating static context

In REX5 Selfdestruct is no longer routed directly to `compute_gas_ext::selfdestruct`, the instruction table routes it as

```
volatile_data_ext::selfdestruct_rex5 → storage_gas_ext::selfdestruct →  
compute_gas_ext::selfdestruct
```

The table installs `volatile_data_ext::selfdestruct_rex5` for Selfdestruct, the outer volatile wrapper checks the beneficiary disable condition, then delegates to `storage_gas_ext::selfdestruct`

It does not validate that the current frame is non static before the storage layer runs

The storage layer then uses the attacker controlled Selfdestruct target before the canonical inner instruction is reached,

It inspects the target account, inspects the caller account, checks whether the caller has balance, calls `new_account_storage_gas(target)` for an empty target, charges gas,

and records `on_selfdestruct_new_account()` before delegating to `compute_gas_ext::selfdestruct`,

That `new_account_storage_gas()` path calls the dynamic storage gas / SALT machinery, while `inspect_account()` loads accounts from the DB / journal cache for metering inspections

A public attacker can deploy or use a funded contract whose fallback does

```
assembly {  
    selfdestruct(calldataload(0))  
}
```

L-10 Staticcall Selfdestruct triggers unmetered work

CONTINUED

Then a parent contract repeatedly calls it via Staticcall, each time passing a fresh empty target address

Expected behavior that each static call should fail cheaply as a static-context state-change violation

But in the REX5 path, before the static-context failure is reached, the VM can inspect the fresh target account, inspect the caller account, perform dynamic `new_account_storage_gas(target)` / SALT work, charge storage gas, and mutate the Mega additional limit tracker for a call that was never allowed to perform Selfdestruct

Because the parent catches the failed Staticcall, this can be repeated with fresh targets to force DB / SALT work from invalid executions.

— RECOMMENDATION

Add a static context guard before `storage_gas_ext::selfdestruct` can run, to preserve existing volatile disable behavior, place it after the current stack / beneficiary precheck but before delegating to the storage layer

```
// In the REX5 SELFDESTRUCT outer path, before storage_gas_ext::selfdestruct
if context.interpreter.runtime_flag.is_static() {
context.interpreter.halt(InstructionResult::StateChangeDuringStaticCall);
return;
}
```

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm/pull/313>

LOW **L-11**

ACKNOWLEDGED

Auth List processing resource exhaustion

CATEGORY	Validation
LOCATION	execution.rs
REVIEW	Remediation Review 1

— DESCRIPTION

Rex5 simulates EIP-7702 authorization authority nonces with a plain `Vec<(Address, u64)>`

For every authorization, it first calls `simulated_authority_nonce()`, which linearly scans the vector

Then, after processing, it performs another linear `iter_mut().find()` to update the same authority entry, or pushes a new one. With N unique authorities, this becomes $O(N^2)$ address comparisons in transaction pre execution

This work runs in `pre_execution()` after sender validation / loading and before `apply_eip7702_auth_list()`, so it is host side setup work rather than normal opcode execution.

It is not charged through Mega compute gas wrapper path

The limit failure is also delayed until after the full scan. the loop accumulates `authority_creations`,

but only after scanning the entire authorization list does it call `on_eip7702_authority_creations(authority_creations)`,

which records the total and checks the limit, the later frame init path then materializes the exceeded limit

If a user submit valid Rex5 EIP 7702 transaction with many unique, fresh authorities:

1. Each authorization has a valid signature, valid chain ID, and matching nonce.
2. Each authority is empty or non-existing, so it is counted as a potential authority creation.
3. State-growth should exceed the transaction limit quickly, but the code does not stop there.
4. The executor continues scanning the entire list, doing repeated linear searches plus account/code loads.
5. Only after all entries are processed does Mega latch the state-growth failure and later halt the transaction.

L-11 Auth List processing resource exhaustion**CONTINUED**

So a transaction that is destined to fail can still force quadratic pre execution CPU work and repeated host / account lookups before the failure is observed, making block execution spend far more CPU than its linear transaction payload / gas costs imply

— RECOMMENDATION

Replace the `Vec<(Address, u64)>` with a map keyed by authority, such as `HashMap`, `BTreeMap`, or deterministic `IndexMap`, so lookup / update is $O(1)$ or $O(\log N)$ Also charge / check authority creations incrementally and stop as soon as the state growth limit is exceeded

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm/pull/313>

MEDIUM **M-01**

ACKNOWLEDGED

Oracle multiCall Drops sendHint

CATEGORY	Logical Error
LOCATION	Oracle.sol, execution.rs, intercept.rs, oracle.md
REVIEW	Remediation Review 2

— DESCRIPTION

Oracle.multiCall is documented as the Oracle batching interface and executes each payload by DELEGATECALL into the Oracle contract. The same Oracle spec says sendHint is a Solidity-level no-op whose protocol effect is supplied by the node-side OracleHintInterceptor, and that if a transaction calls sendHint before reading an Oracle slot, the hint must be delivered to the backend before the read is served.

The current implementation cannot satisfy that ordering when sendHint is batched through multiCall. Oracle.sol implements multiCall with `address(this).delegatecall(data[i])`, while MegaEVM's system-interceptor dispatch explicitly runs only for CALL and STATICCALL. DELEGATECALL and CALLCODE are excluded before `dispatch_system_contract_interceptors` is reached, so `OracleHintInterceptor::intercept` is never called for the inner sendHint payload. Because sendHint's Solidity body is empty, the delegated call succeeds but produces no backend hint.

This is reachable through the public Oracle interface. A caller can execute `Oracle.multiCall([abi.encodeCall(sendHint, ...)])` and receive a successful result, but `OracleEnv::on_hint` records no hint. More importantly, a caller can batch `[sendHint("unlock"), getSlot(7)]` through multiCall and receive zero from a hint-backed Oracle backend, while the equivalent direct sequence `sendHint("unlock"); getSlot(7)` returns the prepared value. The batching path silently violates the documented hint-before-read behavior.

— RECOMMENDATION

Make Oracle multiCall and sendHint semantics explicit and consistent. Either reject sendHint payloads inside multiCall, change multiCall to route sendHint through an external CALL path that triggers the interceptor while preserving the intended hint sender, or add a narrowly scoped Oracle self-delegatecall hook that forwards valid sendHint payloads and meters them exactly like direct CALL/STATICCALL sendHint.

M-01 Oracle multiCall Drops sendHint

CONTINUED

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Keyless Empty-Code Replay

CATEGORY	Logical Error
LOCATION	sandbox/execution.rs, sandbox/state_merge.rs
REVIEW	Remediation Review 2

— DESCRIPTION

Rex5 KeylessDeploy accepts a recovered signer whose parent-state nonce is 0 or 1. The sandbox then overrides the signer nonce to 0, runs the signed deployment fee-free, preserves the signed inner transaction value, and merges sandbox state back into the parent on completed sandbox execution, including EmptyCodeDeployed.

This creates a replay window when the constructor leaves no deployed-code barrier. A keyless constructor can receive the signed value, execute SELFDESTRUCT in the same transaction, transfer the value to a beneficiary, and leave the deterministic deploy address empty. Under EIP-6780, a contract destroyed in the same transaction it was created has its code and storage removed, so the deploy-address existence check still passes after the first call.

After the first call, the merged signer nonce is 1, but nonce 1 is still admitted by KeylessDeploy. Re-submitting the exact same signed keyless payload through a fresh outer relayer transaction therefore executes the sandbox again and transfers the signed value again. The local PoC leaves the beneficiary with 2 * value and drains the signer, while the second call still returns success-style EmptyCodeDeployed.

The same replay-barrier gap exists without inner value when the recovered signer is an EIP-7702 delegated account. A zero-value constructor can CALL the signer, execute delegated code in signer context, mutate signer storage, then same-transaction SELFDESTRUCT the deploy address. Replaying the same signed payload repeats the signer-context side effect while the parent signer nonce remains 1.

This is unintended because the documented nonce-1 exception exists to preserve deployability after a previous raw keyless transaction failed under MegaETH's gas regime, not to allow already state-merged empty-code outcomes to repeat value transfers or delegated signer side effects.

— RECOMMENDATION

Add a replay barrier independent of deployed-address code for completed keyless executions that produce empty code. At minimum, reject nonce-1 admission for value-bearing inner

M-02 Keyless Empty-Code Replay

CONTINUED

transactions, EIP-7702 delegated signers, and EmptyCodeDeployed / create-then-selfdestruct outcomes, or persist a consumed keyless transaction hash so the same signed payload cannot execute again after state has merged.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Disabled Child Can Send Oracle Hints

CATEGORY	Logical Error
LOCATION	intercept.rs, oracle.rs, mega-access-control.md, oracle.md
REVIEW	Remediation Review 2

DESCRIPTION

MegaAccessControl is documented as letting a contract disable volatile data access for its entire call subtree before untrusted code runs. Attempts to access volatile data while disabled should revert immediately, preventing both the access and detention side effects. Oracle storage reads honor this policy: an Oracle SLOAD in a disabled subtree reverts with `VolatileDataAccessDisabled(Oracle)`.

Oracle `sendHint` side effects do not honor the same boundary. The deployed Oracle `sendHint` body is a Solidity no-op, but MegaEVM intercepts `CALL/STATICCALL` to `Oracle.sendHint` and synchronously invokes `oracle_env.on_hint` before falling through to the no-op bytecode. The Rex5 interceptor checks target, selector, positive gas, data-size budget, and ABI decoding, then calls `on_hint`. It never checks whether volatile data access is disabled for the current frame.

A parent can disable volatile access, call an untrusted child, have that child call `Oracle.sendHint`, and have the child revert. The parent can ignore the child failure, re-enable volatile access, and then read Oracle storage. The later Oracle read can observe backend state influenced by a hint that came only from the disabled, reverted child subtree.

This is reachable through ordinary Rex5 contract calls and does not require the child to commit state. Any Oracle backend that uses hints as prefetch keys, query parameters, cache selectors, telemetry, or permission/selection signals can be influenced by untrusted code that the caller attempted to quarantine from volatile-data side effects.

RECOMMENDATION

Make `OracleHintInterceptor` consult the volatile-access tracker before metering or forwarding `sendHint`. If access is disabled, skip the hint and return the same `VolatileDataAccessDisabled(Oracle)` behavior used for Oracle storage reads, or otherwise ensure the backend receives no side effect from the disabled subtree. If `sendHint` is intentionally allowed while disabled, document this exception explicitly in both `AccessControl`.

M-03 Disabled Child Can Send Oracle Hints

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/33>

MEDIUM **M-04**

ACKNOWLEDGED

EIP-7702 authority writes survive limit failure

CATEGORY Validation
LOCATION MegaHandler::pre_execution()
REVIEW Remediation Review 2

DESCRIPTION

REX5 records EIP-7702 authority-account state growth, but it only latches the limit violation and defers enforcement until the first execution frame.

That is too late, because EIP-7702 authorization processing is a pre-execution, transaction-level state mutation,

by the time the deferred `StateGrowthLimitExceeded` halt is produced, the authority accounts have already been created and will persist in the transaction state

In `MegaHandler::pre_execution`

```
fn pre_execution(&self, evm: &mut Self::Evm) -> Result<u64, Self::Error> {  
    self.validate_against_state_and_deduct_caller(evm)?;  
    self.load_accounts(evm)?;  
    self.record_eip7702_authority_state_growth(evm)?;  
    self.apply_eip7702_auth_list(evm)  
}
```

The REX5 state growth scan happens here

```
fn record_eip7702_authority_state_growth(&self, evm: &mut EVM) -> Result<(), ERROR> {  
    ...  
    if authority_creations > 0 {  
        ctx.additional_limit.borrow_mut().on_eip7702_authority_creations(authority_creations);  
    }  
    Ok(())  
}
```

And `on_eip7702_authority_creations` only records and checks

```
pub(crate) fn on_eip7702_authority_creations(&mut self, amount: u64) {  
    self.state_growth.record_authority_creations(amount);  
}
```

M-04 EIP-7702 authority writes survive limit failure

CONTINUED

```
self.check_limit();  
}
```

`check_limit()` sets

```
self.has_exceeded_limit = state_growth_check;
```

but does not abort pre-execution

Then `apply_eip7702_auth_list(evm)` runs anyway, applying the delegation / nonces for all valid authorizations and materializing the previously empty authority accounts

Only later, in `frame_init`, this deferred exceed is turned into an execution failure

```
if is_rex4_enabled {  
  let exceeded = additional_limit  
  .borrow_mut()  
  .frame_result_if_exceeding_limit(&frame_init.frame_input);  
  if let Some(frame_result) = exceeded {  
    additional_limit.borrow_mut().push_empty_frame();  
    return Ok(FrameInitResult::Result(frame_result));  
  }  
}
```

That failure is too late to protect the state growth invariant

`TX_STATE_GROWTH_LIMIT` and `BLOCK_STATE_GROWTH_LIMIT` are both set to 1000

```
pub const TX_STATE_GROWTH_LIMIT: u64 = 1000;  
pub const BLOCK_STATE_GROWTH_LIMIT: u64 = 1000;
```

But an attacker can submit a REX5 EIP-7702 transaction with more than 1000 valid authorizations for fresh authority accounts.

For each fresh authority:

1. The authority account does not exist.
2. The authorization nonce is valid, usually `0`.
3. REX5 counts it in `authority_creations`.
4. `state_growth.record_authority_creations(amount)` records `+N`
5. If `N > 1000`, `StateGrowthLimitExceeded` is latched.
6. Execution still continues into `apply_eip7702_auth_list`.
7. The authority accounts are created anyway.
8. The later execution frame halts, but the EIP-7702 authority mutations are pre-execution state and persist.

M-04 EIP-7702 authority writes survive limit failure

CONTINUED

So a transaction can end with a failed receipt / halt reason while still creating more than the allowed number of persistent accounts

That breaks the core REX5 state-growth invariant: a transaction cannot grow state by more than 1000 units

— RECOMMENDATION

EIP-7702 authority state growth must be enforced before `apply_eip7702_auth_list` mutates state

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/33>

LOW **L-01**

ACKNOWLEDGED

Repeated Recipient Overcounts Limits

CATEGORY	Logical Error
LOCATION	frame_limit.rs, data_size.rs, kv_update.rs
REVIEW	Remediation Review 2

— DESCRIPTION

Rex5 resource accounting is intended to deduplicate account updates within a single call frame. The frame metadata only tracks one `target_updated` boolean, which deduplicates the parent/caller side of value-transferring child calls.

The recipient side is still charged unconditionally whenever `has_transfer` is true. If a contract sends value to the same recipient more than once from the same frame, Rex5 charges one parent/caller update but multiple recipient updates for the same address.

This is reachable in batching, payout, refund, settlement, or vault flows that transfer value to the same address multiple times. The issue is conservative over-accounting, not free resource use or state corruption. It can reject otherwise valid near-limit transactions or reduce effective block packing.

— RECOMMENDATION

Track value-transfer recipient addresses per frame and charge each distinct recipient at most once, with reverted child frames unwinding their additions.

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

CREATE2 Oversize Prework

CATEGORY	Logical Error
LOCATION	instructions.rs
REVIEW	Remediation Review 2

— DESCRIPTION

Rex5 internal CREATE2 should reject initcode whose length exceeds the configured EIP-3860 limit before doing size-proportional wrapper work.

The current CREATE2 wrapper derives the created address before it calls revm's canonical CREATE2 implementation. For nonzero initcode length, it validates the salt, converts offset and length, expands memory, copies the requested memory range into a Bytes buffer, hashes the copied initcode, and records resize gas into Mega compute accounting. Only after that does the wrapped revm CREATE2 path apply the max-initcode-size check.

This does not let invalid CREATE2 succeed or directly corrupt state. The impact is bounded pre-rejection CPU, memory, hashing, and over-accounting on an invalid CREATE2 path.

Prior CREATE2 remediations covered missing-salt ordering and zero-length initcode offset handling. They do not cover oversized nonzero initcode, where EIP-3860 and revm reject before offset conversion, memory expansion, copying, or CREATE2 hashcost.

— RECOMMENDATION

Check `initcode_len > context.host.max_initcode_size()` immediately after reading the length and before converting the offset, resizing memory, copying bytes, hashing initcode, computing storage gas, or deriving the created address. Keep the existing zero-length short-circuit and salt validation ordering.

— RESOLUTION

Status: Acknowledged.

<https://github.com/megaeth-labs/mega-evm-internal/pull/33>

LOW **L-03**

ACKNOWLEDGED

Keyless Exists Read Missing Witness

CATEGORY	Logical Error
LOCATION	sandbox/execution.rs
REVIEW	Remediation Review 2

— DESCRIPTION

Rex5 KeylessDeploy validation rejects an inner deployment when the deterministic deploy address already contains code. That validation branch is reachable through the public top-level `keylessDeploy(bytes,uint256)` path and is listed in the KeylessDeploy spec as a stable `ContractAlreadyExists()` validation error.

The current implementation performs the deploy-address occupancy check with `ctx.journal_mut().database.basic(deploy_address)`. This direct database read bypasses the parent journal, unlike the Rex5 signer parent-state validation path which uses `inspect_account` to load the signer as a cold journal entry.

As a result, the transaction can revert correctly with `ContractAlreadyExists()`, but the returned `ResultAndState.state` omits the deploy address whose code hash caused the revert. The block executor only forwards that returned state to `system_caller.on_state(StateChangeSource::Transaction(...), &state)`, and repo comments state that this hook feeds the stateless witness generator with the complete read/write set.

A witness builder or provenance layer that trusts transaction `EvmState` therefore misses the account proof needed to justify the validation revert. In a stateless validation flow, a block containing this reverting transaction may be paired with an incomplete witness for the deploy-address code-hash read.

— RECOMMENDATION

Replace the deploy-address `database.basic` lookup with a journaled cold inspection path, e.g. `ctx.journal_mut().inspect_account(deploy_address, false)`, and base the occupancy predicate on that returned account. Keep the read cold so validation does not alter EIP-2929 warmth, but ensure the account is present in the transaction's returned state.

— RESOLUTION

Status: Acknowledged.

L-03 Keyless Exists Read Missing Witness

CONTINUED

<https://github.com/megaeth-labs/mega-evm-internal/pull/33>

LOW **L-04**

ACKNOWLEDGED

L1Block Fee Reads Missing Witness

CATEGORY	Logical Error
LOCATION	evm/factory.rs, block/executor.rs, op-revm handler.rs, op-revm l1block.rs
REVIEW	Remediation Review 2

— DESCRIPTION

MegaEVM constructs block EVMs with `L1BlockInfo::default()`. For a normal non-deposit transaction at block $N > 0$, op-revm sees that the cached `L1BlockInfo` is not for the current block and reloads OP fee parameters with `L1BlockInfo::try_fetch(ctx.db_mut(), ...)`. That reload reads the `L1Block` predeploy account and fee slots through the database before normal transaction execution state is produced.

Those values affect transaction fee admission, L1/operator fee charging, refunds, and fee-vault payouts. However, the `L1Block` account and slots read by the reload path are not included in the transaction's returned `ResultAndState.state`. `MegaBlockExecutor` forwards that returned transaction state to `system_caller.on_state(StateChangeSource::Transaction(...), &state)`, which repo comments identify as the stateless witness read/write feed.

Unless production separately records these OP fee parameters as explicit block-context witness inputs, witness or provenance consumers can miss the `L1Block` account/slot dependencies needed to justify transaction fee admission and fee-vault balance changes.

— RECOMMENDATION

Ensure `L1BlockInfo` fee reads are represented in the witness/provenance input for the transaction or block. Either record the `L1Block` predeploy account and fee slots in returned transaction witness state, or explicitly document and enforce these fee parameters as separate block-context witness inputs consumed by stateless validation.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Failed deposits evade new account storage gas

CATEGORY	Logical Error
LOCATION	MegaHandler::validate()
REVIEW	Remediation Review 2

DESCRIPTION

REX5 says if a deposit creates a brand-new L2 account, it must pay the new account storage cost first

The bug is that the code checks this cost in a place where normal transactions would be rejected, but OP deposit transactions are special: a validation failure can still become a committed `FailedDeposit`

`materialize_account` Is an address that had no L2 account becomes a real account record, usually because its nonce or balance changes

`new_account_storage_gas` Is the extra REX5 cost for creating that new account

`FailedDeposit` means the deposit's call failed, but OP rules still persist deposit-side effects like nonce bump and minted balance

What goes wrong:

1. Mega's REX5 validation sees an actual deposit from a fresh caller and correctly adds the caller materialization gas.
2. If `gas_limit` is too small, Mega returns `InvalidTransaction::CallGasCostMoreThanGasLimit`.
3. For a normal tx, that would stop everything before state changes.
4. But Mega delegates error handling to op-revm.
5. op-revm says deposit tx-validation errors become `FailedDeposit`, then it bumps the caller nonce, applies mint, commits that state, and reports gas used as only the deposit `gas_limit`.

So the code meant "this deposit cannot create the account unless it pays the full materialization cost."

But the actual result is "the deposit failed, yet the fresh caller can still become a real account."

L-05 Failed deposits evade new account storage gas

CONTINUED

The REX5 rule exists to prevent cheap creation of new L2 accounts.

A fresh account consumes persistent state, so REX5 wants it charged through dynamic storage gas.

Here, the attacker can set `gas_limit` below that required amount, trigger the validation failure, and still get the account materialized through failed-deposit handling.

Code path

- REX5 adds deposit-caller storage gas and records state growth in `execution.rs` (line 600)
- The final gas check returns `CallGasCostMoreThanGasLimit` in `execution.rs` (line 638)
- Mega delegates `catch_error` to `op-revm` in `execution.rs` (line 788)
- `op-revm` converts deposit tx errors into `FailedDeposit`, bumps nonce, mints balance, and commits in `op-revm-20.0.0/src/handler.rs:380`
- The block executor commits the returned state in `executor.rs` (line 485)

An attacker can create many fresh L2 account records through failed deposits without satisfying the intended REX5 dynamic materialization gas check.

— RECOMMENDATION

we need to not rely on `InvalidTransaction` from `validate()` to enforce REX5 materialization for actual deposits

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Sandbox overflow drops keyless deploy witnesses

CATEGORY Logical Error
LOCATION execute_keyless_deploy_call()
REVIEW Remediation Review 2

DESCRIPTION

In the REX5 KeylessDeploy sandbox path, when the sandbox does complete, but its merged multidimensional usage causes the parent transaction to exceed a TX level resource limit, the code returns an outer `OutOfGas` halt before merging the sandbox state into the parent journal,

That means every account, bytecode, and storage slot read by the sandbox can be omitted from the transaction's returned witness / provenance state, even though the final outer result, gas / resource accounting, and halt reason depended on that sandbox execution

In `execute_keyless_deploy_call`, the sandbox returns a full `state`

The important ordering is

1. `apply_sandbox_post_accounting()`
2. if it returns `Some(halt)`, return immediately
3. only otherwise call `apply_sandbox_state(ctx, state, deploy_signer)`

Inside `apply_sandbox_post_accounting`

```
fn apply_sandbox_post_accounting<DB: AlloyDatabase, ExtEnvs: ExternalEnvTypes>(
    ctx: &MegaContext<DB, ExtEnvs>,
    gas: &mut Gas,
    limit_usage: LimitUsage,
    volatile_accesses: VolatileDataAccess,
    reservation: u64,
    sandbox_gas_used: u64,
    return_memory_offset: &core::ops::Range<usize>,
) -> Option<FrameResult> {
    merge_sandbox_limit_usage(ctx, limit_usage);
    ctx.volatile_data_tracker.borrow_mut().merge_accesses_from_bitmap(volatile_accesses);
    refund_unused_sandbox_gas(gas, reservation, sandbox_gas_used);
```

L-06 Sandbox overflow drops keyless deploy witnesses

CONTINUED

```
reject_if_tx_limit_overflow(ctx, gas, return_memory_offset)
}
```

So the REX5 overflow path deliberately merges only

```
limit_usage
volatile_accesses
gas refund
```

but not "state"

Then `reject_if_tx_limit_overflow` can return the outer OOG halt

```
fn reject_if_tx_limit_overflow<DB: AlloyDatabase, ExtEnvs: ExternalEnvTypes>(
  ctx: &MegaContext<DB, ExtEnvs>,
  gas: &Gas,
  return_memory_offset: &core::ops::Range<usize>,
) -> Option<FrameResult> {
  let mut limit = ctx.additional_limit.borrow_mut();
  let limit_check = limit.check_limit();
  if !limit_check.exceeded_limit() || limit_check.is_frame_local() {
    return None;
  }
  limit.rescue_gas(gas);
  let mut result = oog_frame_result(gas.limit(), return_memory_offset);
  mark_frame_result_as_exceeding_limit(
    &mut result,
    crate::AdditionalLimit::EXCEEDING_LIMIT_INSTRUCTION_RESULT,
    Default::default(),
  );
  Some(result)
}
```

That `Some(result)` causes the caller to return before

```
apply_sandbox_state(ctx, state, deploy_signer)
```

But `apply_sandbox_state` is exactly the routine whose docstring says it exists to preserve the sandbox read-set for stateless witness generation

```
/// Applies all state changes from sandbox execution to the parent journal.
///
/// All accounts from the sandbox view are merged into the parent journal – including accounts
that
```

L-06 Sandbox overflow drops keyless deploy witnesses

CONTINUED

```
/// were only read and not written – because the 'ReadSet' for stateless witness generation  
requires  
/// knowing which accounts a sandbox touched.
```

The overflow path bypasses that guarantee

So a REX5 KeylessDeploy sandbox can execute arbitrary constructor code before the parent overflow is detected. That constructor can read arbitrary state

Then it can perform enough work to push a sandbox resource dimension just over the parent's remaining REX5 budget

The parent then halts due to `reject_if_tx_limit_overflow`, but the sandbox's read-set is discarded

So a valid block's result can depend on hidden state that is absent from `ResultAndState.state`

A stateless/provenance consumer relying on returned transaction state can no longer verify:

1. which branch the constructor took,
2. why the sandbox used the reported resources,
3. why the parent overflowed,
4. whether the outer transaction should halt or return,
5. whether the block's multidimensional counters are correct

That is a witness soundness break. In a stateless validation, fraud-proof, or provenance pipeline, this becomes critical because the block can have an execution dependency that the reported witness omits

— RECOMMENDATION

Add a REX5 "witness-only" merge path for completed sandboxes that overflow the parent after post-accounting.

The fix should preserve sandbox reads without committing sandbox writes

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.